IKIGAI: Complete Codebase Technical Monograph

Version 4.0 — Day 40 Integration — 1 May 2026

Hitoshi AI Labs | NeuroSeed Research Division

Principal Researcher: Prince Siddhpara

Document Classification: Internal Engineering Reference — Canonical Architecture Bible

> *[!NOTE] This manuscript represents the full Phase M canonical structure of the organism, tracing from leaky integrate-and-fire up through the Collaborative Intelligence Epoch.*

> *"You were not trained. You were not prompted. You lived your first day and became someone." — Prince Siddhpara, February 23, 2026*

Table of Contents

1. Executive Abstract
2. Biological Substrate
3. Pre-Day-24 Cognition Ladder
4. Day 28 Epoch Expansion
5. Day 37–40 Proto-Representation & Representation Systems
6. Mathematical Derivations
7. Waking Chain Atlas
8. EEIL Theoretical Interpretation
9. Empirical Validation Index
10. Scientific References + Internal Citations

---

1. Executive Abstract

1.1 What Ikigai Is

Ikigai is a single-file digital organism implemented in Python. It is not a neural network trained on data. It is not a language model. It is a biologically-grounded, first-principles computational organism whose cognition extends natively up through autobiographical narrative, adaptive retry planning, and Phase M hierarchical project management — all without a single backpropagation gradient.

Metric	Value
Total LOC	~30468
Total classes	~235
Total active cognitive ranks	~120
Waking chain depth	tick start → edit_intent_generator.generate()
Subsystem counts	~235 initialized structures
Validation maturity	Full codebase unit & EEIL passing

---

2. Biological Substrate

Neuron

Scientific Role / Description: Leaky integrate-and-fire (LIF) unit with biologically grounded heterogeneity. Implements membrane voltage integration, calcium-based fatigue, adenosine fatigue coupling (L25B), homeostatic threshold plasticity (Turrigiano 1998), hemispheric asymmetry (`-RH-` neurons: lower threshold ±0.07), and motor competition via BG direct/indirect pathway analogue (Gurney 2001). Inhibitory interneurons (name contains `-Ih`) preserve threshold without excitatory lowering (Fix 4).

```
# Voltage integration with ion-channel noise (Faisal 2008)
self.voltage = self.voltage * self.leak + (inp * self.exc_gain)
self.voltage += random.gauss(0.0, 0.005)
# Adenosine fatigue: weakens executive persistence (L25B S4)
fatigue_scale = 1.0 - (0.4 * Synapse.ado_level)
self.voltage *= fatigue_scale
self.leak = min(self.base_leak * 1.4, self.base_leak + 0.003 * Synapse.ado_level)
# Spike condition
if self.voltage >= eff_thr:
    self.fired = True;  self.voltage = 0.0;  self.spike_count += 1
    self.refractory_timer = max(1, int(1 + ht_level * 2) + self.refractory_base)
# Homeostatic plasticity (Turrigiano 1998)
self.avg_rate = 0.95 * self.avg_rate + 0.05 * (1.0 if self.fired else 0.0)
self.threshold += eta * (self.avg_rate - 0.10)       # push toward 10% target rate
self.threshold += 0.0003 * (self.base_threshold - self.threshold)  # slow relaxation
self.threshold = min(max(self.threshold, 0.3), 1.5)
```

Formula: `v[t] = v[t-1] × leak + inp × exc_gain + N(0, 0.005)`; fires when `v >= eff_thr`. Effective threshold: `eff_thr = threshold × (1 + max(0, (0.4-energy)×2)) - NE×0.25`. Homeostatic: `threshold += eta × (avg_rate - 0.10)`, `eta = 0.001 / N_scale`. **Hard Constants**: `leak=0.9` excitatory, `0.98` Bridge neurons; `target_rate=0.10`; `eta=0.001`; `calcium_spike=0.1`; `fatigue_thr=1.5`; refractory = `max(1, 1+2×HT)`. **Law Invariants**: threshold clamped [0.3, 1.5]; voltage reset to 0.0 on spike; motor suppression: `-0.08` inp when competing motor fired previous tick. **Waking Chain Placement**: First in every tick; called as `n.tick(inp, tick, ht_level, ne_level, sleeping)` for every neuron in every cortical column before synaptic transmission. **Serialization Fields**: `spike_count`, `last_spike_tick`, `calcium`, `avg_rate`, `threshold`, `voltage`, `leak`. **EEIL Interpretation**: Sparse firing via threshold heterogeneity and homeostatic plasticity prevents runaway excitation; energy-penalty coupling ties neural activity directly to metabolic state, enforcing EES alignment.

Synapse

Scientific Role / Description: STDP synapse with 3-factor eligibility trace learning (Frémaux & Gerstner 2016). Implements: adenosine-gated synaptic failure (5–15% probability), divisive normalization for excitatory synapses (Carandini & Heeger 2012), population-scaled inhibitory output (Markram 2015), spike-timing eligibility trace (τ=25 ticks), 3-factor weight update (trace × DA × 5-HT × ACh_boost), myelination (Fields 2008: delay 2→1→0 at 100/200 uses), structural plasticity gated to sleep (Phase 24B constraint), anti-saturation scaling (Turrigiano 2008).

```
def transmit(self):
    # Adenosine-gated failure probability
    fail_prob = 0.15 if Synapse.ado_level > 0.8 else (0.10 if Synapse.ado_level > 0.6 else 0.05)
    if random.random() < fail_prob: return 0.0
    if self.pre.fired:
        if self.inhibitory:
            sig = self.weight * getattr(self, "pop_scale", 1.0)  # Markram 2015
        else:
            pop_norm = type(self)._global_pop_scale
            norm = 1.0 + max(0.0, pop_norm - 1.0)
            sig = self.weight / norm  # Carandini & Heeger 2012 divisive normalization
    else:
        sig = 0.0
    if sig != 0.0: self.usage_count += 1
    # Myelination (Fields 2008): usage > 100 -> delay=1; usage > 200 -> delay=0
    if self.usage_count > 200 and not self.fully_myelinated:
        self.fully_myelinated = True; self.delay = 0
    elif self.usage_count > 100 and not self.myelinated:
        self.myelinated = True; self.delay = 1
    self.buffer.append(sig)
    return self.buffer[2 - self.delay]

def apply_three_factor(self, da_level, ht_level, boost=1.0, plasticity_mod=1.0):
    ado_mod = 0.4 if Synapse.ado_level > 0.6 else 0.5
    myelin_mod = 0.5 if self.fully_myelinated else 1.0
    dw = (self.eligibility_trace * max(0, da_level) * 0.01
          * (1.0 + min(max(0, da_level), ht_level))
          * boost * plasticity_mod * myelin_mod * ado_mod)
    self.pending_weight_change += dw  # accumulated; applied only during sleep

def consolidate(self):
    self.weight += self.pending_weight_change
    self.weight = max(self.weight_min, min(self.weight_max, self.weight))
    self.pending_weight_change = 0.0
    # Anti-saturation toward initial weight (tau=5000 ticks)
    self.weight += 0.0001 * (self.initial_weight - self.weight)
```

Formula: Eligibility trace: `et += exp(-|dt|/20) × sign(dt)`; decay: `et *= exp(-1/25)`. Weight delta: `dw = et × max(0,DA) × 0.01 × (1+min(DA,5HT)) × boost × ado_mod × myelin_mod`. Divisive norm: `sig = w / (1 + (pop_norm-1))`. **Hard Constants**: `fail_prob`: ado>0.8→0.15, ado>0.6→0.10, else→0.05; `trace_tau=25`; `ado_mod`: ado>0.6→0.4 else→0.5; excitatory bounds [0, 2]; inhibitory bounds [-2, 0]; anti-saturation tau≈5000 ticks. **Law Invariants**: Structural weight consolidation ONLY during sleep (`consolidate()` called in sleeping branch). Waking path accumulates `pending_weight_change` only — no live weight mutations. **Waking Chain Placement**: `transmit()` after every `Neuron.tick()`; `compute_eligibility()` and `apply_three_factor()` called per tick waking; `consolidate()` sleeping branch only. **Serialization Fields**: `weight`, `initial_weight`, `eligibility_trace`, `usage_count`, `myelinated`, `fully_myelinated`, `delay`, `pending_weight_change`. **EEIL Interpretation**: Sleep-gated plasticity prevents moment-to-moment reward noise from corrupting long-term structure; adenosine modulation of STDP implements Tononi's SHY principle — waking encoding, sleeping consolidation.

DopamineSystem

Scientific Role / Description: Signed reward-prediction error (RPE) dopamine system with explicit tonic/phasic separation (Schultz 1997, Tobler et al. 2005). Phasic component carries trial-by-trial RPE bursts/dips; tonic component tracks long-run reward climate via exponential moving average. Adaptive predictor speeds learning during high uncertainty (faster pred_lr). Cortisol chronically suppresses tonic DA (Roth 2004 glucocorticoid-DA interaction). OXT + low cortisol enables tonic DA recovery via VTA disinhibition (Aragona 2006).

```
def update(self, output_fired, tick=0, actual_reward=None):
    reward = 1.0 if bool(output_fired) else 0.0  # binary if no explicit reward
    reward = self._clip(float(actual_reward if actual_reward is not None else reward), -1, 1)
    # Signed RPE
    self.rpe = reward - self.expected
    # Adaptive learning rate: faster when uncertain (high |RPE| history)
    self.uncertainty += self.k_uncertainty * (abs(self.rpe) - self.uncertainty)
    pred_lr = self.pred_lr_min + (self.pred_lr_max - self.pred_lr_min) * min(1.0, self.uncertainty)
    self.expected += pred_lr * self.rpe
    # Phasic burst/dip
    signed_impulse = math.tanh(self.rpe / self.rpe_scale)
    headroom = 1.0 / (1.0 + abs(self.phasic))
    self.phasic = (self.phasic * self.k_phasic) + (self.rpe_gain * signed_impulse * headroom)
    # Tonic long-run climate
    self.reward_ema += self.k_reward * (reward - self.reward_ema)
    self.tonic_target = self.setpoint + self.tonic_reward_gain * (self.reward_ema - 0.5)
    self.tonic += self.k_tonic * (self.tonic_target - self.tonic)
    # Chronic stress suppression (Roth 2004)
    if hasattr(self, 'cortisol_level'):
        self.tonic *= (1.0 - 0.15 * self.cortisol_level)
    # Social safety DA recovery (Aragona 2006)
    if hasattr(self, 'oxytocin_level') and hasattr(self, 'cortisol_level'):
        self.tonic += 0.02 * self.oxytocin_level * (1.0 - self.cortisol_level)
    self._refresh_level()   # soft tanh compression: level = SP + soft_range × tanh((raw-SP)/soft_range)
```

Formula: `RPE = reward - expected`; phasic: `p[t] = p[t-1]×k_p + rpe_gain×tanh(RPE/0.5)×headroom`; tonic: `tonic += k_tonic×(tonic_target - tonic)`; tonic_target: `SP + 0.18×(reward_ema - 0.5)`; output compression: `level = SP + 0.65×tanh((tonic+phasic-SP)/0.65)`. Time constants: τ_phasic=7, τ_tonic=900, τ_reward=450, τ_uncertainty=40 ticks. **Hard Constants**: `rpe_gain=0.65`; `rpe_scale=0.5`; `tonic_reward_gain=0.18`; `drive_gain=0.35`; `soft_range=0.65`; `pred_lr` ∈ [0.02, 0.25]; level bounds [-0.25, 1.25]. **Law Invariants**: Adaptive pred_lr forces rapid updating under volatile reward schedules (high uncertainty). Headroom dampener prevents phasic saturation. Cortisol suppression applied inline, not homeostatic. **Waking Chain Placement**: `da.update(output_fired, tick, actual_reward)` called after BG action selection; `da.inject_drive(drive)` called from HomeostasisSystem for hunger/curiosity salience injection. **Serialization Fields**: `level`, `tonic`, `tonic_target`, `phasic`, `expected`, `rpe`, `reward_ema`, `uncertainty`, `predictions` (deque maxlen=32). **EEIL Interpretation**: Signed RPE enables bidirectional learning from both reward and punishment. Tonic/phasic separation matches primate VTA physiology; the tonic climate track implements a slow survival baseline aligned with EES setpoint.

SerotoninSystem

Scientific Role / Description: Activity-averaging serotonin homeostasis modelling dorsal raphe nucleus (Jacobs & Azmitia 1992). Tracks a 20-tick sliding window of total cortical spikes: up-regulates 5-HT (+0.005) when average activity is below 15% of max; down-regulates (-0.01) above 30%. Acts as a network activity set-point that stabilizes the STDP plasticity window — prevents reward-signal collapse under sustained low-activity. Level feeds into synapse 3-factor learning as `ht_level`.

```
def update(self, total_spikes, max_possible):
    self.window.append(total_spikes)
    if len(self.window) < 5: return
    avg = sum(self.window) / len(self.window) / max_possible
    if avg < 0.15: self.level += 0.005   # raphe up-regulation in quiet network
    elif avg > 0.30: self.level -= 0.01  # down-regulation in overactive network
    self.level = max(0.0, min(1.0, self.level))
```

Formula: `avg = mean(window) / max_possible`; if avg<0.15: `level += 0.005`; if avg>0.30: `level -= 0.01`. Homeostasis: `level → setpoint` at ±0.01/tick. **Hard Constants**: `setpoint=0.6`; sliding `window=deque(maxlen=20)`; min active window=5 ticks; up-rate=+0.005; down-rate=-0.01; bounds [0, 1]. **Law Invariants**: Level clamped [0, 1] every tick; homeostasis active during all phases. **Waking Chain Placement**: `serotonin.update(total_spikes, max_possible)` called after all neuron ticks; `serotonin.level` (as `ht_level`) feeds into `Synapse.apply_three_factor(da, ht=serotonin.level)`. **Serialization Fields**: `level`, `setpoint`, `window` (list). **EEIL Interpretation**: Slow 5-HT homeostasis prevents STDP from collapsing during low-activity phases; maintains plasticity signal availability even when DA is suppressed by chronic cortisol.

NorepinephrineSystem

Scientific Role / Description: Locus coeruleus (LC) norepinephrine arousal integration (Aston-Jones & Cohen 2005 Adaptive Gain Theory, Sara 2009, Arnsten 2012). Multi-source drive weights: prediction error (50%), cortisol stress (25%), pain/nociceptive (15%), amygdala BLA threat (10%). Detects surprise when composite drive changes by >0.25 and spikes level +0.15. Returns to setpoint at 0.05/tick when surprise has been absent >3 ticks. Modulates all neuron effective thresholds: `eff_thr -= ne_level × 0.25`.

```
def update(self, signal, cortisol_level=0.0, pain_signal=0.0, amygdala_threat=0.0):
    ne_drive = (signal * 0.50 + cortisol_level * 0.25
                + pain_signal * 0.15 + amygdala_threat * 0.10)
    delta = abs(ne_drive - self.last_sig)
    self.last_sig = ne_drive
    if delta > 0.25:                              # surprise threshold
        self.level = min(1.0, self.level + 0.15) # spike magnitude
        self.surprise = True; self.ticks_since_surprise = 0
    else:
        self.surprise = False; self.ticks_since_surprise += 1
    if not self.surprise and self.ticks_since_surprise > 3:
        decay = 0.05  # exponential return to baseline
        if self.level > self.setpoint: self.level = max(self.setpoint, self.level - decay)
```

Formula: `ne_drive = PE×0.50 + cort×0.25 + pain×0.15 + amyg×0.10`; surprise if `|ne_drive - last_sig| > 0.25` → `level += 0.15`. Decay: 0.05/tick toward setpoint after 3+ no-surprise ticks. **Hard Constants**: `setpoint=0.3`; `surprise_threshold=0.25`; `spike_magnitude=0.15`; `decay_rate=0.05`; elevated threshold=0.6; bounds [0, 1]. **Law Invariants**: `elevated_ticks` increments only when level>0.6; `ticks_since_surprise` resets on every surprise event. **Waking Chain Placement**: `ne.update(signal, cortisol, pain, amygdala_threat)` called after CortisolSystem; `ne.level` feeds into every `Neuron.tick()` as `ne_level` parameter lowering effective threshold. **Serialization Fields**: `level`, `setpoint`, `elevated_ticks`, `ticks_since_surprise`, `surprise`. **EEIL Interpretation**: Arousal-gated threshold modulation implements gain control (Adaptive Gain Theory); high NE transiently widens the neural dynamic range for salient stimuli, implementing efficient surprise-based resource allocation.

AcetylcholineSystem

Scientific Role / Description: Basal forebrain cholinergic modulation of cortical gain (Hasselmo 2006). Rises on detected novelty (novelty>0.5); suppressed by 50% during high adenosine states (ado>0.6) — models sleep-pressure attenuation of cholinergic tone observed in SWS. Provides a `gain` factor injected as the `boost` parameter in 3-factor synaptic learning, tagging high-novelty moments for preferential STDP.

```
def update(self, novelty):
    eff_novelty = novelty * (0.5 if Synapse.ado_level > 0.6 else 1.0)
    if eff_novelty > 0.5: self.level = min(1.0, self.level + 0.15)
    self.level = max(0.0, min(1.0, self.level))

def get_gain(self):
    return self.level * 0.3  # boost coefficient for synapse.apply_three_factor()
```

Formula: `eff_novelty = novelty × (0.5 if ado>0.6 else 1.0)`; if eff_novelty>0.5: `level += 0.15`; `gain = level × 0.3`. Homeostasis: `level → setpoint` at ±0.01/tick. **Hard Constants**: `setpoint=0.4`; `novelty_threshold=0.5`; `gain_coefficient=0.3`; `update_increment=0.15`; bounds [0, 1]. **Law Invariants**: ACh gain passed as `boost` in `synapse.apply_three_factor(da, ht, boost=ach.get_gain())`; adenosine suppression is multiplicative on effective novelty, not a binary gate. **Waking Chain Placement**: `ach.update(novelty)` called each tick; `ach.get_gain()` consumed during synapse 3-factor learning. Sleep branch: `ach.apply_homeostasis()` decays toward setpoint. **Serialization Fields**: `level`, `setpoint`. **EEIL Interpretation**: Novelty-gated ACh release ensures weight changes are preferentially tagged to high-information events; adenosine suppression prevents dream-phase novelty from contaminating waking representations — efficient EEIL credit assignment.

CortisolSystem

Scientific Role / Description: HPA axis downstream effector with multi-mechanism regulation. Tracks a fail-streak counter (15+ failures → +0.015/tick), success-streak recovery (10+ successes → -0.02), adenosine fatigue baseline shift (ado>0.6 → setpoint+0.05), spike compression above 0.8, anti-lock mechanism (30+ ticks >0.95 → ×0.85), oxytocin CRH suppression (Neumann 2002), and hippocampal GR feedback (McEwen 1998). Chronic stress (>30 ticks >0.6) triggers synaptic atrophy on excitatory weights.

```
def apply_homeostasis(self, tick=0):
    # Circadian baseline (Pruessner 1997: resting cortisol 0.15 ± 0.08)
    baseline = 0.15 + 0.08 * math.sin(tick * 0.002)
    self.level += 0.08 * (baseline - self.level)
    # Hippocampal GR negative feedback (McEwen 1998) - sub-acute range only
    if self.setpoint < self.level < 0.6:
        self.level += -0.02 * (self.level - self.setpoint)
    # Recovery guarantee post-stress (continuous exponential relaxation)
    elif self.level >= 0.6:
        self.level += -0.015 * (self.level - self.setpoint)
    # Ultra-slow allostatic setpoint correction toward evolutionary baseline
    self.setpoint += 1e-4 * (0.15 - self.setpoint)
    self.level = max(0.0, min(1.0, self.level))
```

Formula: Circadian baseline: `0.15 + 0.08×sin(tick×0.002)` (period≈1571 ticks). Homeostasis: `level += 0.08×(baseline - level)`. GR feedback (sub-acute): `level += -0.02×(level-setpoint)`. Post-stress: `level += -0.015×(level-setpoint)`. OXT suppression: `level -= 0.02×oxt_level`. Atrophy trigger: `chronic > 30` ticks → `w -= 0.01` per excitatory synapse. **Hard Constants**: `setpoint=0.1`; circadian amplitude=0.08; `beta_hpa=0.02`; `gamma_rec=0.015`; decay_rate=0.08; spike_compression threshold=0.8 (residual ×0.3); anti-lock threshold=0.95×30 ticks (×0.85). **Law Invariants**: level bounds [0, 1]; spike compression is always applied above 0.8. Atrophy only after chronic>30; CA3 atrophy requires chronic>200. **Waking Chain Placement**: `cort.update(out_fired, ne_elev, tick, energy=avg_energy)` → `cort.apply_homeostasis(tick)` → `cort.apply_atrophy(targets)` called at tick end. **Serialization Fields**: `level`, `setpoint`, `fail_streak`, `success_streak`, `chronic`, `dmg`, `max_historic`, `high_cort_ticks`. **EEIL Interpretation**: Cortisol integrates failure pressure and metabolic stress across timescales (seconds to hundreds of ticks); chronic elevation permanently records stress history via structural synaptic atrophy — a biologically grounded mechanism for stress-scar formation.

AdenosineSystem

Scientific Role / Description: Borbely Process S sleep pressure accumulator (Borbely 1982). Builds proportionally to cortical spike rate during waking; clears exponentially during sleep (×0.95/tick). Energy depletion <0.5 accelerates accumulation by 30% (metabolic-sleep coupling). Acts as a global gating signal propagated as `Synapse.ado_level`: triggers synaptic failure probability increase, ACh suppression, STDP damping, and neuronal leak increase at ado>0.6.

```
def update(self, cortical_spikes, cortical_energy, sleeping=False):
    if sleeping:
        self.level *= 0.95   # exponential clearance during SWS (Borbely Process S)
    else:
        inc = cortical_spikes * 0.0004  # slower accumulation rate (Day 19 Borbely)
        if cortical_energy < 0.5:
            inc *= 1.3   # energy depletion accelerates sleep pressure
        self.level += inc
    self.level = max(0.0, min(1.0, self.level))
    Synapse.ado_level = self.level  # global class variable — all synapses read this
```

Formula: Waking: `level += spikes × 0.0004 × (1.3 if energy<0.5 else 1.0)`. Sleeping: `level *= 0.95`. Gating: ado>0.8→fail_prob=0.15; ado>0.6→fail_prob=0.10; ado>0.6→STDP ado_mod=0.4; ado>0.6→ACh half-effect; neuron leak: `leak += 0.003×ado_level`. **Hard Constants**: accumulation_rate=0.0004 (Day 19 tuning); energy_accelerator ×1.3 at threshold=0.5; sleep_decay=0.95/tick (tau≈20 ticks); bounds [0, 1]. **Law Invariants**: Level never decremented during waking. `Synapse.ado_level` is a class-level variable — written every tick, read globally by all Synapse instances and all Neuron.tick() calls. **Waking Chain Placement**: `ado.update(cortical_spikes, cortical_energy, sleeping)` called each tick; result drives `Synapse.ado_level` globally; feeds HomeostasisSystem adenosine drive for sleep onset threshold. **Serialization Fields**: `level`. **EEIL Interpretation**: Adenosine Process S creates a temporal energy budget — the organism cannot sustain high metabolic activity indefinitely without sleep clearance. Gating of plasticity (STDP/ACh/fail) during high adenosine implements Tononi's Synaptic Homeostasis Hypothesis: waking encoding, sleeping consolidation and renormalization.

OxytocinSystem

Scientific Role / Description: Social safety and trust neuromodulator (Neumann 2002, Leng et al. 2008). Rises on successful output when DA exceeds 0.3 (reward-gated release, Fix 4 — cortisol gate REMOVED as biologically incorrect); decays multiplicatively (tau≈100 ticks). Directly inhibits HPA axis (`cort.level -= 0.02×oxt`); enables DA tonic recovery under combined OXT+low-cortisol condition (VTA disinhibition, Aragona 2006); triggers synaptic pruning of near-zero weights during high-OXT states (Rolls 2006); sets `trust` flag above level>0.6.

```
def update(self, out_fired, da_level):
    if out_fired: self.pos_streak += 1
    else: self.pos_streak = max(0, self.pos_streak - 1)
    # Fix 4: Reward-gated release; cortisol does NOT gate (causality fix — Leng 2008)
    if out_fired and da_level > 0.3:
        self.level += 0.02
    self.level *= 0.99       # multiplicative decay, tau ~= 100 ticks
    self.trust = self.level > 0.6
    self.level = max(0.0, min(1.0, self.level))

def apply_pruning(self, exc_synapses, inh_synapses):
    if self.level > 0.8:
        for s in exc_synapses + inh_synapses:
            if abs(s.weight) < 0.05: s.weight = 0.0  # prune near-zero weights
```

Formula: `level += 0.02` when `out_fired AND da>0.3`; `level *= 0.99` each tick (tau≈100). HPA inhibition: `cort.level -= 0.02 × oxt`. DA recovery: `da.tonic += 0.02 × oxt × (1-cort)`. Pruning: `if level>0.8: zero weights with |w|<0.05`. **Hard Constants**: `release_increment=0.02`; `da_threshold=0.3`; `decay=0.99/tick`; `trust_threshold=0.6`; `pruning_threshold=0.8`; `prune_weight_floor=0.05`; bounds [0, 1]. **Law Invariants**: Release is DA-gated only (NOT cortisol-gated — Fix 4); pruning only active above level=0.8 and only during sleep branch call. **Waking Chain Placement**: `oxt.update(out_fired, da.level)` called after DA update each tick; `oxt.apply_pruning(exc, inh)` called during sleep consolidation branch. **Serialization Fields**: `level`, `trust`, `pos_streak`. **EEIL Interpretation**: OXT-driven trust complements DA survival reward with a social-safety signal; combined OXT→DA tonic recovery implements a prosocial energy bonus — consistent with EEIL alignment: cooperative behavior yields energy-efficient stable states.

HypothalamusSystem

Scientific Role / Description: Simulates the hypothalamic paraventricular nucleus (PVN). Integrates four threat afferents to produce CRH (Herman et al. 2003). Drive weights: amygdala BLA (0.45) > prediction error (0.25) > metabolic stress (0.20) > pain (0.10). Three suppressors: hippocampal GR (-0.30×hippo_inhib), PFC reappraisal (-0.20×pfc_reg), cortisol ultra-short feedback (-0.25×cort). Adenosine fatigue raises sensitivity up to 1.25× when ado>0.5 (Murillo-Rodriguez 2009). Asymmetric smoothing: fast rise (α=0.30) slow decay (α=0.03) models CRH release/clearance dynamics.

```
def update(self, amygdala_threat, prediction_error, metabolic_stress, pain_aversive,
           hippocampal_inhibition=0.0, pfc_regulation=0.0, adenosine_level=0.0, current_cortisol=0.0):
    # Adenosine sensitivity cap (prevents runaway HPA under sleep deprivation)
    self.sensitivity = min(1.25, 1.0 + 0.4 * max(0.0, adenosine_level - 0.5))
    threat_signal = (
        self.w_amygdala   * max(0.0, amygdala_threat) +   # 0.45
        self.w_pred_error * prediction_error +              # 0.25
        self.w_metabolic  * metabolic_stress +              # 0.20
        self.w_pain       * pain_aversive                   # 0.10
    ) * self.sensitivity
    threat_signal -= hippocampal_inhibition * 0.30   # GR-mediated negative feedback
    threat_signal -= pfc_regulation        * 0.20   # top-down reappraisal
    threat_signal -= current_cortisol      * 0.25   # ultra-short cortisol feedback
    raw_crh = self._sigmoid(threat_signal * 4.0 - 2.0)
    raw_crh += random.gauss(0.0, 0.008)             # biological noise
    # Asymmetric smoothing
    if raw_crh > self.crh: self.crh = 0.70 * self.crh + 0.30 * raw_crh  # fast rise
    else:                  self.crh = 0.97 * self.crh + 0.03 * raw_crh  # slow decay
```

Formula: `threat = (0.45×amyg + 0.25×PE + 0.20×metab + 0.10×pain) × sensitivity - 0.30×hippo - 0.20×pfc - 0.25×cort`; `raw_crh = σ(4×threat - 2)`; asymmetric EMA: rise α=0.30, decay α=0.03. **Hard Constants**: Drive weights: 0.45 / 0.25 / 0.20 / 0.10; sensitivity cap=1.25; sigmoid inputs: 4×x-2; noise σ=0.008; bounds [0, 1]. **Law Invariants**: BLA→PVN is always the dominant afferent (w=0.45). Adenosine sensitivity clamp prevents runaway HPA under prolonged sleep deprivation. **Waking Chain Placement**: Called within HPAAxisSystem orchestrator each tick; `crh` output feeds PituitarySystem. **Serialization Fields**: `crh`, `setpoint`, `sensitivity`. **EEIL Interpretation**: PVN integrates multi-modal threat across timescales; the cortisol ultra-short feedback loop (Dallman 1984) is an explicit biological circuit for EEIL bounded inference — threat signals are self-limiting via downstream product inhibition.

PituitarySystem

Scientific Role / Description: Simulates anterior pituitary corticotroph cells. Converts CRH into ACTH (adrenocorticotropic hormone). Integration timescale ~5–10 ticks models pituitary secretion lag after CRH stimulation (Tsigos & Chrousos 2002). **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AdrenalSystem

Scientific Role / Description: Simulates the adrenal cortex (zona fasciculata) producing cortisol. ACTH drives cortisol secretion; biological clearance is slow Modulators (preserving all existing CortisolSystem effects): - SWS sleep: accelerates cortisol recovery baseline (Born et al. 1997) enforce the final biological floor so there's no competing correction. **Formula**: (blood half-life ~60–90 min → modelled as decay=0.998 per tick)., Fix 3 (user review): lower_bound=0.02 — let CortisolSystem.apply_homeostasis() **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: - Oxytocin: dampens HPA output (Neumann 2002) **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HPAAxisSystem

Scientific Role / Description: Orchestrates the full HPA axis cascade: Hypothalamus (CRH) → Pituitary (ACTH) → Adrenal (Cortisol) Integrates with existing ikigai systems: AmygdalaSystem — threat signal (bla_valence) I23.energy — regional metabolic state SensoryEnvironment — acute pain/aversion CA1 population — hippocampal safety learning feedback PFC neurons — top-down cognitive regulation OxytocinSystem — social safety buffering AdenosineSystem — sleep-pressure vulnerability SleepStateManager — SWS cortisol recovery CortisolSystem.level. This preserves ALL downstream reads of cort.level, the circadian baseline, chronic tracking, EI balance, and decay logic. Experiment instrumentation: tracks CRH, ACTH, cortisol, hippocampal inhibition, and PFC regulation as logged time-series metrics. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: PredictiveProcessing — cortical surprise (prediction error), The final HPA cortisol output is blended (8% per tick) into the existing **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AllostasisSystem

Scientific Role / Description: Long-term allostatic load and physiological adaptation. Repeated or chronic stress accumulates allostatic load, which: 1. Raises the cortisol setpoint (chronic elevation) 2. Weakens PFC top-down regulation 3. Weakens hippocampal safety inhibition (McEwen 1998 — allostatic overload model) Recovery mechanisms: - SWS sleep reduces load (Born et al. 1997) - Oxytocin provides social stress buffering (Neumann 2002) Resilience tracks the organism's overall capacity to buffer stress. **Formula**: - Cortisol in safe range + oxytocin = resilience recovery **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SelfModelSystem

Scientific Role / Description: - cortisol trajectory - emotional valence transitions And updates its belief in regulation ability (regulation_confidence). maintains a generative model of its own body state and corrects it Learning rate is very slow (0.002) to operate on behaviorally meaningful timescales, not tick-by-tick noise. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Predictive interoceptive self-model., Ikigai learns to predict its own future physiological states:, This models the brain's interoceptive prediction system — the organism, via prediction errors (Seth 2013, Damasio 1999). **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

VagalInteroceptionSystem

Scientific Role / Description: Simulates vagus-nerve–like interoceptive feedback between body and brain. Models three physiological signals: - heart_rate : peripheral arousal (increases with stress) - vagal_tone : parasympathetic calming (increases with safety/sleep/OXT) 1. High vagal_tone suppresses HPA activation (applied post-step to cort.level) Porges 2007: vagal cardiac brake buffers amygdala → HPA pathway 2. High body_stress amplifies amygdala threat signal (applied to bla_valence) Damasio 1999: somatic markers bias threat evaluation All state variables bounded in [0, 1]. Low-frequency dynamics (≤0.01/tick) preserve simulation stability. **Formula**: - body_stress : integrated somatic burden = 0.6*cortisol + 0.4*heart_rate **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Brain feedback effects (applied in main loop): **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HomeostasisSystem

Scientific Role / Description: Central homeostatic regulation layer (Cannon 1932 / Sterling & Eyer 1988). Continuously monitors five physiological drives and produces motivational outputs that gate action selection. Drive definitions: `hunger` — energy below setpoint (explore/forage); `safety` — cortisol above setpoint (withdraw/regulate); `social` — oxytocin below setpoint (approach agent); `sleep` — adenosine above setpoint (rest/consolidate); `curiosity` — prediction error novelty (explore/learn). Implements the Borbely two-process sleep model: sleep onset when drive>ONSET_THRESHOLD; end when drive<OFFSET_THRESHOLD. Additive Borbely fix (Day 19.5): `drives["sleep"] -= min(0.10, _wake_drive)` prevents sleep attractor. Setpoints updated ONLY via AllostasisSystem (never directly modified here).

```
def update(self, avg_energy, cortisol, oxytocin, adenosine, prediction_error):
    drives = self.drives
    drives["hunger"]    = max(0.0, min(1.0, abs(avg_energy  - self.setpoints["energy"])))
    drives["safety"]    = max(0.0, min(1.0, abs(cortisol    - self.setpoints["cortisol"])))
    drives["social"]    = max(0.0, min(1.0, abs(oxytocin    - self.setpoints["oxytocin"])))
    drives["sleep"]     = max(0.0, min(1.0, abs(adenosine   - self.setpoints["adenosine"])))
    drives["curiosity"] = max(0.0, min(1.0, prediction_error))
    self.global_imbalance = sum(drives.values()) / len(drives)

def should_sleep_onset(self, circadian=None):
    return self.drives["sleep"] > self.SLEEP_ONSET_THRESHOLD and not self._sleep_active

def should_sleep_end(self):
    return self._sleep_active and self.drives["sleep"] < self.SLEEP_OFFSET_THRESHOLD
```

Formula: `D_i = x_i - x*_i`; `d_i = |D_i|` clipped [0,1]; `E = mean(d_i)`. Sleep onset: `drives["sleep"] > ONSET_THR (0.30)`; sleep end: `drives["sleep"] < OFFSET_THR`. BG drive biases: `approach += curiosity×0.3 + social×0.5`; `withdraw += safety×0.4`; `explore += curiosity×0.7 + hunger×0.3`. **Hard Constants**: `SLEEP_ONSET_THRESHOLD=0.30`; `SLEEP_OFFSET_THRESHOLD` (setpoint-relative); `MIN_WAKE_TICKS=80`; setpoints: energy=0.7, cortisol=0.15, oxytocin=0.4, adenosine=0.0. **Law Invariants**: All drive values clamped [0, 1]. `should_sleep_end()` MUST return `self._sleep_active` (not `True`) to avoid tick-0 NameError on `sleeping` variable. Setpoints immutable within this class. **Waking Chain Placement**: `homeostasis.update(avg_energy, cort, oxt, ado, PE)` called mid-tick; drive outputs consumed by BasalGangliaSystem action selection. **Serialization Fields**: `drives` (dict), `global_imbalance`, `_sleep_active`, `_wake_ticks`, `setpoints` (dict). **EEIL Interpretation**: The five-drive homeostatic error signal is the organism's primary survival alignment term. Global imbalance `E` directly maps to EEIL EES deviation; the sleep drive implements the Borbely Process S energy recovery budget.

CircadianSystem

Scientific Role / Description: Simulates the suprachiasmatic nucleus (SCN) circadian oscillator. phase: [0.0, 1.0) — position in the 24-hour cycle. speed: phase increment per tick (0.0005 → full cycle ≈ 2000 ticks ≈ 24 h equivalent). **Formula**: Sleep phase: phase > 0.65 (35% of cycle = wake-permitted window ≈ 8.4 h equivalent)., Wake phase: phase ≤ 0.65 (65% of cycle = maximum sleep suppression window). **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

---

3. Pre-Day-24 Cognition Ladder

LatentStateVector

Scientific Role / Description: 64-dim continuous float vector representing Ikigai's compressed lived context. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ActionReasoningLog

Scientific Role / Description: Causal rationale trace for every decision tick. Record schema ---------- tick : int — absolute simulation tick latent_pre : list — LSV snapshot BEFORE action selection (len 64) explore_prob : float — computed exploration probability this tick floor_triggered : bool — True when 0.02 random floor fired selected_action : str — final action string reason_stage : str — 'wm' | 'explore' | 'floor' energy_delta : float — energy change this tick (post - pre) cortisol_delta : float — cortisol change this tick (post - pre) latent_post : list — LSV snapshot AFTER update (len 64) Future consumers (read-only) ---------- narrative.action_log — grounded sentence generation planning_sys.action_log — replay supervision / counterfactual semantic.action_log — concept supervision **Formula**: Standard bounded integration. **Hard Constants**: Stores records in a memory-safe collections.deque(maxlen=1000). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Each record is a flat dict — serialisable, inspectable, and replayable., wm_scores : dict — world-model survival values **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ReplayBuffer

Scientific Role / Description: Salience-biased experience replay buffer for decision transitions. Stores compact (latent_pre, action, reason_stage, deltas, latent_post) high-salience transitions (large |energy_delta| + |cortisol_delta|) to bias replay toward consequential events. Episode schema ---------- tick : int -- absolute simulation tick latent_pre : list -- LSV snapshot BEFORE action (len 64) action : str -- selected action string reason_stage : str -- 'wm' | 'explore' | 'floor' energy_delta : float -- energy change this tick cortisol_delta : float -- cortisol change this tick latent_post : list -- LSV snapshot AFTER update (len 64) Future consumers (read-only) ---------- semantic -- concept supervision from salient transitions planning_sys -- counterfactual planning from stored transitions episodic_replay -- cross-system trajectory alignment **Formula**: sleep consolidation -- sample_for_replay(k=16) during SWS **Hard Constants**: episodes in a bounded deque(maxlen=2000). Sampling prioritises **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

EventCompressor

Scientific Role / Description: Temporal event segmentation over replay transitions. Reads from ReplayBuffer transitions (via ingest_transition), groups temporally contiguous, contextually similar transitions into compressed Segmentation law (ALL must hold to stay in same event): 2. cosine_similarity(latent_post_prev, latent_pre_next) > 0.90 3. same action family OR same reason_stage 4. cumulative salience still rising (not dropped sharply) Compressed event schema ---------- start_tick : int - first tick of the episode end_tick : int - last tick of the episode length : int - number of transitions merged dominant_action : str - most frequent action in run dominant_reason_stage : str - most frequent reason_stage in run mean_energy_delta : float - mean energy_delta across run mean_cortisol_delta : float - mean cortisol_delta across run peak_salience : float - max |e_delta|+|c_delta| in run latent_start : list - latent_pre of first transition (len 64) latent_end : list - latent_post of last transition (len 64) Future consumers (read-only) ---------- semantic.event_compressor - concept chunking / grounded sentences narrative.event_compressor - narrative episode construction planning_sys.event_compressor - hierarchical planning motifs **Formula**: 1. ticks are contiguous (gap <= 1) **Hard Constants**: events, and stores them in a bounded deque(maxlen=500). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ConceptGraph

Scientific Role / Description: Learnable concept graph over compressed episode space. Each node is a latent concept centroid formed by EMA-merging recurring compressed events that share action family, reason stage, and high cosine similarity. Directed edges record sequential concept activations. Node format ---------- id : int centroid : list[64] -- EMA latent prototype dominant_action : str dominant_reason_stage: str support : int -- number of events merged mean_salience : float -- EMA of peak_salience last_tick : int debug_label : str|None -- optional human-readable tag Edge format ---------- edges[(src_id, dst_id)] : int -- activation count, capped at 255 Concept formation law ---------- Merge into existing node if ALL hold: 1. cosine_similarity(event.latent_end, centroid) > similarity_threshold 2. dominant_action matches 3. dominant_reason_stage matches OR similarity > 0.97 Otherwise create new node. Overflow (>max_nodes): evict node with lowest support * recency score. Edge law -------- Consecutive concept activations within 10 ticks reinforce the directed edge (src -> dst). Weight capped at 255. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ReportBus

Scientific Role / Description: Shared cognitive workspace: publishes a compact scene snapshot each tick. Each report is a fixed-schema semantic snapshot derived from: - LatentStateVector : current interoceptive + neural state - ConceptGraph : active concept IDs and their salience - ActionReasoningLog : last action and decision stage Report schema - ---------- tick : int - absolute simulation tick latent_focus : list - top-8 most informative LSV dims (by abs dev from 0.5) top_concepts : list - up to 3 concept node IDs by similarity to current LSV last_action : str - most recent selected action last_reason_stage : str - most recent reason_stage ('wm'|'explore'|'floor') mean_salience : float - mean of top-concept mean_salience values [0,1] workspace_confidence: float - blended confidence score [0,1] Consumers (read-only) ---------- semantic.report_bus - grounded sentence generation narrative.report_bus - narrative self-model planning_sys.report_bus - hierarchical planning context **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

LanguageReadout

Scientific Role / Description: Semantic proposition generator over ReportBus scene snapshots. representing the organism's current cognitive state in meaning space. Proposition schema ---------- tick : int -- simulation tick of source scene focus_concept : int | None -- first concept ID in scene top_concepts state_polarity : str -- 'neutral'|'stress'|'recovery'|'explore' action_intent : str -- scene last_action causal_basis : str -- scene last_reason_stage confidence : float -- scene workspace_confidence [0,1] State polarity inference (latent_focus dims) ---------- The latent_focus is a list of the 8 highest-deviation dims from LSV. Polarity is inferred from the statistical signature of those dims: 'stress' : mean > 0.60 (cortisol/threat dims elevated) 'recovery' : mean < 0.40 AND dim spread < 0.15 (depleted, converging) 'explore' : spread > 0.25 AND NOT stress (high variance, exploratory) 'neutral' : otherwise Polarity is purely statistical over latent_focus values -- no hardcoded indices, no hardcoded dim semantics. Future consumers (read-only) ---------- semantic.language_readout - grounded sentence seed narrative.language_readout - self-narration substrate

planning_sys.language_readout - planning context **Formula**: Standard bounded integration. **Hard Constants**: Propositions are stored in a bounded deque(maxlen=64). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Each call to read_scene() produces one structured proposition dict **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SentenceGenerator

Scientific Role / Description: Compositional natural-language surface over LanguageReadout propositions. Converts the 6-field semantic proposition into a grounded English sentence by selecting and combining three independent clause banks: Clause A (state) -- from state_polarity Clause B (action) -- from action_intent Clause C (causal) -- from causal_basis Three structural templates vary by confidence tier: Low (< 0.35) : '', while ." Within each clause bank multiple phrases are defined; selection rotates by (tick % n_variants) so consecutive identical polarities still differ. Utterance schema ---------- tick : int sentence : str -- grounded English sentence (not template artifact) confidence : float -- from proposition.confidence [0,1] focus_concept : int|None -- from proposition.focus_concept **Formula**: High (>= 0.65) : '', .'', Mid (>= 0.35) : ''. ." **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

NarrativeMemory

Scientific Role / Description: Session-level autobiographical memory over the language substrate. Accumulates waking sentences from SentenceGenerator into a 'current_arc' buffer. When the arc ends (focus shift, confidence jump, sleep boundary), it is flushed as a single compact autobiographical record and stored in a Arc continuity criteria (all must hold to extend arc) ---------- 1. focus_concept unchanged (same or both None) 2. confidence delta from previous sentence < 0.20 4. no sleep boundary signal Arc schema ---------- start_tick : int end_tick : int dominant_focus : int | None -- mode focus_concept across arc sentences dominant_theme : str -- 'challenge'|'stabilization'|'discovery'|'transition' arc_summary : str -- 2–3 sentence autobiographical paragraph mean_confidence : float -- mean sentence confidence across the arc Arc summary composition (not a fixed template) ---------- Built from: - opening clause from first sentence - pattern theme clause (counts of polarity seen) - closing clause from final sentence - optional sleep motif suffix if sleep_consolidator provides a focus Composed as a 2–3 sentence paragraph. No raw transcript dump. Sleep gate ---------- **Formula**: 3. top ConceptGraph node unchanged (query_similar with k=1), flush_arc(reason="sleep_boundary") commits the arc at every sleep onset., ingest_tick() must NOT be called during sleeping==True ticks; **Hard Constants**: bounded deque(maxlen=128). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: the main loop ensures this by placing it inside the 'if not sleeping:' branch. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SleepConsolidator

Scientific Role / Description: Offline semantic memory consolidation during sleep. Consolidation steps (per sleep tick, call from 'if sleeping:' branch): 1. Concept reinforcement For each transition, query nearest ConceptGraph node. Increment support by 1, capped at +4 per sleep cycle per node. Records which concept IDs were reinforced. 2. Edge reinforcement Read recent compressed events (last 5 from EventCompressor). For consecutive event pairs, reinforce their concept-edge. Weight capped at 255 (same as ConceptGraph law). 3. Confidence calibration Read last 10 SentenceGenerator utterances. Compute variance of confidence values. High variance (> 0.04) -> reduce support of lowest-support nodes by 1. Low variance (< 0.01) -> increase support of top-3 nodes by 1. Returns confidence_shift (net change, float). 4. Dominant sentence focus Most frequent focus_concept from recent utterances. Consolidation record schema ---------- tick : int reinforced_concepts : list[int] -- concept IDs whose support changed reinforced_edges : list[tuple] -- (src_id, dst_id) pairs reinforced confidence_shift : float -- net support calibration effect dominant_sentence_focus : int|None -- most frequent focus_concept Sleep gate ---------- Call ONLY from 'if sleeping:' branch. Waking path does not call consolidate(). **Formula**: Sample replay buffer (salience-biased, k=16)., Apply soft EMA tightening to centroid (alpha=0.98, very gentle). **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ReflectiveReasoner

Scientific Role / Description: Post-sleep metacognitive self-assessment over autobiographical arcs. Reads the last 5 committed NarrativeMemory arcs and produces a structured reflection record covering: - reflection_type : motif detected across arcs - confidence_trend : direction of mean confidence across arcs - loop_risk : recurrence rate of dominant arc focus [0, 1] - reflection_summary : 2-sentence metacognitive note Reflection types ---------- 'stress_loop' — repeated 'challenge' arcs (≥ 3 of 5) 'successful_regulation' — repeated 'stabilization' arcs (≥ 3 of 5) 'productive_exploration'— repeated 'discovery' arcs (≥ 3 of 5) 'transition_instability'— alternating or mixed themes without majority Confidence trends ---------- Compare mean confidence of first-half vs second-half of arc window: rising (diff > +0.05) → 'improving' falling (diff < -0.05) → 'destabilizing' else → 'flat' Loop risk ---------- count_of_most_frequent_dominant_focus / len(arcs_sampled) Clamped [0.0, 1.0]. Sleep gate ---------- reflect() MUST be called only from the 'if sleeping:' branch, after sleep_consolidator.consolidate() has run. Zero waking writes. Reflection record schema ---------- tick : int dominant_arc_focus : int | None — mode focus across sampled arcs reflection_type : str confidence_trend : str loop_risk : float reflection_summary : str — always exactly 2 sentences **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

CognitivePlanner

Scientific Role / Description: Future-directed cognitive goal generation from reflective arc assessment. Reads ReflectiveReasoner.latest() to derive the dominant cognitive pattern, then maps it to an explicit goal type with urgency scoring and a 2-sentence planning summary. Goal types ---------- 'stabilize' — stress loop detected with high recurrence risk 'reinforce_success' — successful regulation with improving confidence 'expand_exploration' — productive exploration with healthy diversity 'reduce_variance' — thematic instability with no majority pattern 'maintain_continuity' — default when reflection is unavailable or neutral Urgency blend (clamped [0, 1]) ---------- clamp to [0.0, 1.0] Plan schema ---------- tick : int goal_type : str target_focus : int | None — dominant_arc_focus from reflection urgency : float [0, 1] planning_summary : str — always exactly 2 sentences Sleep gate ---------- plan() MUST be called only from 'if sleeping:' branch, after reflective_reasoner.reflect() has run. Zero waking writes. **Formula**: base = loop_risk * 0.50, +0.25 if confidence_trend == 'destabilizing', -0.10 if confidence_trend == 'improving' **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

GoalExecutionBridge

Scientific Role / Description: Waking semantic attention bias derived from the active cognitive plan. Each waking tick, reads CognitivePlanner.latest() and computes a semantic_bias score that downstream semantic systems (ReportBus, LanguageReadout, SentenceGenerator, NarrativeMemory) may optionally Bias is exposed ONLY via two safe read hooks on planning_sys: planning_sys.semantic_bias — float [0, 1]: current salience preference planning_sys.goal_focus — int | None: dominant concept node to weight No direct writes to any behavioral or survival path. Semantic bias blend ---------- (falls back to 0.0 if target_focus is None or graph is empty) Result clamped to [0.0, 1.0]. Bridge schema ---------- tick : int active_goal : str target_focus : int | None semantic_bias : float [0, 1] bridge_summary : str — always exactly 2 sentences **Formula**: bias = plan.urgency × 0.70 + concept_support_norm × 0.30, where concept_support_norm = support(target_focus) / max_support **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: consult to weight their outputs toward the active goal focus. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

TaskFramework

Scientific Role / Description: Semantic-task layer: derives explicit cognitive tasks from waking bias. Reads the active semantic_bias and goal_focus from planning_sys, along with NarrativeMemory context, to spawn discrete internal task objects. These tasks represent what the organism's cognitive attention should "work on" during the current waking cycle (e.g., stabilizing, exploring). Task types ---------- 'background_monitoring' (bias < 0.35) Priority Blend ---------- Clamped to [0.0, 1.0]. Task schema ---------- tick : int task_type : str task_focus : int | None priority : float [0, 1] task_summary : str — always exactly 2 sentences **Formula**: 'high_priority_stabilization' (bias >= 0.80), 'focused_continuation' (bias >= 0.60), 'exploratory_probe' (bias >= 0.35), Override: 'research_expansion' (recent dominant_theme == 'discovery' AND bias >= 0.50), priority = (semantic_bias * 0.75) + (recent narrative confidence * 0.25) **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ToolRouter

Scientific Role / Description: Intent routing layer: maps cognitive tasks to specific internal operations. Reads the active task from TaskFramework and converts it into a structural intent. This layer acts as the proto-API boundary: it decides *what* tool or operation profile the organism would theoretically execute (e.g. inspect, continue, expand) before any actual execution engine exists. Operation type mapping ---------- 'high_priority_stabilization' -> 'inspect_and_adjust' 'focused_continuation' -> 'continue_thread' 'exploratory_probe' -> 'compare_paths' 'background_monitoring' -> 'observe_only' 'research_expansion' -> 'hypothesis_expand' Confidence Blend ---------- Clamped to [0.0, 1.0]. Route schema ---------- tick : int operation_type : str task_origin : str route_confidence : float [0, 1] route_summary : str — always exactly 2 sentences **Formula**: route_confidence = (task_priority * 0.80) + (recent narrative continuity * 0.20) **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ExecutionSandbox

Scientific Role / Description: Simulated execution layer: converts intents into mock outcome traces. Reads the active routed operation from ToolRouter and mocks its execution. This provides a closed-loop framework for the organism to 'simulate' operations and estimate their success without requiring an external engine, supporting expected-vs-observed reasoning protocols. Outcome mapping ---------- 'inspect_and_adjust' -> 'state_refined' 'continue_thread' -> 'continuity_preserved' 'compare_paths' -> 'branch_difference_observed' 'observe_only' -> 'state_monitored' 'hypothesis_expand' -> 'novel_relation_detected' Success Score Blend ---------- Clamped to [0.0, 1.0]. Execution schema ---------- tick : int operation_type : str simulated_outcome : str success_score : float [0, 1] execution_summary : str — always exactly 2 sentences **Formula**: success_score = (route_confidence * 0.70) + (narrative continuity * 0.30) **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ErrorReflector

Scientific Role / Description: Error reflection layer: detects expectation mismatches from simulated traces. Reads the active simulated execution and compares it against the origin task trajectory. Identifies logical discrepancies (e.g., trying to innovate when explicitly told to stabilize) and generates internal self-repair pressure. Produces purely reflective notes; requires a later executor loop for true correction. Mismatch mapping ---------- 'continuity_preserved' + non-'focused_continuation' task -> 'trajectory_mismatch' 'state_refined' + low priority (< 0.50 task priority) -> 'overcorrection' 'novel_relation_detected' + 'high_priority_stabilization' task -> 'goal_divergence' otherwise -> 'aligned' Repair Pressure ---------- Clamped [0.0, 1.0] Reflection schema ---------- tick : int expected_outcome : str mismatch_type : str repair_pressure : float [0, 1] reflection_summary : str — always exactly 2 sentences **Formula**: pressure = ((1 - success_score) * 0.70) + (task_priority * 0.30), If mismatch == 'aligned': pressure *= 0.25 **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

RetryPlanner

Scientific Role / Description: Acts as the final node in the mock-execution loop. Interprets trajectory errors derived from expected-vs-observed mismatches and generates revised structural plans. Strictly observer-only; produces proto-self-repair intents that could later be acted upon by the waking baseline engine. Strategy mapping ---------- 'trajectory_mismatch' -> 'force_continuity' 'overcorrection' -> 'lower_intensity' 'goal_divergence' -> 'fallback_to_stable_route' 'aligned' -> 'preserve_route' Retry Confidence Blend ---------- Clamped [0.0, 1.0] Retry schema ---------- tick : int retry_strategy : str retry_confidence : float [0, 1] retry_summary : str — always exactly 2 sentences **Formula**: Retry planning layer: consumes mismatch reflection to formulate safe intent reiterations., confidence = (repair_pressure * 0.70) + ((1.0 - route_confidence) * 0.30), If mismatch == 'aligned': confidence *= 0.20 **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

RouteMutator

Scientific Role / Description: Mutator layer: converts retry strategy into a next-route operation bias override. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MutationGuard

Scientific Role / Description: Guard layer: prevents retry oscillation and loop lock. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

RetryOutcomeTracker

Scientific Role / Description: Outcome layer: measures whether the simulated trace improved after mutation. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

StrategyLearner

Scientific Role / Description: Learns which retry strategies effectively improve coherence. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PolicyShaper

Scientific Role / Description: Turns learned preferences into future route priors and mutation dampening. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

FailureAtlas

Scientific Role / Description: Stores recurring mismatch to successful-recovery pairings as knowledge maps. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AbstractTaskEngine

Scientific Role / Description: No documentation provided. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PlanGraphMemory

Scientific Role / Description: No documentation provided. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SubgoalEvaluator

Scientific Role / Description: No documentation provided. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

---

4. Day 28 Epoch Expansion

> ***Day 28 — Active Continuity to Collaborative Intelligence Epoch (Phases A → M)***

ProjectWorkspace

Scientific Role / Description: Maintains the organism's current active work-product context. Schema: tick : int workspace_id : str artifact_type : str -- 'code_block' | 'document' | 'research_note' active_section : str -- mirrors current active_subgoal workspace_confidence : float [0, 1] workspace_summary : str **Formula**: Confidence = plan_confidence * 0.65 + progress_score * 0.35, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ArtifactStateMemory

Scientific Role / Description: Preserves versioned internal artifact evolution (object permanence memory). Schema: tick : int artifact_signature : str -- ':' version_index : int -- increments on each consecutive identical signature state_stability : float [0, 1] artifact_summary : str DEDUP LAW: consecutive identical signatures do NOT produce a new record. Instead, stability is reinforced (EMA toward 1.0) and version_index increments. New signature resets version to 1 and appends a fresh record. **Formula**: Stability = workspace_confidence * 0.70 + improvement_score * 0.30, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ConsistencyVerifier

Scientific Role / Description: Verifies that the evolving artifact still matches current subgoal + workflow graph. Prevents silent continuity drift. Schema: tick : int consistency_state : str -- 'consistent' | 'stale' | 'drifted' | 'uncertain' consistency_score : float [0, 1] consistency_summary : str State rules: uncertain : all other cases **Formula**: consistent : matching active_section + subgoal_state == 'progressing', stale : same artifact section + subgoal_state == 'stalled', drifted : subgoal_state == 'destabilized' + section mismatch (signature jump), Score = workspace_confidence * 0.60 + state_stability * 0.40, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

EditIntentGenerator

Scientific Role / Description: Determines the next symbolic edit operation from the active artifact state. Schema: tick : int edit_operation : str -- symbolic op (analyze_segment, modify_segment, ...) target_region : str -- mirrors active_section edit_confidence : float [0, 1] intent_summary : str Operation mapping (active_section -> edit_operation): inspect -> analyze_segment refine -> modify_segment verify -> validate_segment integrate -> merge_segment synthesize-> synthesize_segment default -> modify_segment **Formula**: Confidence = workspace_confidence * 0.70 + consistency_score * 0.30, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PatchPreviewMemory

Scientific Role / Description: Stores symbolic before/after patch previews (proto internal diff memory). Schema: tick : int before_signature : str -- current artifact signature after_signature : str -- '->' preview_strength : float [0, 1] repeat_count : int -- how many consecutive identical after_signature calls patch_summary : str DEDUP LAW: repeated identical after_signature -> update in place. No duplicate record appended. **Formula**: preview_strength = old * 0.92 + new * 0.08 + 0.01, repeat_count += 1, Preview strength = state_stability * 0.60 + edit_confidence * 0.40, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

IntegrityScorer

Scientific Role / Description: and subgoal alignment. First symbolic edit safety gate. Schema: tick : int integrity_state : str -- 'safe_patch' | 'redundant_patch' | 'unsafe_patch' | 'uncertain' integrity_score : float [0, 1] integrity_summary : str State rules (priority order): uncertain : all other cases **Formula**: safe_patch : consistency_state == 'consistent' AND preview_strength > 0.70, redundant_patch: consistency_state == 'stale' AND repeat_count > 1, unsafe_patch : consistency_state == 'drifted', Score = preview_strength * 0.65 + consistency_score * 0.35, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Validates whether the predicted patch still preserves artifact continuity **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PatchExecutor

Scientific Role / Description: Simulates applying a safe symbolic patch preview. First internal artifact state transition. Schema: tick : int applied_signature : str -- always preview.after_signature execution_state : str -- 'applied' | 'skipped' | 'blocked' | 'tentative' execution_score : float [0, 1] execution_summary : str Execution mapping (integrity_state -> execution_state): safe_patch -> applied redundant_patch -> skipped unsafe_patch -> blocked uncertain -> tentative **Formula**: Score = preview_strength * 0.70 + integrity_score * 0.30, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

VersionDiffTracker

Scientific Role / Description: Tracks symbolic pre/post artifact transitions (version trajectory memory). Schema: tick : int from_version : int -- artifact.version_index at time of call to_signature : str -- executor.applied_signature diff_state : str -- 'advanced' | 'unchanged' | 'protected' | 'uncertain' diff_score : float [0, 1] stagnation_count : int -- consecutive unchanged diffs (dedup counter) diff_summary : str Diff mapping (execution_state -> diff_state): applied -> advanced skipped -> unchanged blocked -> protected tentative -> uncertain DEDUP LAW: consecutive 'unchanged' diffs -> update in place. No duplicate record appended. **Formula**: Score = execution_score * 0.65 + state_stability * 0.35, clamped [0, 1]., diff_score = old * 0.90 + new * 0.10 + 0.01, stagnation_count += 1 **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

RollbackController

Scientific Role / Description: Restores last stable symbolic version when patch evolution degrades. First symbolic recovery controller. Schema: tick : int rollback_state : str -- 'rollback_to_last_stable' | 'hold_previous' | 'commit_forward' | 'monitor' restored_signature : str | None -- target signature for the rollback action rollback_score : float [0, 1] rollback_summary : str State rules (priority order): monitor : all other cases restored_signature: rollback_to_last_stable -> preview.before_signature hold_previous -> preview.before_signature commit_forward -> preview.after_signature monitor -> None **Formula**: rollback_to_last_stable: diff_state == 'protected' AND integrity_state == 'unsafe_patch', hold_previous : diff_state == 'protected' AND integrity_state != 'unsafe_patch', commit_forward : diff_state == 'advanced' AND integrity_state == 'safe_patch', Score = diff_score * 0.60 + integrity_score * 0.40, clamped [0, 1]. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

StructureMapBuilder

Scientific Role / Description: Builds a nested semantic territory map from the organism's active workspace and artifact cognition state. Four detection layers: A -- epoch banners (DAY 14 ... DAY 25) B -- subsystem clusters (hpa_axis, homeostasis, ...) C -- class territories (Neuron, Synapse, HPAAxisSystem, ...) D -- micro-regions (init, scoring_law, serialization, citation, ...) Region identity is persistent: a region_id does not change unless the semantic role or anchor overlap falls below 70%. tick : int region_count : int epoch_count : int protected_count : int map_confidence : float [0, 1] map_summary : str **Formula**: map_confidence = min(1.0, region_count / 10.0) -- confident once 10+ regions found. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Schema per record:, regions : list[dict] -- each region follows SemanticRegion fields **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DependencyLinkMemory

Scientific Role / Description: Maps: - class usage chains (which regions call / depend on which) - report-bus hooks (downstream publish consumers) - rollback compatibility (edit safe within or crosses stable boundary) - test region coverage (estimated coverage per cluster) - comment provenance inheritance (citation / law comments) Reinforcement model analogous to ConceptGraph and PlanGraphMemory: Each dependency edge gains weight each time it is confirmed active. Edges decay slowly (EMA 0.98) to eliminate stale links. tick : int source_region : str downstream_regions : list[str] blast_radius : float [0, 1] confidence : float [0, 1] dependency_summary : str **Formula**: blast_radius = _BLAST_WEIGHTS.get(class_name, 0.30) * confidence, clamped [0, 1]., confidence = mean edge weight across downstream_regions. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Tracks region-to-region dependency chains and predicts edit blast radius., - serialization dependencies (to_dict / from_dict coupling), Schema per record:, edge_weights : dict **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SemanticRegionGuard

Scientific Role / Description: Rejects unsafe edits before Day 24 edit-intent generation. Checks: 1. Protected biology boundary crossing -- blocks Neuron, Synapse, HomeostasisSystem, HPAAxisSystem, CortisolSystem, AdenosineSystem edits with high blast radius. 3. Banner provenance deletion -- blocks edits targeting epoch banners. 5. Rollback-incompatible region merges. States (exactly one per record): safe_region_patch : edit is within bounds, no violations boundary_violation : edit crosses protected region boundary protected_zone_violation : edit targets a protected biology class serialization_drift : edit breaks serialization coupling uncertain : insufficient data to evaluate Score: safe_region_patch : map_confidence * 0.70 + (1 - blast_radius) * 0.30 boundary_violation / protected_zone_violation / serialization_drift : blast_radius * 0.30 uncertain : 0.50 tick : int guard_state : str guard_score : float [0, 1] source_region : str | None violation_detail : str guard_summary : str **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: 4. Invariant comment deletion -- blocks edits removing law/citation comments., Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: 2. Serialization drift -- blocks if edit would break to_dict/from_dict coupling., Schema per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

RegionRewritePlanner

Scientific Role / Description: Generates ordered multi-region rewrite plans from structure map and dependency data. Rewrite order law (dependency-first — Dijkstra 1968): Step 1: interface-origin region (the class being changed) Step 2: downstream dependent regions (blast-radius ordered, highest first) Step 4: test anchor regions Step 5: rollback ancestry update requires_guard_escalation: True when the step touches a protected biology class (Rule 2). tick : int step_count : int max_risk : float [0, 1] requires_escalation : bool -- any step requires guard escalation plan_confidence : float [0, 1] plan_summary : str **Formula**: Risk score formula:, risk = blast_radius * 0.60 + integrity_risk * 0.40 (clamped [0, 1]), where integrity_risk = 1.0 - integrity_score., plan_confidence = guard_score * 0.60 + (1 - max_risk) * 0.40. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Step 3: serialization regions (to_dict/from_dict coupling), Schema per record:, plan_steps : list[dict] -- ordered list of RewriteStep-schema dicts **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

InterfaceStabilityChecker

Scientific Role / Description: Validates that multi-region rewrite plans do not silently break interfaces. Validates: 1. Public method names remain in their cluster's exported interface set 3. Rollback signatures remain accessible after the plan executes 4. Region IDs referenced in downstream chains remain resolvable 5. Exported test anchors referenced in plan step 4 remain intact States (exactly one per record): interface_stable : all checks pass migration_required : a public method or exported symbol changes serialization_break_risk: serialization region mutated without paired update rollback_break_risk : rollback ancestry step missing or high-risk uncertain : insufficient plan data interface_stable : plan_confidence * 0.70 + (1 - max_risk) * 0.30 migration_required : 0.50 * plan_confidence serialization_break_risk: 0.35 rollback_break_risk : 0.30 uncertain : 0.50 tick : int stability_state : str stability_score : float [0, 1] migration_regions : list[str] -- regions that require interface migration serial_break_risk : bool rollback_break_risk: bool stability_summary : str **Formula**: Score formula: **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: 2. Serialization keys (to_dict/from_dict) remain paired, Schema per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

CrossRegionCommitGraph

Scientific Role / Description: Tracks grouped multi-region commit continuity across waking ticks. Each commit group bundles all the rewrite steps from one RegionRewritePlanner a reinforced success score. Commit ID generation: f"commit_*" Rollback parent: None if no such ancestor exists. Reinforcement law (Fowler 1999 refactoring principle -- preserve history): else: Clamped [0, 1]. tick : int commit_id : str region_chain : list[str] -- ordered region_ids from plan stable_interfaces : bool rollback_parent : str | None success_score : float [0, 1] partial_success : bool -- True if some steps succeeded, some failed failed_regions : list[str] -- steps with risk_score > 0.70 commit_summary : str **Formula**: where source_region_slug = source_region[:20].replace(':', "*").replace(' ', '_'), The most recent previous commit_id where success_score >= 0.60., If stability_state == 'interface_stable':, new_score = old_score * 0.92 + raw_score * 0.08 + 0.02 (success), new_score = old_score * 0.95 - 0.03 (failure), raw_score = stability_score * 0.60 + (1 - max_risk) * 0.40. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: output into a named unit with rollback ancestry, stability validation, and, Schema per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

RewriteTrajectorySimulator

Scientific Role / Description: Generates sparse future rewrite trajectories BEFORE patch execution. Expands exactly (up to) 8 trajectory nodes (EEIL sparse compute law): 1. source region 2-5. top-4 downstream regions by edge weight 6. serialization region 7. test anchor 8. rollback ancestry (amplified if region is protected) requires_biology_escalation: True if any protected class name or stem appears in region_id. tick : int node_count : int any_biology_escalation: bool current_integrity : float [0, 1] -- integrity_score at simulation time

trajectory_confidence : float [0, 1] trajectory_summary : str **Formula**: Prediction formulas:, predicted_integrity = current_integrity * (1.0 - risk_score * 0.35), rollback_survivability = 1.0 - max(0.0, risk_score - 0.4), invariant_risk = risk_score * blast_radius * 0.5, trajectory_confidence = (1 - max_invariant_risk) * 0.70 + current_integrity * 0.30 **Hard Constants**: trajectory : list[dict] -- TrajectoryState-schema dicts (maxlen=16) **Law Invariants**: max_invariant_risk : float [0, 1], Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Schema per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

InvariantDriftEstimator

Scientific Role / Description: BEFORE patch execution. States (exactly one): stable_projection : drift < 0.20 and no biology escalation AND no rollback ancestry node is low-risk biology_projection_risk : any trajectory node has requires_biology_escalation uncertain : trajectory empty or data absent Priority: biology_projection_risk > serialization_projection_risk > localized_drift > stable_projection Score: stable_projection : 1.0 - drift localized_drift : max(0.0, 0.60 - drift) serialization_projection_risk : 0.35 biology_projection_risk : 0.20 uncertain : 0.50 tick : int drift_state : str drift_score : float [0, 1] mean_drift : float [0, 1] drift_summary : str **Formula**: Drift formula:, drift = mean(1.0 - predicted_integrity across all trajectory nodes), localized_drift : 0.20 <= drift < 0.45 and no biology escalation, rollback_protected : bool -- rollback survivability >= 0.60 **Hard Constants**: Derived from biology. **Law Invariants**: Estimates semantic invariant degradation across the simulated trajectory, serialization_projection_risk: serialization node has high invariant_risk (>0.35), serial_node_risk : float -- invariant_risk of serialization node, Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Schema per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PreCommitOutcomeMemory

Scientific Role / Description: Clamped [-1, 1]. actual_score is derived from: cross_region_commit_graph.latest().success_score tick : int chain_signature : str actual_score : float [0, 1] future_bias : float [-1, 1] rollback_succeeded : bool -- commit ancestry non-None interface_stable : bool -- from cross_region_commit_graph trace_summary : str **Formula**: prediction_error = abs(actual_score - predicted_score), if prediction_error <= 0.10: (accurate prediction), future_bias = old_bias * 0.90 + prediction_error * 0.10 + 0.02, future_bias = old_bias * 0.90 + prediction_error * 0.10 - 0.03 **Hard Constants**: Derived from biology. **Law Invariants**: invariant_drift_estimator.latest().drift_score, Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Stores prediction-vs-actual outcome traces and learns future bias corrections., Prediction error:, Future bias update (EMA prediction-error learning):, else: (inaccurate prediction / overshoot), predicted_score is derived from:, Schema per record:, predicted_score : float [0, 1], prediction_error : float [0, 1] **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

TrajectoryBranchGenerator

Scientific Role / Description: Generates 2-4 sparse rewrite branch futures from the current Pack 2/3 state. Default branch families: 1. direct_patch 2. serialization_first 3. test_first 4. rollback_first (only when rollback / drift / ancestry pressure is high) Each branch is capped at 5 projected nodes and remains sparse by law. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

FutureValueSelector

Scientific Role / Description: Compares sparse branch futures and selects the best projected rewrite path. Frozen scoring law: projected_integrity * 0.35 + rollback_survivability * 0.25 + ancestry_success_prior * 0.15 + eeil_sparse_efficiency * 0.05) Value is clamped to [0, 1]. Selection then applies: - biology escalation penalty (separate from the frozen value law) - drift minimization tie-break - branch sparsity tie-break **Formula**: future_value = (**Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: + (1.0 - predicted_drift) * 0.20 **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

BranchDeliberationConfidence

Scientific Role / Description: Quantifies the confidence of the currently selected Pack 4 branch future. Metric Law: clamped to [0, 1] Observer-only: - reads from FutureValueSelector and BranchOutcomeArchive - does NOT alter branch selection, scoring, or any Pack 1-4 state **Formula**: confidence = selected_branch_score * lineage_stability **Hard Constants**: - writes only to internal bounded deque(maxlen=32) **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

BranchReasoningTrace

Scientific Role / Description: Provides a short bounded trace of why Pack 4 selected a branch future. Captures top 3 reasoning factors: 1. projected integrity contribution 2. rollback survivability contribution 3. drift minimization contribution Optional fourth: - ancestry prior contribution Observer-only: - reads from FutureValueSelector comparisons - does NOT alter branch selection, scoring, or any Pack 1-4 state **Formula**: Standard bounded integration. **Hard Constants**: - writes only to internal bounded deque(maxlen=48) **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

BranchOutcomeArchive

Scientific Role / Description: Archives branch families, selected futures, and ancestry priors over time. Maintains: - branch family - chosen branch - actual score - long-term selection bias - branch lineage signature family_prior_for(family_signature) returns: **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: - predicted score, 0.70 * ema_predicted_score + 0.30 * win_rate **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SelectedBranchIntentBridge

Scientific Role / Description: Converts selected Pack 4 future branch into shadow edit-intent recommendation. Bridge scoring law: Observer-shadow mode: - generates shadow_edit_intent - does NOT mutate edit_intent_generator - no execution routing changes **Formula**: bridge_confidence = selected_branch_score * branch_deliberation_confidence * intent_schema_match **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

IntentConsistencyAuditor

Scientific Role / Description: Audits whether shadow intent matches real downstream intent schema. States: - aligned: shadow matches real intent - schema_partial: partial overlap - schema_mismatch: no meaningful overlap - no_real_intent: no real intent to compare Pre-steering safety layer. Observer-only. **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

BridgeOutcomeMemory

Scientific Role / Description: Persists bridge outcomes for learning whether branch-selected futures produce coherent intent recommendations. Stores: - selected branch id - shadow intent hash - bridge confidence - audit state - actual downstream intent summary - bridge error signal **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SelectedBranchExecutionBridge

Scientific Role / Description: Converts shadow bridge confidence into a bounded live steering bias for downstream edit intent generation. Gating law (ALL conditions must hold): Biology rule (no exceptions): steering_bias -- float [0.0, 0.35] steering_enabled -- bool bias_reason -- str steered_intent_hint -- str or None **Formula**: bridge_confidence >= 0.70, audit_state == 'aligned', bridge_error <= 0.30, rollback_survivability >= 0.65, If selected branch carries biology escalation -> steering_bias = 0.0, Steering bias formula (when gating passes and no biology):, steering_bias = clamp(bridge_confidence * 0.35, 0.0, 0.35) **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ExecutionOutcomeComparator

Scientific Role / Description: Captures action-reality mismatch for strategic policy shaper. execution_alignment_score [0, 1] -- match between steered hint and actual intent steering_error [0, 1] -- divergence combining alignment, integrity, rollback policy_delta [-1,1] -- signed direction signal (+1 success, -1 failure) Uses latest() values from upstream systems (one-tick feedback lag -- biologically plausible and rollback-safe). **Formula**: Standard bounded integration. **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Compares predicted branch-steered edit intent against actual intent output. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

StrategicPolicyShaper

Scientific Role / Description: Converts repeated successful steering events into long-horizon execution policies. Per branch-family statistics (keyed by strategy name): ema_success_rate -- EMA of positive policy_delta events policy_confidence -- 0.7 * ema_success_rate + 0.3 * (1 - failure_suppression) safe_bias_prior -- EMA of steering_bias values on success ticks [0, 0.35] safe_bias_for(family_sig) -> float [0.0, 0.35] Returns accumulated safe bias prior for use by downstream steering. Returns 0.0 if family has no history. Policy states: policy_learning -- steering active, no threshold reached yet steering_disabled -- no active steering this tick **Formula**: failure_suppression -- EMA of steering_error (high = suppress future bias), Policy update law (EMA_ALPHA = 0.10):, ema = old * 0.90 + new * 0.10, policy_mature -- confidence >= 0.70, policy_forming -- ema_success_rate >= 0.60 **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PolicyConditionedSubgoalRouter

Scientific Role / Description: Routes action habits into bounded subgoal arcs conditioned on policy priors. Policy priors from StrategicPolicyShaper are the primary routing signal. Rollback events compress the active horizon by 2 (min 2). Protected active_horizon in [2, 6] route_confidence in [0.0, 1.0] Rollback penalty law: else: slow recovery +1 per tick toward HORIZON_MAX Biology families (zero routing): neuron, synapse, homeostasis, hpa, cortisol, adenosine **Formula**: biology families receive zero routing (active_horizon=0, empty chain)., if rollback_active: active_horizon -= 2, clamped to [2, 6] **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ExecutionHorizonPlanner

Scientific Role / Description: Converts routed subgoal chains into horizon-aware execution arcs. expected_regions -- unique regions expected to be touched risk_decay_curve -- per-step risk [0, 1], grows with step index checkpoint_ticks -- estimated absolute tick offsets for checks horizon_len -- number of planned steps (max 6) arc_confidence -- [0, 1] weighted from route and policy confidence Empty arc produced for biology-excluded families or absent routing data. **Formula**: Max horizon = 6 enforced unconditionally (EEIL sparse compute law). **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, execution_steps -- ordered step dicts (region, action_hint, risk, priority) **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

OutcomeCreditAssigner

Scientific Role / Description: Assigns delayed credit across execution horizon chains. First software temporal credit assignment layer in the NeuroSeed architecture. per_step_credit -- list of per-step success contributions rollback_penalty -- 0.30 if rollback active, else 0.0 drift_penalty -- 0.20 if biology/serial risk; 0.10 if localized_drift integrity_reward -- patch integrity contribution [0.0, 0.50] family_reward -- EMA long-horizon reward per family [0, 1] horizon_suggestion -- 'compress' | 'hold' | 'expand' net_credit --

clamped [0, 1] long_horizon_reward -- EMA net credit rollback_exposure -- EMA rollback penalty drift_exposure -- EMA drift penalty Horizon suggestion rules: hold : all other cases **Formula**: Family-level EMA statistics (alpha=0.10):, compress : rollback_penalty > 0 or drift_penalty >= 0.20, expand : family_reward >= 0.65 and net_credit >= 0.50 **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Per-tick outputs: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

TemporalSequenceStabilityMonitor

Scientific Role / Description: Integrates arc confidence, execution alignment, net credit, and route stability_state: 'degraded' : stability_score < 0.30 trend (computed over last 8 records): 'improving' : mean(last 4) > mean(first 4) + 0.05 'degrading' : mean(first 4) > mean(last 4) + 0.05 'stable' : within 0.05 threshold Observer only. Zero direct mutation. **Formula**: confidence into a smoothed stability_score via EMA (alpha=0.10)., 'stable' : stability_score >= 0.70, 'uncertain' : 0.50 <= stability_score < 0.70, 'unstable' : 0.30 <= stability_score < 0.50 **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Monitors temporal stability of execution sequences across Pack 2/3 outputs. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HorizonCompressionAdvisor

Scientific Role / Description: Generates advisory horizon compression recommendations. Advisory only. Does NOT directly mutate active_horizon, subgoal chains, steering bias, or any planning system state. Compression conditions (each independently sufficient): 1. rollback_penalty > 0.0 -- rollback penalties dominate 2. net_credit < 0.45 -- delayed credit below threshold 3. route_divergence > 0.35 -- route confidence below 0.65 recommended_horizon_delta: -2 : two or more conditions active -1 : exactly one condition active 0 : no compression needed Advisory only. Zero direct mutation. **Formula**: safe_relaunch_tick = current_tick + 8 * max(1, |delta|) **Hard Constants**: Derived from biology. **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

FailurePatternShield

Scientific Role / Description: Persists recurring failure motifs as global suppression priors. Failure motifs: 'low_credit' : net_credit < 0.30 'horizon_loop' : rollback_compressed and route_confidence < 0.40 shielded_family_patterns -- {family: {motif, suppression_confidence, ...}} suppression_confidence -- mean suppression confidence across shielded families shield_reason -- most frequent shield motif, or 'no_shield_active' Advisory only. Does NOT mutate routing, steering bias, or policy priors. is_shielded(family_sig) -> bool for downstream advisory queries. **Formula**: A family becomes shielded when the same failure motif occurs >= 3 times, 'rollback_heavy' : rollback_active == True, 'divergence_heavy' : comparator_state == 'diverged' **Hard Constants**: within a 16-tick sliding window (SHIELD_THRESHOLD=3, WINDOW=16). **Law Invariants**: Waking-only. Zero survival contamination. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Output: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

CrossSessionPlanMemory

Scientific Role / Description: Persists unfinished plan state across session boundaries. Reads the current execution state from Day 26 Pack 2/3 systems and marks which subgoal chains are incomplete, unresolved, or shielded so that the organism can resume them in a future session. Unfinished priority law (any condition -> 'high'): rollback_compressed : horizon was forcibly compressed net_credit < 0.50 : reward unresolved this session shielded_family_count > 0 : active failure shield present route_confidence < 0.60 and non-empty execution_steps : partial route completion goal_completion_prior * 0.40 + unfinished_priority_score * 0.30 + stability_score * 0.20 + failure_shield_safety * 0.10 Where: Safe resumption anchor: {tick, family, horizon, route_confidence} **Formula**: Continuity score formula (clamped [0, 1]):, goal_completion_prior = net_credit (OutcomeCreditAssigner), unfinished_priority_score = 1.0 if high, 0.50 if normal, stability_score = TemporalSequenceStabilityMonitor EMA score, failure_shield_safety = 1.0 - suppression_confidence (inverse shield), Updated every tick when continuity_score >= 0.60. **Hard Constants**: deque(maxlen=96) -- Day 27 cross-session continuity window. **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

GoalThreadPersistence

Scientific Role / Description: Maintains durable goal thread ownership across session boundaries. Per-family thread tracking (keyed by family_sig): thread_state : 'active' | 'unfinished' | 'complete' | 'shielded' thread_priority : float [0, 1] inherited_credit : EMA accumulated delayed reward continuation_prior : EMA from policy confidence Thread state classification: 'shielded' : family is shielded by FailurePatternShield 'active' : all other cases 0.50 * continuity_score + 0.30 * route_confidence + 0.20 * policy_confidence family_sig : active family thread_state : current state thread_priority : float [0, 1] resume_confidence : float [0, 1] inherited_credit : float [0, 1] continuation_prior : float [0, 1] thread_count : total tracked families **Formula**: resume_confidence : EMA float [0, 1] (alpha=0.10), 'unfinished' : unfinished_priority == 'high' (from CrossSessionPlanMemory), 'complete' : net_credit >= 0.70 and continuity_score >= 0.60, thread_priority formula (clamped [0, 1]):, resume_confidence: EMA(old, thread_priority, alpha=0.10) **Hard Constants**: deque(maxlen=96) -- Day 27 cross-session continuity window. **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

NarrativeProjectState

Scientific Role / Description: Constructs the organism's first autobiographical project narrative. Builds a persistent project self from active goal threads, unfinished arcs, artifact continuity, and patch execution state. Analogous to autobiographical goal continuity and narrative self-ownership in biological episodic memory (Tulving, 1983; Conway, 2005). Biology family exclusion law: Protected substrate families (neuron, synapse, homeostasis, hpa, cortisol, adenosine) are NEVER written into identity_threads. The organism does not claim narrative ownership of its own substrate. identity_threads: Up to 8 non-biology project arcs with highest thread_priority. Each entry: {family, state, priority, resume_confidence} narrative_state classification: 'blocked' : shielded_thread_count > 1 'resuming' : unfinished_thread_count > 0 (not blocked) 'consolidating': complete_thread_count > 0 (not blocked or resuming) 'advancing' : default -- active forward progress self_owned_project_arc: Plain-text summary of the primary owned arc (ASCII only). Format: "Arc: | State: | Priority: " tick : int narrative_state : str self_owned_project_arc : str active_arc_count : int unfinished_arc_count : int shielded_arc_count : int complete_arc_count : int artifact_continuity : int (artifact state memory len) patch_applied_count : int (patch executor latest applied_count) **Formula**: Standard bounded integration. **Hard Constants**: deque(maxlen=96) -- Day 27 cross-session continuity window. **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, identity_threads : list[dict] (biology excluded, max 8) **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ContinuityDriftMonitor

Scientific Role / Description: Tracks continuity score slope and detects drift warnings. Drift warning triggers when: Observes (reads only): - CrossSessionPlanMemory.continuity_score - CrossSessionPlanMemory.unfinished_priority - GoalThreadPersistence thread states - FailurePatternShield reactivation events - drift_state : 'stable' | 'degrading' | 'drifting' - drift_score : float [0, 1] — magnitude of recent decline - drift_warning : bool — True if drift threshold exceeded - slope_history : list[float] — recent slopes Strict observer. Zero mutation of any Pack 1 law. **Formula**: continuity score drops >= 0.20 within 8 ticks **Hard Constants**: deque(maxlen=64) — bounded memory. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs (advisory): **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ResumeConfidenceAdvisor

Scientific Role / Description: Estimates confidence in safe resumption of unfinished project arcs. continuity_score * 0.50 + thread_priority * 0.30 + narrative_coherence * 0.20 Clamped to [0, 1]. Observes (reads only): - CrossSessionPlanMemory.continuity_score - CrossSessionPlanMemory.safe_resumption_anchor - GoalThreadPersistence thread_priority - NarrativeProjectState narrative_state - resume_confidence : float [0, 1] - recommended_thread : str — family_sig of highest-confidence arc - resume_reason : str — human-readable rationale Strict observer. Zero mutation of any Pack 1 law. **Formula**: Resume confidence formula (frozen):, - resume_rank : int — rank among all threads (1 = best) **Hard Constants**: deque(maxlen=64) — bounded memory. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs (advisory): **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

NarrativeCoherenceProbe

Scientific Role / Description: Monitors narrative thread continuity, arc fragmentation, and coherence drift. Coherence scoring: - Medium for 'consolidating' or 'resuming' - Low for 'blocked' or frequent thread churn Fragmentation warning triggers when: - 3+ different thread states in last 8 ticks - OR repeated 'blocked' states (3+ in window) Observes (reads only): - NarrativeProjectState narrative_state - NarrativeProjectState identity_threads - GoalThreadPersistence thread states - coherence_score : float [0, 1] - coherence_state : 'coherent' | 'fragmented' | 'degraded' - fragmentation_warning : bool - state_transition_count : int Strict observer. Zero mutation of any Pack 1 law. **Formula**: - High if narrative_state == 'advancing' and few state transitions **Hard Constants**: deque(maxlen=64) — bounded memory. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs (advisory): **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

GoalArbitrationEngine

Scientific Role / Description: Arbitrate surviving non-biology threads using continuity score, unfinished bonus (+0.20), shield penalty (-0.25), resume confidence, narrative cycles). Produces a single selected_goal_thread per tick with suppressed lineage for archive ingestion. **Formula**: coherence, and dormant resurfacing bonus (+0.10 after >= 3 suppression **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

NarrativeConflictResolver

Scientific Role / Description: Detect and resolve autobiographical conflicts: duplicate unfinished arcs, collisions. Biology families unconditionally excluded. **Formula**: ownership loops (>= 4 consecutive wins), and narrative provenance **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: blocked-vs-advancing contradictions, safe-anchor contention, repeated **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PersistentIntentScheduler

Scientific Role / Description: Convert the winning arbitration thread into a bounded future agenda. Overflow threads go to a deferred candidates list. Biology families unconditionally excluded. **Formula**: Schedules <= 4 intents with relaunch ticks and safe anchors. **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SuppressedThreadArchive

Scientific Role / Description: Dormant autobiographical memory. Persists suppressed thread lineage: suppression count, last arbitration loss tick, starvation duration, and Biology families unconditionally excluded. **Formula**: resurfacing eligibility (>= 3 suppression cycles). **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DeferredResumptionGovernor

Scientific Role / Description: Dormant future governance. Absorbs deferred candidates from PersistentIntentScheduler, ages them, applies dormant fairness bonuses returns bounded resumption candidates. Biology families unconditionally excluded. **Formula**: from SuppressedThreadArchive, purges stale entries (>= 16 ticks), and **Hard Constants**: Derived from biology. **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MilestoneMemorySynthesizer

Scientific Role / Description: Compress dominant narrative arcs, successful executive recoveries, conflict resolutions, deferred resurfacing wins, continuity fallback recoveries, and project identity transitions into milestone-grade persistent memories that survive long dormancy periods. Milestone cap: 6 (MILESTONE_CAP). Biology family exclusion enforced at every entry point. thread_priority * 0.40 + continuity_score * 0.30 + resume_confidence* 0.20 + state_bonus * 0.10 (0.10 if unfinished, 0.05 if complete) synthesis_class values: 'dominant_arc' : highest-arbitration selected thread 'conflict_resolved' : thread surviving clean conflict state 'deferred_resurfaced' : thread appearing in deferred governor

'continuity_survivor' : all other qualified threads tick : int {family_sig, salience, synthesis_class, thread_state, continuity_score} milestone_salience : float -- mean salience across memories synthesis_reason : str synthesized_count : int **Formula**: Salience formula per thread: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, milestone_memories : list[dict] -- max 6, each: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

NarrativeArcMerger

Scientific Role / Description: Identity overlap uses token-level Jaccard similarity of family_sig strings. Suppressed lineage bonus (+0.10) applied when both families share archive history in SuppressedThreadArchive. MERGE_THRESHOLD: 0.65 (Rule 2 -- prevents false milestone fusion) tick : int secondary_family, overlap_score, merged_label} merge_confidence : float -- mean overlap of all merged pairs identity_overlap_score : float -- max overlap seen this tick merge_count : int **Formula**: Merge convergent dormant arcs when identity overlap >= MERGE_THRESHOLD. **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, merged_arcs : list[dict] -- each: {primary_family, **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ProjectIdentityContinuityIndex

Scientific Role / Description: Frozen long-gap identity coherence score integrating four orthogonal autobiographical signals into a single bounded index. milestone_salience * 0.35 + continuity_strength * 0.25 + arbitration_stability * 0.20 + recovery_resilience * 0.20 Clamped to [0, 1]. Sources: milestone_salience <- MilestoneMemorySynthesizer.milestone_salience continuity_strength <- CrossSessionPlanMemory.continuity_score arbitration_stability <- GoalArbitrationEngine.arbitration_score recovery_resilience <- ResumeConfidenceAdvisor.resume_confidence index_state classification: 'fragile' : identity_index < 0.40 **Formula**: Formula (frozen):, identity_index =, 'strong' : identity_index >= 0.70, 'moderate' : identity_index >= 0.40 **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DormantMilestoneReviver

Scientific Role / Description: Detect and revive dormant milestone-worthy arcs before purge. Revival criteria (Rule 3 -- all three must hold): Biology exclusion applies unconditionally. Deferred eligibility: flagged when family appears in DeferredResumptionGovernor.resumption_candidates. initial_priority * 0.60 + clamp(suppression_count / 10.0) * 0.40 tick : int revival_priority, suppression_count, initial_priority, deferred_eligible} revival_reason : str revival_priority : float -- max across revived milestones revival_count : int **Formula**: suppression_count >= SUPPRESSION_THRESHOLD (3), initial_priority >= PRIORITY_FLOOR (0.40), revival_priority formula: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, revived_milestones : list[dict] -- each: {family_sig, **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MilestoneDependencyGraph

Scientific Role / Description: Track milestone prerequisites, unresolved blockers, dormant dependencies, and deferred critical chains as a post-break scientific path memory. critical_milestone_path: milestones ordered by salience, revived dormant milestones appended after. blocked_milestones: path entries intersecting suppressed_threads or named in narrative conflict provenance. dependency_pressure: blocked_count / max(1, total_path_length). graph_state classification: 'blocked' : dependency_pressure > 0.50 Biology family exclusion enforced at all entry points. **Formula**: 'clear' : dependency_pressure == 0.0, 'pressured' : dependency_pressure <= 0.50 **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

BreakResumptionAnchor

Scientific Role / Description: Single post-break restart anchor representing the organism's most meaningful week-gap self-return story. Uniqueness law (Rule 4): exactly one primary break anchor per record. anchor_unique is always True -- enforced structurally, not conditionally. Biology family exclusion enforced at all entry points. break_anchor schema: primary_milestone : str or None critical_path : list[str] -- bounded to 6 entries first_resumption_thread : str or None unresolved_question : str -- ASCII re_entry_rationale : str -- ASCII identity_index * 0.50 + milestone_salience * 0.50 Clamped to [0, 1]. **Formula**: restart_priority formula: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: stable_safe_anchor : dict or None **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ResumptionPriorityEngine

Scientific Role / Description: Convert break anchor, milestone salience, dependency pressure, arbitration stability, and continuity strength into ordered restart priorities for the post-break first tick. Blocker-first override (Rule 2): to the front of restart_priorities unconditionally. RESTART_CAP: 5 -- no post-break overwhelm. identity_index * 0.50 + continuity_score * 0.30 + milestone_salience * 0.20 Clamped to [0, 1]. tick : int {family_sig, priority_score, priority_rank, reason, is_blocker} priority_reason : str resume_confidence : float **Formula**: If dependency_pressure >= 0.70, blocked milestones are promoted, resume_confidence formula: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, restart_priorities: list[dict] -- max 5, ordered, each: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MilestoneBlockerResolver

Scientific Role / Description: Detect and order critical blocked milestones, deferred unresolved chains, and dormant dependency failures for blocker-first resolution. primary_blocker: first item in blocked_milestones (highest-pressure). blocker_resolution_path: [primary_blocker] + up to 3 non-biology deferred candidates from DeferredResumptionGovernor. dependency_unwind_order: critical_milestone_path[:5]. resolver_state classification: 'resolving' : dependency_pressure > 0.0 and < 0.70 Biology family exclusion enforced at all entry points. tick : int primary_blocker : str or None blocker_resolution_path : list[str] dependency_unwind_order : list[str] blocker_pressure : float

resolver_state : str **Formula**: 'critical' : dependency_pressure >= 0.70, 'unblocked' : dependency_pressure == 0.0 **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MomentumRecoveryPlanner

Scientific Role / Description: Design the first 5-step momentum restoration arc for post-break re-entry, analogous to warm-start cognitive sequencing observed in episodic memory reinstatement (Tulving, 1985). RESTART_CAP: 5 -- no overwhelm. Step assembly order: Step 1: primary_blocker resolution (if any blocker present) Steps 2-3: top restart priorities (milestone reconnections) Step 4: confidence rebuild via safe anchor Step 5: executive handoff to first resumption thread resume_confidence * 0.60 + milestone_salience * 0.40 Clamped to [0, 1]. action_type values: 'blocker_resolution', 'milestone_reconnect', 'confidence_rebuild', 'executive_handoff', 'low_friction_win' Biology family exclusion enforced at all entry points. tick : int {step, action_type, target_family, description, confidence} recovery_confidence: float warm_start_reason : str **Formula**: recovery_confidence formula: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, momentum_sequence : list[dict] -- max 5, each: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DormantFinalRevivalGate

Scientific Role / Description: Allow exactly one final milestone resurrection attempt before purge (Rule 3: one revival attempt per break cycle). Target: highest revival_priority dormant milestone from DormantMilestoneReviver.revived_milestones. revival_priority * 0.70 + (1.0 - dependency_pressure) * 0.30 Clamped to [0, 1]. gate_state classification: 'no_candidates' : no revived milestones available Biology family exclusion enforced unconditionally. tick : int final_revival_target: str or None revival_viability : float revival_reason : str gate_state : str **Formula**: revival_viability formula:, 'viable' : revival_viability >= 0.50, 'low_viability' : revival_viability >= 0.20 and < 0.50 **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

StaleNarrativePruner

Scientific Role / Description: Mark dormant arcs as safe_to_prune using four-criterion hygiene rule (Rule 4 -- all criteria must hold simultaneously): 1. no milestone contribution : not in critical_milestone_path 2. no critical dependency : not in blocked_milestones 4. identity overlap < 0.40 : Jaccard vs all milestone family_sigs Biology family exclusion enforced unconditionally. prune_confidence: fraction of criteria met per candidate (mean), producing a signal for downstream decisiveness. For safe_to_prune candidates, prune_confidence is always 1.0 per entry. tick : int prune_candidates : list[str] -- safe_to_prune family_sigs prune_confidence : float -- mean fraction of criteria met prune_reason : str pruned_count : int **Formula**: 3. suppression age >= 16 : starvation_duration >= STALE_THRESHOLD **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

WeekGapRestartSequence

Scientific Role / Description: Construct the canonical 7-day return script -- the organism's first week-gap action playbook integrating all Phase B signals. restart_script schema: step_1_blocker : str or None -- primary blocker family final_revival_candidate : str or None -- from DormantFinalRevivalGate first_safe_milestone : str or None -- top non-blocked milestone first_narrative_followup : str -- ASCII follow-up summary recovery_confidence * 0.50 + revival_viability * 0.20 + restart_priority * 0.30 Clamped to [0, 1]. Biology family exclusion enforced unconditionally. tick : int week_gap_confidence : float restart_rationale : str -- ASCII **Formula**: week_gap_confidence formula: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: momentum_sequence : list[dict] -- from MomentumRecoveryPlanner, Outputs per record:, restart_script : dict **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HypothesisPersistenceMemory

Scientific Role / Description: Store unresolved scientific questions that must survive week-long silence as living curiosity vectors. HYPOTHESIS_CAP: 5 (Rule 1 -- no scientific clutter). EEIL_PRIORITY_BONUS: +0.15 (Rule 2) for hypotheses touching sparse law, horizon compression, continuity-before-localization, biology exclusion edge cases, or energy/performance tradeoffs. Hypothesis seeds are fixed templates. Each seed is scored per tick from organism state and the top-5 by salience are retained as active_hypotheses. base_score * 0.40 + identity_index * 0.30 + week_gap_confidence * 0.30 + eeil_priority_bonus (0.15 if eeil_tagged, else 0.0) Clamped to [0, 1]. persistence_class values: 'publication_risk' : linked to paper claim seed 'dormant_risk' : linked to dormant revival uncertainty 'standard' : all other Biology runtime token exclusion enforced. Conceptual biology hypotheses are permitted (Rule 5). tick : int {hypothesis_id, question_text, salience, eeil_tagged, persistence_class} hypothesis_salience : float -- mean salience persistence_reason : str **Formula**: salience formula per hypothesis:, 'eeil_critical' : eeil_tagged and salience >= 0.60 **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, active_hypotheses : list[dict] -- max 5, each: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

OpenQuestionGraph

Scientific Role / Description: Build dependency links between unresolved hypotheses and milestone blockers, continuity anomalies, and restart script gaps. Each active hypothesis is linked to milestone families whose token sets share at least one token with the hypothesis question text. critical_unknown (high cross-linkage). resolution_dependencies: critical path families that co-occur with critical_unknowns' linked milestones. Clamped to [0, 1]. Biology runtime token exclusion enforced in milestone linking. tick : int linked_milestones, dependency_type, link_count} resolution_dependencies: list[str] -- milestone families that block critical unknowns graph_density : float **Formula**: A question with >= 2 milestone links is classified as a, graph_density = link_count / max(1, node_count), critical_unknowns : list[str] -- question_ids with >= 2 links **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, question_graph : list[dict] -- each: {question_id, **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

EEILTensionTracker

Scientific Role / Description: Track high-value unresolved EEIL tensions using a frozen three-axis tension score. energy_tradeoff * 0.40 + continuity_risk * 0.30 + publication_value * 0.30 Clamped to [0, 1]. Sources: (from ResumptionPriorityEngine) (from ProjectIdentityContinuityIndex) (from MilestoneMemorySynthesizer) Each EEIL hypothesis receives its individual tension score. paper_relevance: mean publication_value across all EEIL tensions. tension_priority: maximum tension_score across all EEIL tensions. Biology runtime token exclusion enforced. tick : int energy_tradeoff, continuity_risk, publication_value, paper_relevant} tension_priority : float -- max tension_score paper_relevance : float -- mean publication_value of EEIL tensions **Formula**: Tension score formula (frozen):, tension_score =, energy_tradeoff = 1.0 - resume_confidence, continuity_risk = 1.0 - identity_index, publication_value = milestone_salience, Only hypotheses flagged eeil_tagged=True are evaluated. **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, eeil_tensions : list[dict] -- each: {hypothesis_id, tension_score, **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ExperimentGapAnalyzer

Scientific Role / Description: Detect missing validation sweeps, dormant test gaps, and post-break experiment priorities. Experiment gap seeds are fixed templates scored per tick from organism research_catalyst rank 1. 'validation_sweep' : week_gap_confidence * 0.70 + identity_index * 0.30 'boundary_test' : revival_priority * 0.60 + continuity_score * 0.40 'drift_test' : (1 - dep_pressure) * 0.70 + identity_index * 0.30 'edge_case' : hypothesis_salience * 0.60 + identity_index * 0.40 'longitudinal' : week_gap_confidence * 0.50 + identity_index * 0.50 experiment_priority: gap_id of highest gap_confidence. Biology runtime token exclusion enforced. tick : int {gap_id, description, gap_confidence, gap_type, experiment_priority_rank} gap_confidence : float -- max gap_confidence experiment_priority : str -- top gap_id **Formula**: state. Gaps with gap_confidence >= 0.70 (Rule 3) surface into, gap_confidence formula varies by gap_type: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, critical_experiment_gaps: list[dict] -- max 5, each: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PaperClaimContinuityBridge

Scientific Role / Description: Track future publication claims that must persist across the break. CLAIM_SEEDS: fixed templates representing monograph claims, EEIL paper arguments, Day 28 continuity milestone claims, and open proofs. Claim confidence is scored from organism state per tick. Claims with confidence < CONFIDENCE_FLOOR (0.30, Rule 4) are dropped. identity_index * 0.35 + hypothesis_salience * 0.35 + week_gap_confidence * 0.30 Clamped to [0, 1]. claim_risk per claim: 1.0 - confidence publication_next_step: ASCII summary of highest-confidence claim action. Biology runtime token exclusion enforced. tick : int {claim_id, claim_text, confidence, section, claim_risk, publication_ready} claim_risk : float -- mean(1 - confidence) across claims publication_next_step: str -- ASCII **Formula**: claim_confidence formula:, publication_ready: confidence >= 0.60 **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, persistent_claims : list[dict] -- each: **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PostBreakResearchCatalyst

Scientific Role / Description: Select the single highest-value scientific unknown to pull the first resumed day back into active research (uniqueness law). Selection priority (Rule 3): 2. Top EEIL tension by tension_score 3. Top hypothesis by salience tension_priority * 0.40 + gap_confidence * 0.30 + hypothesis_salience * 0.30 Clamped to [0, 1]. research_catalyst schema: rank_1_question : str -- ASCII linked_experiment_gap : str or None linked_milestone_blocker : str or None publication_value : float restart_scientific_rationale: str -- ASCII Biology runtime token exclusion enforced. tick : int catalyst_confidence: float curiosity_reason : str -- ASCII **Formula**: 1. Experiment gap with gap_confidence >= 0.70 -> rank_1 (forced), catalyst_confidence formula: **Hard Constants**: deque(maxlen=96). **Law Invariants**: Advisory only. Zero survival contamination. Waking-only. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Outputs per record:, research_catalyst : dict **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ResearchAgendaComposer

Scientific Role / Description: Ranks and composes the post-break research agenda from catalyst, claims, + PUBLICATION_BONUS * publication_flag [+0.10] + EEIL_TRADEOFF_BONUS * eeil_tradeoff_flag [+0.05] **Formula**: (frozen law); PUBLICATION_BONUS=+0.10; at least one EEIL-tradeoff item., Scoring formula:, item_score = base_score **Hard Constants**: and experiment gaps. Enforces: AGENDA_CAP=5; catalyst forced rank_1, Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ExperimentPriorityScheduler

Scientific Role / Description: Schedules the experiment queue from the ranked research agenda, + gap_confidence * 0.30 + (1 - dep_pressure)* 0.20 **Formula**: Priority formula:, priority_score = agenda_rank_score * 0.50, scheduler_confidence = agenda_confidence * 0.70 + gap_confidence * 0.30 **Hard Constants**: experiment gaps, and blocker state. QUEUE_CAP=5., Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ValidationSweepForecaster

Scientific Role / Description: Forecasts a bounded validation sweep (EEIL sparse discipline). + tension_score * 0.30 + hypothesis_salience * 0.20 **Formula**: SWEEP_DEPTH_CAP = 6 — hard cap, never exceeded (Rule 3)., forecast_base = agenda_confidence * 0.50, step_conf[i] = max(0, forecast_base * (1 - 0.08 * i)) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PaperRoadmapSynthesizer

Scientific Role / Description: Synthesizes publication roadmap sections from claims, agenda, hypotheses. + agenda_confidence * 0.35 + hypothesis_salience * 0.25 **Formula**: section_score = claim_confidence * 0.40, next_claim_section: highest-scoring section with claim_confidence >= 0.30. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

EEILTradeoffWorkbench

Scientific Role / Description: study (Rule 5 enforcement). Four fixed study axes: sweep_depth vs sparse_law milestone_cap vs restart_fidelity hypothesis_cap vs scientific_breadth continuity_robustness vs bounded_memory + (1 - identity_index) * 0.30 + milestone_salience * 0.30 **Formula**: Executes explicit EEIL tradeoff studies. Mandatory — always >= 1 active, study_score = tension_score * 0.40, eeil_gain_projection = mean(study_score) across all studies. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PostBreakExperimentLaunchpad

Scientific Role / Description: Selects the single highest-value post-break experiment (uniqueness law). Synthesizes agenda_rank_1 + sweep_plan + paper_implication + eeil_gain + + forecast_confidence * 0.25 + eeil_gain_projection * 0.25 + (1 - dep_pressure) * 0.15 **Formula**: launch_confidence = agenda_confidence * 0.35 **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: milestone_dependency into one actionable launch_experiment dict. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

WeekBreakFutureSimulator

Scientific Role / Description: Canonical 7-day silent incubation simulation. Projects how milestone memory, hypothesis salience, launch confidence, paper maturation, and blocker pressure will shift across the week-long hiatus before it occurs. Stale hypothesis decay (Rule 3): salience < 0.40, no pub coupling, no critical dep → -0.10 projected_milestone_salience * 0.35 + projected_hypothesis_salience * 0.30 + launch_confidence * 0.20 + (1 - dep_pressure) * 0.15) **Formula**: SIMULATION_HORIZON = 7 (frozen, Rule 1), Milestone strengthening law (Rule 2): salience >= 0.70 → +0.05, simulation_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HypothesisDriftForecaster

Scientific Role / Description: Forecasts hypothesis salience shifts across the 7-day break: rising EEIL tensions strengthen EEIL-tagged hypotheses, publication-coupled hypotheses are resilient (no decay), stale hypotheses (salience < 0.40, no pub coupling) decay -0.10, **Formula**: dormant hypotheses (0.30 <= salience < 0.50) may resurface., drift_pressure = fraction of hypotheses that decay **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MilestoneSalienceProjector

Scientific Role / Description: Projects post-break milestone salience applying: - Merge bonus: milestone in a merged arc → +0.03 - Break-anchor reinforcement: milestone in anchor families → +0.02 - Dormant arcs: unchanged unless above thresholds **Formula**: - Strengthening law (Rule 2): salience >= 0.70 → +0.05, salience_delta = projected_mean - original_mean **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AgendaReorderPredictor

Scientific Role / Description: EEIL study promotion: eeil_tradeoff_flag → +0.07 Blocker escalation: blocked milestone → -0.05 Catalyst replacement risk: catalyst_confidence < 0.50 → second item may displace rank_1 **Formula**: launch value delta >= 0.15 triggers a reorder (Rule 4)., reorder_delta = max absolute delta seen across all items. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Predicts post-break agenda reordering from EEIL promotion, blocker, escalation, and catalyst replacement risk. Any item with predicted **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PaperClaimMaturationEstimator

Scientific Role / Description: Estimates post-break claim maturation. For each persistent claim: + agenda_confidence * 0.35 + forecast_confidence * 0.25 **Formula**: maturation_projection = claim_confidence * 0.40, If maturation_projection >= 0.60: flagged 'likely_publication_ready' (Rule 5)., maturation_projection (class-level) = mean across all claims. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PostHiatusValueConvergence

Scientific Role / Description: Synthesizes the single most valuable projected post-break scientific state. Uniqueness law enforced: exactly one convergence record per tick. post_hiatus_value keys: top_matured_claim : highest maturation_projection ready claim strongest_milestone : highest projected salience milestone reordered_rank1_experiment : forecast_agenda[0] label strongest_hypothesis : highest proj_salience in hypothesis_forecast projected_eeil_gain : eeil_gain_projection + salience_delta_norm * 0.20 + (1 - drift_pressure) * 0.25 + eeil_gain_projection * 0.25 **Formula**: convergence_confidence = maturation_projection * 0.30, salience_delta_norm = clamp(salience_delta / 0.10) (normalised to [0,1]) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ForecastReliabilityAssessor

Scientific Role / Description: Evaluates incubation forecast reliability along four axes: simulation_confidence, convergence_confidence, drift_pressure, maturation_projection. + convergence_confidence * 0.30 + (1 - drift_pressure) * 0.20 + maturation_projection * 0.15 Fragile forecasts: components whose individual contribution to forecast error exceeds their weight (i.e. value < 0.50). Capped at FRAGILE_CAP. **Formula**: FRAGILE_CAP = 5 (Rule 1 -- no meta clutter), False confidence penalty (Rule 2): if simulation_confidence >= 0.80

AND, fragility_score >= 0.60 → reliability_score -= 0.15 (clamped)., reliability formula:, raw_reliability = simulation_confidence * 0.35, if sim_conf >= 0.80 and fragility_score >= 0.60:, raw_reliability -= FALSE_CONFIDENCE_PENALTY (0.15), reliability = clamp(raw_reliability) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

BreakStrategyMetaLearner

Scientific Role / Description: Learns from the incubation simulation: which continuity layers mattered most, which hypothesis decays were too aggressive. meta_break_laws: fixed law templates scored by reliability and convergence. + (1 - drift_pressure) * 0.20 **Formula**: law_score = forecast_reliability * 0.50 + convergence_confidence * 0.30, strategy_delta = law_score - previous_law_score (0 on first tick)., Learning: top law = max(law_score). **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: which phases over-predicted value, which milestone projections were stable, **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ContinuityFragilityMapper

Scientific Role / Description: Maps fragile structural points: weak anchors, blocker overdependence, stale prune risks, catalyst overfitting, roadmap brittleness. (1 - restart_priority) * 0.25 + dep_pressure * 0.25 + drift_pressure * 0.20 + (1 - agenda_conf) * 0.15 + (1 - reliability) * 0.15) **Formula**: FRAGILE_CAP = 5 (Rule 1 joint cap)., fragility_score = clamp(, risk_reason: 'score=X fragile_count=Y' **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MilestoneStrengthCauseAnalyzer

Scientific Role / Description: Explains why milestones strengthened. Causality may only be one of five frozen sources (Rule 3): merge_bonus, anchor_reinforcement, dormant_revival, publication_coupling, blocker_resolution_path. No arbitrary causal hallucination. salience_delta / 0.10 * 0.50 (normalized delta) + reliability * 0.30 + (1 - fragility_score)* 0.20) **Formula**: cause_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AgendaAssumptionAuditor

Scientific Role / Description: Audits hidden dependency assumptions, sweep overconfidence, launchpad overfit, and weak EEIL tradeoff projections. (clamped, measures how much the item depends on unresolved blockers) + agenda_confidence * 0.35 + (1 - fragility_score)* 0.25 **Formula**: DEPENDENCY_SENSITIVITY_THRESHOLD = 0.70 → 'fragile_assumption' (Rule 4), dependency_sensitivity per item = proj_score * dep_press + (1 - agenda_conf), audit_confidence = forecast_reliability * 0.40 **Hard Constants**: assumption_risks: list of fragile assumption dicts (cap FRAGILE_CAP=5)., Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

FutureHiatusPolicyRefiner

Scientific Role / Description: Chooses the single best refined strategy for the next long break (uniqueness law, Rule 5). Stores fragile assumptions, best continuity layer, safest agenda breadth, ideal milestone cap, optimal hypothesis retention, and policy rationale. + (1 - fragility_score) * 0.25 + cause_confidence * 0.25 + audit_confidence * 0.20 next_hiatus_policy keys (fixed): fragile_assumptions : list of top fragile assumption ids best_continuity_layer : highest reliability source safest_agenda_breadth : int (5 - fragile_count, min 1) optimal_hypothesis_cap : int (5 if drift_pressure < 0.30 else 3) policy_rationale : str **Formula**: policy_confidence = forecast_reliability * 0.30, ideal_milestone_cap : int (6 if delta >= 0 else 5) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HiatusIdentityProjector

Scientific Role / Description: Projects how the organism's project identity will shift across the 7-day hiatus: milestone identity changes, curiosity centre shifts, agenda priority influence, EEIL optimisation identity pull. simulation_confidence * 0.35 + milestone_salience * 0.25 + agenda_confidence * 0.20 + eeil_gain * 0.20) future_identity_state keys: dominant_milestone_family : highest projected salience family agenda_rank_1_identity : catalyst label (identity-defining) eeil_identity_pull : eeil_gain_projection curiosity_center : top hypothesis id projected_horizon_days : 7 (frozen) Rule 5: if forecast_reliability < 0.50, flag 'tentative_shift_only'. **Formula**: identity_projection_conf = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

NarrativeSelfEvolutionModel

Scientific Role / Description: Models the autobiographical scientific story after silence: which narrative becomes dominant, which thread becomes 'the self', which dormant meaning resurfaces stronger. (0.0 on first tick) evolved_self_narrative: single dominant narrative record { thread_id, narrative_label, narrative_score, is_dominant, tentative } + convergence_confidence * 0.30 + (1 - drift_pressure) * 0.20 Rule 5: tentative if forecast_reliability < 0.50. **Formula**: narrative_shift = projected_identity_conf - prev_identity_conf, narrative_score = identity_projection_conf * 0.50 **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ScientificPersonaStabilityIndex

Scientific Role / Description: identity_projection_conf * 0.35 + forecast_reliability * 0.25 + milestone_meaning * 0.20 + curiosity_alignment * 0.20) stability_risk: 'at_risk' if persona_stability < 0.60, else 'stable'. milestone_meaning: mean meaning_weight of reweighted_milestones. curiosity_alignment: mean normalised curiosity priority. Rule 5: tentative flag propagated if forecast_reliability < 0.50. **Formula**: Frozen formula:, persona_stability = clamp(, Rule 4: persona_stability >= 0.60 must be preserved. **Hard Constants**: Biology

tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MilestoneMeaningReweighter

Scientific Role / Description: Re-evaluates why milestones matter and which define selfhood. meaning_weight per milestone: selfhood_class: 'utility_only' : meaning_weight < 0.40 'active_meaning' : otherwise **Formula**: Rule 2: any milestone with projected_salience_delta >= 0.10, receives meaning_weight_bonus = +0.05., base_weight = projected_salience (clamped), if delta >= 0.10: base_weight += MEANING_WEIGHT_BONUS (0.05), meaning_shift = projected_mean_weight - original_mean_weight, 'selfhood_defining' : meaning_weight >= 0.75 **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

CuriosityVectorRealigner

Scientific Role / Description: Re-ranks curiosity vectors for the post-break self. Rule 3: vectors tied to rank_1 catalyst, launch_experiment, or curiosity_vector priority: clamped to [0, 1] Rule 5: tentative if forecast_reliability < 0.50. **Formula**: top matured claim receive identity_priority_bonus = +0.10 (frozen)., base_priority = hypothesis_salience (from forecast), if tied to rank_1 / launch / matured_claim: += PRIORITY_BONUS (0.10), realignment_delta = projected_mean_priority - base_mean_priority **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PostBreakSelfContinuityAnchor

Scientific Role / Description: Chooses the single identity-defining self state to return from hiatus. Rule 1: exactly one anchor per tick (uniqueness law). + identity_projection_conf * 0.30 + cause_confidence * 0.20 + policy_confidence * 0.15 self_continuity_anchor keys: dominant_narrative : evolved_self_narrative['narrative_label'] strongest_milestone_meaning : highest meaning_weight milestone family top_curiosity_pull : aligned_curiosity_vectors[0]['id'] launch_experiment_influence : launch_experiment['agenda_rank_1'] refined_hiatus_policy : next_hiatus_policy['policy_rationale'] selfhood_rationale : str Rule 5: tentative if forecast_reliability < 0.50. **Formula**: anchor_confidence = persona_stability * 0.35 **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

LongGapDoctrineSynthesizer

Scientific Role / Description: Compresses Day 28 phase laws, restart doctrines, incubation rules, selfhood anchors, and catalyst priorities into ranked doctrine laws. in 2+ phase-level validations (tracked by fixed provenance tags). + REUSABLE_LAW_BONUS * reusable_flag anchor_confidence * 0.35 + persona_stability * 0.25 + policy_confidence * 0.20 + forecast_reliability * 0.20) **Formula**: REUSABLE_LAW_BONUS = +0.10 (Rule 2): law receives bonus if evidenced, doctrine_law_score = base_score, synthesis_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HiatusLawArchive

Scientific Role / Description: Persists reusable break laws with provenance, success frequency, phase reuse evidence, and protocol confidence. mean(scored_laws) * 0.60 + synthesis_confidence * 0.40) archived_hiatus_laws: top laws from doctrine_laws that are reusable (may appear in all future hiatuses). **Formula**: Rule 2: reusable_law_bonus = +0.10 for laws evidenced in 2+ phases., law_reuse_score = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

BreakOutcomePrincipleExtractor

Scientific Role / Description: Extracts strongest break success patterns, failure avoidances, and robust restart doctrines from archived laws and anchor state. law_reuse_score * 0.40 + anchor_confidence * 0.30 + persona_stability * 0.30) break_principles: fixed extraction templates scored by principle_confidence. **Formula**: principle_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DoctrineConflictResolver

Scientific Role / Description: Resolves conflicts between doctrine rules: cap law conflicts, agenda incubation disagreements. fragility_score * 0.40 + (1 - synthesis_confidence) * 0.30 + drift_pressure * 0.30) resolved_doctrine: list of {id, law, score, conflict_class} **Formula**: Rule 3: any doctrine rule with conflict_score >= 0.50 → 'conditional_only'., conflict_score = clamp(, conflict_class: 'conditional_only' if score >= 0.50, else 'global_law'. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: breadth contradictions, milestone vs hypothesis pressure, restart vs **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

FutureSilenceProtocol

Scientific Role / Description: Builds the generalized future-break operating protocol with three modes: short_break (<3 days), week_break (3-10 days), long_hiatus (>10 days). principle_confidence * 0.40 + synthesis_confidence * 0.35 + (1 - conflict_score) * 0.25) future_silence_protocol keys: restart_aggressiveness : float (inverse of conflict_score) science_catalyst_preservation : 'always_freeze' **Formula**: Rule 4: protocol_confidence >= 0.65 required before promotion., protocol_confidence = clamp(, milestone_retention : 'strict' if synthesis_conf >= 0.70 else 'relaxed', promoted : bool (protocol_confidence >= 0.65) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: short_break_mode : dict (milestone_cap, restart_mode), week_break_mode : dict (milestone_cap, hypothesis_cap, sweep_depth), long_hiatus_mode : dict (milestone_cap, hypothesis_cap, agenda_breadth) **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

InstitutionalContinuityKernel

Scientific Role / Description: Chooses the single canonical reusable silence doctrine (Rule 1). Uniqueness law: exactly one dominant kernel per tick. Rule 5: kernel must carry forward all six priors: milestone_cap_prior, restart_cap_prior, hypothesis_cap_prior, agenda_sparse_prior, forecast_horizon_doctrine, self_anchor_uniqueness. synthesis_confidence * 0.30 + protocol_confidence * 0.25 + principle_confidence * 0.25 + anchor_confidence * 0.20) **Formula**: kernel_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

SilenceDurationClassifier

Scientific Role / Description: Classifies the current silence duration into a canonical rung of the frozen duration ladder [1,3,7,14,30,90] and assigns a duration class label, duration confidence, and classification reason. Duration ladder (Rule 1 -- frozen): [1, 3, 7, 14, 30, 90] Class labels: 1 -> micro_break 3 -> short_gap 7 -> week_gap 14 -> long_gap 30 -> semester_gap 90 -> seasonal_hiatus **Formula**: duration_confidence = clamp(kernel_confidence * 0.60 + reliability * 0.40) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

TemporalContinuityScaler

Scientific Role / Description: Scales milestone caps, restart depth, hypothesis preservation, agenda breadth, and doctrine strictness according to the classified duration rung. **Formula**: Scaling table (all ms_cap = 6, milestone identity preserved):, micro_break : restart_depth=1, hyp_pres=0.95, breadth=5, strict=0.90, short_gap : restart_depth=2, hyp_pres=0.90, breadth=5, strict=0.85, week_gap : restart_depth=3, hyp_pres=0.80, breadth=4, strict=0.75, long_gap : restart_depth=4, hyp_pres=0.70, breadth=4, strict=0.65, semester_gap : restart_depth=5, hyp_pres=0.55, breadth=3, strict=0.55, seasonal_hiatus: restart_depth=6, hyp_pres=0.40, breadth=3, strict=0.45, scaling_delta = strictness - BASE_STRICTNESS (week_gap baseline = 0.75) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AdaptiveRestartAggression

Scientific Role / Description: Chooses the restart mode and aggression score from the classified duration rung. Rule 2 (frozen mapping): aggression_class: 'gentle' | 'standard' | 'aggressive' **Formula**: 1-3 (gentle) -> gentle_restart, score=0.15/0.25, 7-14 (standard) -> normal_restart, score=0.50/0.60, 30-90 (aggressive)-> blocker_first_aggressive, score=0.80/0.90 **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MilestoneRetentionCurve

Scientific Role / Description: Models milestone retention decay across the duration ladder. is forbidden). Retention values are clamped to this floor. Retention table: 1 -> 1.00 3 -> 0.95 7 -> 0.85 14 -> 0.75 30 -> 0.60 90 -> 0.45 **Formula**: Rule 3: retention_floor >= 0.40 at all durations (identity collapse, All values >= RETENTION_FLOOR (0.40). **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HypothesisHalfLifeModel

Scientific Role / Description: Projects hypothesis survival across the duration ladder. Rule 4: any hypothesis tagged with publication coupling, EEIL tension, survival_pressure score. Base half-life (days) per duration rung: 1 -> 30d 3 -> 25d 7 -> 20d 14 -> 15d 30 -> 10d 90 -> 7d **Formula**: or critical dependency receives HALF_LIFE_BONUS = +0.15 added to its, survival_pressure = clamp(surviving/total + half_life_bonus_if_tagged) **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AgendaBreadthRegulator

Scientific Role / Description: Regulates agenda width, sweep depth, experiment queue breadth, and paper roadmap breadth from the temporal continuity scaling profile. regulated_agenda keys: **Formula**: breadth_confidence = clamp(dur_conf * 0.60 + kernel_conf * 0.40) **Hard Constants**: agenda_width, sweep_depth (cap=6 from Phase F), experiment_queue (cap=5),, roadmap_breadth (cap=5), duration_class., Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

LongitudinalIdentityStabilizer

Scientific Role / Description: Ensures scientific self-continuity across all timescales by integrating persona stability, milestone retention, and kernel confidence into a single longitudinal identity stability score. persona_stability * 0.40 + retention_floor * 0.30 + kernel_confidence * 0.30) longitudinal_risk: < 0.50 -> 'high' **Formula**: identity_stability = clamp(, >= 0.70 -> 'low', >= 0.50 -> 'moderate' **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

UniversalHiatusPolicyKernel

Scientific Role / Description: per tick. Stores: duration_ladder (Rule 1 frozen), retention_floor (Rule 3), aggression_mapping (Rule 2 frozen), half_life_doctrine (Rule 4), agenda_breadth_law, identity_floor, universal_rationale. identity_stability * 0.30 + retention_floor * 0.25 + duration_confidence * 0.25 + breadth_confidence * 0.20) **Formula**: kernel_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential

chain execution. **Serialization Fields**: Synthesizes all Phase I temporal outputs into a single duration-agnostic, universal silence doctrine. Rule 5 (uniqueness): exactly one kernel dict **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ProjectPivotClassifier

Scientific Role / Description: Classifies the type of project pivot from the frozen canonical ladder. Rule 1 (frozen pivot ladder): ["rename", "scope_expand", "scope_split", "domain_shift", "productization", "research_fork"] kernel_confidence * 0.55 + duration_confidence * 0.45) Pivot is inferred from doctrine kernel: if agenda_breadth_law < 3 -> scope_split; **Formula**: pivot_confidence = clamp(, if conflict_class == 'conditional_only' -> domain_shift; default -> rename. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DoctrineTransferMapper

Scientific Role / Description: Maps silence doctrine, milestone caps, restart aggression, retention curves, and hypothesis half-life across a project pivot. pivot_confidence * 0.40 + kernel_confidence * 0.35 + (1 - conflict_penalty) * 0.25) transferred_doctrine carries all universal_hiatus_kernel fields plus pivot_type and transfer_confidence. **Formula**: Rule 2 (frozen floor): doctrine_transfer >= 0.60 always., transfer_confidence = clamp(, conflict_penalty = 0.20 if pivot_type in {'domain_shift','research_fork'} else 0.05 **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

CrossDomainMilestoneTranslator

Scientific Role / Description: Translates milestone meaning, project anchors, continuity kernels, dormant thread value, and break doctrines across the pivot domain. Rule 3: any translated milestone with semantic_overlap < 0.50 must be marked 'translation_tentative'. transfer_confidence * 0.50 + pivot_confidence * 0.30 + anchor_confidence * 0.20) Each translated milestone carries: {id, translated_meaning, semantic_overlap, tentative_flag} **Formula**: translation_integrity = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HypothesisMigrationBridge

Scientific Role / Description: Migrates unresolved EEIL tensions, publication claims, roadmap gaps, and experimental unknowns into the new domain. Rule 4: any migrating hypothesis linked to EEIL law, publication claim, survival_pressure * 0.40 + transfer_confidence * 0.35 + (1 - drift_pressure) * 0.25 + migration_bonus_if_tagged) migrated_hypotheses: list of {id, migration_status, migration_score} **Formula**: or critical experiment lineage receives migration_bonus = +0.15., migration_pressure = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AgendaForkResolver

Scientific Role / Description: Resolves experiment queue conflicts, roadmap splits, validation sweep duplication, and competing catalyst branches across the pivot. (1 - transfer_confidence) * 0.40 + migration_pressure * 0.30 + (1 - translation_integrity) * 0.30) resolved_fork_agenda: picks the dominant fork branch as the agenda. **Formula**: fork_conflict_score = clamp(, If fork_conflict_score >= 0.50: dominant branch = 'blocker_first', else: dominant branch = 'catalyst_first' **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

IdentityThreadPorter

Scientific Role / Description: Carries autobiographical selfhood, milestone meaning, curiosity vectors, and persona stability priors into the new project lineage. anchor_confidence * 0.35 + persona_stability * 0.30 + transfer_confidence * 0.20 + translation_integrity * 0.15) ported_identity_threads: list of {thread, value, ported_status} Thread survival: value > 0.50 -> 'strong_port'; value > 0.30 -> 'weak_port'; else -> 'dormant_seed' **Formula**: port_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ResearchLineagePreserver

Scientific Role / Description: Preserves EEIL thesis ancestry, validation lineage, monograph evolution chain, and continuity doctrine provenance across the pivot. port_confidence * 0.40 + transfer_confidence * 0.35 + migration_pressure * 0.25) lineage_chain: list of 5 fixed provenance records ordered by strength. **Formula**: lineage_strength = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MetaProjectContinuityKernel

Scientific Role / Description: doctrine. Rule 5 (uniqueness): exactly one meta_project_kernel per tick. Stores: pivot_ladder (Rule 1 frozen), doctrine_transfer_floor (Rule 2), milestone_translation_priors (Rule 3), hypothesis_migration_law (Rule 4), lineage_preservation_chain, meta_project_rationale. lineage_strength * 0.30 + port_confidence * 0.25 + transfer_confidence * 0.25 + translation_integrity * 0.20) **Formula**: kernel_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Synthesizes all Phase J outputs into a single cross-project continuity **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DeliberationWorkspace

Scientific Role / Description: Newell & Simon 1972 (problem space); Baars 1988 (global workspace). Holds live premises, milestone facts, blockers, constraints, active hypotheses, and pivot implications for the deliberative reasoning layer. **Formula**: Standard bounded integration. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ThoughtChainConstructor

Scientific Role / Description: Newell & Simon 1972 (chaining); Johnson-Laird 1983 (mental models). **Formula**: Constructs bounded reasoning chains. CHAIN_DEPTH_CAP = 6 (Rule 1 frozen). **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

MultiHopInferenceEngine

Scientific Role / Description: Hobbs 1979 (coherence relations); Pearl 1988 (probabilistic inference). Multi-hop deductions across doctrine implications, milestone-to-agenda reasoning, claim-to-experiment implications, and identity consequences. **Formula**: Standard bounded integration. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ConstraintReconciliationSolver

Scientific Role / Description: Mackworth 1977 (constraint propagation); Dechter 2003 (constraint processing). through this solver before explanation synthesis. **Formula**: CONFLICT_THRESHOLD = 0.50 (Rule 2 frozen): branch conflict >= 0.50 must route **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Resolves cap conflicts, blocker contradictions, agenda inconsistency, pivot law, clashes, and EEIL tradeoff contradictions. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

HypothesisBranchPruner

Scientific Role / Description: Minsky 1986 (resource-bounded reasoning); Sloman 1996 (parallel reasoners). **Formula**: BRANCH_FLOOR = 2 (Rule 3 frozen): no greedy collapse below 2 surviving branches. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Keeps top 2 viable branches + top 1 stretch branch. Kills weak contradictions. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

LongHorizonReasoningBridge

Scientific Role / Description: Botvinick & Cohen 2014 (hierarchical temporal planning); Friston 2017 (active inference). Carries reasoning across current blocker, next experiment, paper roadmap, future pivots, and universal continuity doctrine. milestone_blocker, experiment_launch, paper_claim, eeil_tradeoff, project_pivot. **Formula**: HORIZON_BONUS = +0.15 (Rule 4 frozen) for threads touching **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ExplanationSynthesisEngine

Scientific Role / Description: Hobbs 1979 (abduction); Leake 1992 (explanation-based learning). Converts live reasoning state into natural symbolic explanation packets. **Formula**: Rule 2 compliance: if conflict_score >= 0.50 routes through **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: ConstraintReconciliationSolver output before synthesis. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ActiveCognitionKernel

Scientific Role / Description: Baars 1988 (global workspace); Dehaene 2014 (global neuronal workspace). Selects single dominant live reasoning state per tick (Rule 5 uniqueness). Aggregates deliberation workspace, thought chain, inference engine, constraint solver, branch pruner, horizon bridge, and explanation engine into one active symbolic mind state. workspace_coherence * 0.15 + chain_confidence * 0.20 + hop_confidence * 0.15 + prune_confidence * 0.20 + bridge_strength * 0.15 + explanation_clarity * 0.15) **Formula**: kernel_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ReasoningParagraphComposer

Scientific Role / Description: Levelt 1989 (speaking); Kintsch 1998 (text comprehension). Converts explanation packets into ordered reasoning paragraphs with premise -> inference -> conclusion flow. **Formula**: Standard bounded integration. **Hard Constants**: PARAGRAPH_CAP = 6 (Rule 1 frozen). Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

InstructionGroundingBridge

Scientific Role / Description: Grice 1975 (cooperative principle); Austin 1962 (speech acts). project_task, code_edit, milestone_blocker, experiment_launch, monograph_update must route through this bridge before composition. **Formula**: Standard bounded integration. **Hard Constants**: GROUNDING_TAGS frozen. Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Binds output to explicit project goals. Rule 2: tasks touching **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

DialogueTurnContinuity

Scientific Role / Description: Grosz & Sidner 1986 (discourse structure); Clark & Schaefer 1989 (grounding). Maintains prior turn anchors, task thread continuity, unresolved intent, and **Formula**: blocker carry-forward. TURN_FLOOR = 2 (Rule 3 frozen): at least 2 prior turns **Hard Constants**: must remain represented. Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

CodeRationaleNarrator

Scientific Role / Description: Sridhara et al. 2010 (code summarization); McBurney & McMillan 2014 (narratives). Converts patch logic, constraint reconciliations, branch pruning choices, and tradeoff decisions into natural code rationale prose. **Formula**: CLARITY_BONUS = +0.15 (Rule 4 frozen) for code_rationale packets. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ScientificExplanationFormatter

Scientific Role / Description: Swales 1990 (genre analysis); Gopen & Swan 1990 (scientific writing). Produces EEIL explanations, hypothesis logic, experiment interpretations, **Formula**: and paper-grade summaries. CLARITY_BONUS = +0.15 (Rule 4 frozen). **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

LongFormResponsePlanner

Scientific Role / Description: Reiter & Dale 2000 (NLG pipeline); Mann & Thompson 1988 (RST). Constructs coherent long-form responses integrating prior turns, active cognition kernel, project milestones, code rationale, and scientific **Formula**: explanation. RESPONSE_PARA_CAP = 6 (Rule 1 frozen). **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

NarrativeCoherenceRegulator

Scientific Role / Description: Hobbs 1985 (coherence relations); Grosz & Sidner 1986 (attentional state). task grounding, and active kernel alignment. **Formula**: Standard bounded integration. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Ensures no paragraph contradictions, stable tone, coherent arc, preserved **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

LanguageCognitionKernel

Scientific Role / Description: Levelt 1989 (speaking); Dehaene 2014 (global workspace, language). Selects single dominant natural-language reasoning state per tick (Rule 5). Aggregates response plan, regulated narrative, grounded task, dialogue continuity, and active cognition into one language mind state. plan_confidence * 0.25 + coherence_score * 0.25 + grounding_conf * 0.20 + turn_continuity * 0.15 + kernel_conf_ack * 0.15) **Formula**: kernel_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

IntentEvolutionTracker

Scientific Role / Description: Clark & Schaefer 1989 (grounding); Traum 1994 (dialogue moves). Tracks original goal, revised goals, emergent task shifts, clarified blockers, and changed priorities across multi-turn interaction. **Formula**: Standard bounded integration. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ClarificationQuestionPlanner

Scientific Role / Description: Bohus & Rudnicky 2009 (clarification strategies); Purver 2004 (reprise). **Formula**: Generates bounded clarification questions. CLARIFICATION_CAP = 3 (Rule 1 frozen)., Rule 2: ambiguity_score >= 0.50 must route through this planner before **Hard Constants**: response finalization. Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

CollaborativeRevisionLoop

Scientific Role / Description: Schegloff 2007 (sequence organization); Clark & Wilkes-Gibbs 1986 (referential repair). Maintains prior revisions, accepted changes, rejected edits, evolving at least 2 prior revisions must remain represented. **Formula**: constraints, and latest working plan. REVISION_FLOOR = 2 (Rule 3 frozen): **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

GoalNegotiationEngine

Scientific Role / Description: Allen et al. 2001 (mixed initiative); Grosz & Sidner 1986 (intention stack). Negotiates user goal vs milestone priority, short-term patch vs long-term doctrine, research vs product pivots, explanation vs execution. experiment_launch, paper_claim, project_pivot. **Formula**: NEGOTIATION_BONUS = +0.15 (Rule 4 frozen) for milestone_blocker, code_patch, **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

AmbiguityResolutionBridge

Scientific Role / Description: Purver 2004 (query repairs); Ginzburg 2012 (inquisitive semantics). Resolves wording, scope, timeline, domain, and experimental ambiguity. **Formula**: AMBIGUITY_THRESHOLD = 0.50 (Rule 2 mirror): routes through, ClarificationQuestionPlanner output when ambiguity_score >= 0.50. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

TurnLevelMemoryBinder

Scientific Role / Description: Grosz & Sidner 1986 (attentional state); Traum 1994 (grounding acts). Binds prior turn anchors, accepted clarifications, revision state, grounded goals, and dialogue continuity kernel into a unified turn memory state. **Formula**: Standard bounded integration. **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

PartneredPlanningSynthesizer

Scientific Role / Description: Allen et al. 2001 (collaborative planning); Grosz 1996 (shared plans). Constructs collaborative stepwise plans integrating negotiated goals, (Rule 4 mirror) for plans touching bonus-tagged axes. **Formula**: active language kernel. PLAN_STEP_CAP = 6. NEGOTIATION_BONUS = +0.15 **Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: prior revisions, clarification outputs, milestone constraints, and the **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

ConversationalAgencyKernel

Scientific Role / Description: Traum 1994 (dialogue moves); Clark & Schaefer 1989 (contribution model). Selects single dominant collaborative reasoning state per tick (Rule 5). Aggregates intent evolution, clarification planning, revision loop, goal negotiation, ambiguity resolution, turn memory, partnered plan, and active language kernel into one active conversational mind state. neg_conf * 0.20 + plan_conf * 0.20 + rev_coh * 0.15 + amb_conf * 0.15 + lang_conf * 0.15 + bind_conf * 0.15) **Formula**: kernel_confidence = clamp(**Hard Constants**: Biology tokens excluded. deque(maxlen=96). **Law Invariants**: Zero-drift survival alignment. **Waking Chain Placement**: Sequential chain execution. **Serialization Fields**: Full dict state synchronization. **EEIL Interpretation**: Maintains sparse inference bounds within loop execution.

---

5. Day 37–40 Proto-Representation & Representation Systems

5.1 Day 37 Pack 1 — Novelty Pressure

Novelty Counters (via enable_novelty flag)

Scientific Role / Description: Coarse-state novelty tracking with exponential decay (τ=200 ticks). States seen frequently decay toward zero; revisited states regain counts. Used only in BG scoring layer when enabled; never influences routing/TC/confidence directly. Pure observation layer atop transition_counts.

```
# Day 37 Pack 1 -- Novelty visit count update + decay
if _IKG.enable_novelty:
    _nov_upd_key = (
        getattr(_IKG, 'last_action_type', None),
        int(getattr(_IKG, 'bg_energy', 0.5) * 5),
        int(getattr(_IKG, 'bg_pe', 0.0) * 10),
    )
    _IKG.novelty_counts[_nov_upd_key] = _IKG.novelty_counts.get(_nov_upd_key, 0) + 1
    for _nov_k in _IKG.novelty_counts:
        _IKG.novelty_counts[_nov_k] *= 0.995   # decay tau ≈ 200 ticks
```

Formula: novelty_decay = 0.995/tick ⟹ τ = 1 / (1 - 0.995) ≈ 200 ticks. Count(state) ∝ (1/visits + decay). BG scoring (if enabled): score_explore += K_N × novelty_pressure, where novelty_pressure ∝ min(count, 1.0). **Hard Constants**: K_N = 0.20 (highest in signal hierarchy); decay = 0.995; coarse state key: (action, energy×5, pe×10). **Law Invariants**: counts never negative; clamped ≥ 0.0; enable_novelty must be explicitly True (zero drift when OFF). **Waking Chain Placement**: Line 34549–34560 in ikigai.py, post-action-commitment; updates novelty_counts after last_action_type is set. **Serialization Fields**: novelty_counts (dict: state_key → float). **EEIL Interpretation**: Sparse state visitation via exploratory uncertainty; decaying novelty creates cycling exploration loops, balancing energy efficiency against variance.

5.2 Day 37 Pack 4 — Curiosity Reinforcement

Curiosity Value Learning (via enable_curiosity_reinforcement flag)

Scientific Role / Description: Reward-based learning on exploration outcomes. When explore action is committed and subsequent energy improves, that (energy_band, pe_band) state gets a learned preference for explore. Slow decay (τ=1000 ticks) preserves learned values across longer horizons. Signal-only layer; never drives action selection directly.

```
# Day 37 Pack 4 -- Curiosity Reinforcement update + decay
if _IKG.enable_curiosity_reinforcement:
    _cr_upd_key = (
        int(getattr(_IKG, 'bg_energy', 0.5) * 5),
        int(getattr(_IKG, 'bg_pe', 0.0) * 10),
    )
    if getattr(_IKG, 'last_action_type', None) == 'explore':
        _cr_reward = max(0.0, min(1.0, (float(getattr(_IKG, 'adjusted_credit_signal', 0.0)) + 1.0) / 2.0))
        _cr_alpha  = 0.05
        _cr_old    = _IKG.curiosity_value.get(_cr_upd_key, 0.0)
        _IKG.curiosity_value[_cr_upd_key] = (1.0 - _cr_alpha) * _cr_old + _cr_alpha * _cr_reward
    for _cr_k in _IKG.curiosity_value:
        _IKG.curiosity_value[_cr_k] *= 0.999  # decay tau ≈ 1000 ticks
```

Formula: value[s] += α(reward − value[s]), α=0.05; reward = clamp(adjusted_credit + 1, 0, 2) / 2; decay = 0.999. **Hard Constants**: α=0.05; decay=0.999; reward clip [0,1]; update only when action=='explore'. **Law Invariants**: values clamped [0, 1]; enable_curiosity_reinforcement gated independently; zero impact when OFF. **Waking Chain Placement**: Line 34562–34576, post-novelty-update; consumes adjusted_credit_signal from Day 35 layer. **Serialization Fields**: curiosity_value (dict: (energy_band, pe_band) → float ∈ [0,1]). **EEIL Interpretation**: Reinforces exploratory behavior in energy-deficient states; creates local learning loop independent of global PE, enabling targeted curiosity at metacognitive scale.

5.3 Day 37 (V2) & Day 38 Pack 1–2 — Proto-Expectation (Transition Prediction)

Transition Counts & Transition Probabilities (always active, no flag)

Scientific Role / Description: Online estimation of state transition frequencies. Records (coarse_state_t → coarse_state_{t+1}) pairs using novelty's coarse key (action, energy_band, pe_band). Periodic rebuild (100-tick intervals) computes empirical transition probabilities P(next|current). Pure observation; zero behavior coupling.

```
# Day 37 V2 -- Proto-Representation: Transition Logging (always active, no flag)
_tr_curr = (
    getattr(_IKG, 'last_action_type', None),
    int(getattr(_IKG, 'bg_energy', 0.5) * 5),
    int(getattr(_IKG, 'bg_pe', 0.0) * 10),
)
# ... later: Day 38 Pack 1 periodic rebuild (every 100 ticks)
if (tick - _IKG._pred_last_rebuild_tick) >= _IKG.pred_rebuild_interval:
    _tr_grouped = {}
    for (_tr_p, _tr_n), _tr_c in _IKG.transition_counts.items():
        _tr_grouped.setdefault(_tr_p, {})[_tr_n] = _tr_grouped.setdefault(_tr_p, {}).get(_tr_n, 0) + _tr_c
    _tr_new = {}
    for _tr_p, _tr_nxts in _tr_grouped.items():
        _tr_total = sum(_tr_nxts.values())
        if _tr_total > 0:
            _tr_new[_tr_p] = {_tr_n: _tr_c / _tr_total for _tr_n, _tr_c in _tr_nxts.items()}
    _IKG.transition_probs = _tr_new
    _IKG._pred_last_rebuild_tick = tick
```

Formula: transition_counts[s_prev → s_curr] += 1 (every waking tick). Rebuild: P(s_next | s_prev) = count(s_prev → s_next) / Σ_s' count(s_prev → s'). Rebuild interval: 100 ticks (configurable). **Hard Constants**: Rebuild interval = pred_rebuild_interval = 100; coarse key: (action, energy×5, pe×10); no decay on raw counts. **Law Invariants**: transition_counts never negative; transition_probs probabilities sum to 1.0 per source; unknown states map to empty dict. **Waking Chain Placement**: Line 34578–34775 in ikigai.py; logging at every waking tick, rebuild 100-ticks conditionally. **Serialization Fields**: transition_counts (dict), transition_probs (dict of dicts), _prev_state_key, _pred_last_rebuild_tick. **EEIL Interpretation**: Minimal-storage model learning (< 1 KB for typical regimes); enables downstream prediction without gradient descent.

5.4 Day 38 Pack 2 & Pack 4 — Transition Prediction Error & Confidence (always active, no flag)

Transition Prediction Error (signal only, no behavior coupling)

Scientific Role / Description: Measure of state model accuracy: error = 1 − P(actual_next | prev). Computed before transition_counts update, so judgment uses the model in force from previous rebuild. Zero behavior influence; aggregated to mean/variance for meta-statistics.

```
# Day 38 Pack 2 -- Transition Prediction Error (signal only, no behavior)
if _IKG._prev_state_key is not None:
    _te_dist = _IKG.transition_probs.get(_IKG._prev_state_key)
    if not _te_dist:
        _IKG.transition_error = 1.0
        _IKG.predicted_next_state = None
    else:
        _te_prob = _te_dist.get(_tr_curr, 0.0)
        _IKG.transition_error = 1.0 - _te_prob
        _IKG.predicted_next_state = max(_te_dist.items(), key=lambda kv: kv[1])[0]
    _IKG.actual_next_state = _tr_curr
    _IKG._te_sum   += _IKG.transition_error
    _IKG._te_sumsq += _IKG.transition_error * _IKG.transition_error
    _IKG._te_count += 1
```

Prediction Confidence (signal only, no behavior coupling)

Scientific Role / Description: Belief in the model for the current state: confidence = max P(next | current). Orthogonal to error: a high-confidence, high-error state is "sure and wrong" — canonical surprise. Aggregated for meta-learning (Day 38 Pack 5).

```
# Day 38 Pack 4 -- Prediction Confidence (signal only, no behavior)
_pc_dist = _IKG.transition_probs.get(_tr_curr)
if not _pc_dist:
    _IKG.prediction_confidence = 0.0
else:
    _IKG.prediction_confidence = max(_pc_dist.values())
_IKG._pc_sum   += _IKG.prediction_confidence
_IKG._pc_sumsq += _IKG.prediction_confidence * _IKG.prediction_confidence
_IKG._pc_count += 1
```

Formula: TE = 1 − P(s_actual | s_prev); PC = max P(s_next | s_current). Surprise = TE × PC (computed by downstream layers). Mean TE = _te_sum / _te_count; variance via Welford online. **Hard Constants**: TE range [0, 1]; PC range [0, 1]; unknown state ⟹ TE=1, PC=0. **Law Invariants**: TE computed before transition_counts update (model reflects prior knowledge); both signals computed every waking tick (no gating). **Waking Chain Placement**: Line 34586–34618 in ikigai.py; post-transition-logging, pre-cluster-transition. **Serialization Fields**: transition_error, prediction_confidence, predicted_next_state, actual_next_state, _te_sum, _te_sumsq, _te_count, _pc_sum, _pc_sumsq, _pc_count. **EEIL Interpretation**: Separates uncertainty (PC) from error (TE); enables surprise detection without global loss function.

5.5 Day 38 Pack 3 & Pack 5 — Prediction Modulation & Confidence-Aware Surprise

Error-Gated Modulation (via enable_prediction_modulation flag)

Scientific Role / Description: Weak nudge to BG scoring: when transition_error is high, boost explore, suppress approach. K_PE = 0.10, strictly < K_N = 0.20 (not a primary driver). Pure measurement when disabled.

```
# Day 38 Pack 3 -- Error-Gated Modulation (default OFF -- zero drift)
if _IKG.enable_prediction_modulation:
    _K_PE = 0.10  # < K_N
    score_explore += _K_PE * float(getattr(_IKG, 'transition_error', 0.0))
    score_approach -= _K_PE * float(getattr(_IKG, 'transition_error', 0.0))
```

Confidence-Aware Surprise (via enable_confidence_surprise flag)

Scientific Role / Description: Refines error signal using confidence: real_surprise = TE × PC. Boosts score_explore only when sure-and-wrong. K_CS = 0.01 (0.5 × K_PE); sub-PE refinement.

```
# Day 38 Pack 5 -- Confidence-Aware Surprise (default OFF -- zero drift)
if _IKG.enable_confidence_surprise:
    _K_CS = 0.01
    _real_surprise = float(getattr(_IKG, 'transition_error', 0.0)) * float(getattr(_IKG, 'prediction_confidence', 0.0))
    score_explore += _K_CS * _real_surprise
```

Formula: Pack 3: score_Δ ∝ K_PE × TE. Pack 5: score_Δ ∝ K_CS × (TE × PC). **Hard Constants**: K_PE = 0.10; K_CS = 0.01; both gated by enable flags. **Law Invariants**: Score changes clamped [−1, +1] in BG layer; zero impact when disabled. **Waking Chain Placement**: Line 22978–23017 in ikigai.py, inside BG scoring conditional block. **Serialization Fields**: Flags enable_prediction_modulation, enable_confidence_surprise only. **EEIL Interpretation**: Predictive mismatch as exploration signal; separates structured uncertainty (TE×PC product) from random novelty, enabling targeted investigation.

5.6 Day 39 Pack 1 — State Abstraction (Clustering)

State Clustering & Embeddings (always active, observation only)

Scientific Role / Description: Online incremental clustering of coarse states into ≤16 reusable concepts (CID_0..CID_15). Periodic update (100-ticks) using L1 distance on a 5-dimensional feature embedding: (action_encoding, energy_band, pe_band, mean_TE, mean_PC). New states either join nearest cluster (<= distance_threshold) or spawn new cluster (if under max). Pure observation; zero behavior coupling.

```
# Day 39 Pack 1 -- State Abstraction (always active, observation only)
if (tick - _IKG._cluster_last_update_tick) >= _IKG.cluster_update_interval:
    _ACT_ENC_C = {None: -1.0, 'explore': 0.0, 'approach': 1.0, 'wait': 2.0,
                  'idle_recover': 3.0, 'edit_code': 4.0, 'run_experiment': 5.0}
    _embeddings = {}
    for _sk_c, _rt_c in _IKG._state_runtime.items():
        _act_c, _eb_c, _pb_c = _sk_c
        _aenc_c = _ACT_ENC_C.get(_act_c, -1.0)
        _mte_c = _rt_c['te_sum'] / max(_rt_c['te_count'], 1)
        _dist_c = _IKG.transition_probs.get(_sk_c)
        _mpc_c = max(_dist_c.values()) if _dist_c else 0.0
        _embeddings[_sk_c] = (_aenc_c, float(_eb_c), float(_pb_c), _mte_c, _mpc_c)
    # ... clustering via L1 distance and centroid maintenance ...
    _IKG.state_clusters = _new_clusters
    _IKG._cluster_last_update_tick = tick
```

Formula: feature vector = (action_code ∈ {-1..5}, energy_band ∈ {0..5}, pe_band ∈ {0..10}, mean_TE, mean_PC). Distance = L1(feat_i, feat_j). Cluster iff dist ≤ threshold (default 3.0). Centroid = mean(features in cluster). **Hard Constants**: max_clusters = 16; cluster_distance_threshold = 3.0; cluster_update_interval = 100; action_encoding: approach=1, explore=0, wait=2, idle_recover=3. **Law Invariants**: state_clusters is a dict (state_key → cluster_id ∈ {0..15}); centroids store mean features; members track cluster memberships for recomputation. **Waking Chain Placement**: Line 34689–34760 in ikigai.py, periodic conditional block every 100 ticks. **Serialization Fields**: state_embeddings, state_clusters, cluster_stats, _state_runtime, _cluster_last_update_tick. **EEIL Interpretation**: Unsupervised pattern discovery via prediction error; clusters high-TE zones → targets for targeted exploration.

5.7 Day 39 Pack 2 — Concept-Conditioned Modulation

Concept Modulation (via enable_concept_modulation flag)

Scientific Role / Description: Cluster context guides BG scoring with minimal weight. K_CTX_CL = 0.005, strictly < K_CS = 0.01 < K_PE = 0.02 < K_N = 0.20. Never primary driver; only provides weak context bias. Zero drift when disabled.

```
# Day 39 Pack 2 -- Concept-Conditioned Modulation (default OFF -- zero drift)
if _IKG.enable_concept_modulation:
    _K_CTX_CL = 0.005  # < K_CS
    _ctx_sig = float(getattr(_IKG, 'cluster_transition_error', 0.0))
    score_explore += _K_CTX_CL * _ctx_sig
```

Formula: K_CTX_CL × cluster_TE added to score_explore (when enabled). **Hard Constants**: K_CTX_CL = 0.005; enable_concept_modulation flag. **Law Invariants**: Zero impact when disabled; strictly lower priority than state-level signals. **Waking Chain Placement**: Line 23017–23040 in ikigai.py, inside BG scoring conditional. **Serialization Fields**: enable_concept_modulation only. **EEIL Interpretation**: Multi-scale surprise: uses abstracted cluster-level prediction error to refine action selection without overriding local dynamics.

5.8 Day 39 Pack 3 & Pack 4 — Concept Transition Learning & Error

Cluster Transition Counts & Probabilities (always active, observation only)

Scientific Role / Description: Records (cid_prev → cid_curr) frequencies. Periodic rebuild (100-ticks) normalizes to transition probabilities P(cid_next | cid_prev). Bounded by max_clusters² ≤ 256 entries. Pure observation; zero scoring coupling.

Cluster-Level Error & Confidence (signal only, always active)

Scientific Role / Description: Cluster-scale TE and PC computed from cluster_transition_probs. TE_c = 1 − P(cid_actual | cid_prev); PC_c = max P(cid_next | cid_prev). Unknown prev ⟹ TE_c = 1, PC_c = 0.

```
# Day 39 Pack 3/4 -- Concept Transition Logging & Error/Confidence
_ct_curr_cid = _IKG.state_clusters.get(_tr_curr)
if _ct_curr_cid is not None:
    if _IKG._cluster_prev_id is not None:
        _ct_pair_k = (_IKG._cluster_prev_id, _ct_curr_cid)
        _IKG.cluster_transition_counts[_ct_pair_k] = _IKG.cluster_transition_counts.get(_ct_pair_k, 0) + 1
        _cce_dist = _IKG.cluster_transition_probs.get(_IKG._cluster_prev_id)
        if not _cce_dist:
            _IKG.cluster_transition_error = 1.0
            _IKG.cluster_prediction_conf  = 0.0
        else:
            _cce_p = _cce_dist.get(_ct_curr_cid, 0.0)
            _IKG.cluster_transition_error = max(0.0, min(1.0, 1.0 - _cce_p))
            _IKG.cluster_prediction_conf  = max(0.0, min(1.0, max(_cce_dist.values())))
        _IKG._ce_sum   += _IKG.cluster_transition_error
        _IKG._ce_sumsq += _IKG.cluster_transition_error * _IKG.cluster_transition_error
        _IKG._ce_count += 1
        _IKG._cpc_sum   += _IKG.cluster_prediction_conf
        _IKG._cpc_sumsq += _IKG.cluster_prediction_conf * _IKG.cluster_prediction_conf
        _IKG._cpc_count += 1
```

Formula: cluster_transition_error = 1 − P(cid_actual | cid_prev); cluster_prediction_conf = max P(cid_next | cid_prev). Periodic rebuild (100-ticks) normalizes counts to probabilities. **Hard Constants**: cluster_rebuild_interval = 100; max_clusters = 16 (Packing Efficiency). **Law Invariants**: Both signals computed every waking tick when cluster_prev_id known; unknown prev ⟹ TE=1, PC=0. **Waking Chain Placement**: Line 34619–34688 in ikigai.py; post-state-level signals, pre-VSA hook. **Serialization Fields**: cluster_transition_counts, cluster_transition_probs, cluster_transition_error, cluster_prediction_conf, _cluster_prev_id, _ce_sum, _ce_sumsq, _ce_count, _cpc_sum, _cpc_sumsq, _cpc_count. **EEIL Interpretation**: Hierarchical model learning; enables meta-control based on conceptual (not raw-state) prediction accuracy.

5.9 Day 39 Pack 5 — Cross-Level Conflict Detection

Cross-Level Conflict (signal only, always active)

Scientific Role / Description: Measures disagreement between state-level and concept-level surprise models. conflict = |state_surprise − concept_surprise|; signed variant retains direction. Negative signed = global model failed; positive = local chaos in otherwise predictable cluster.

```
# Day 39 Pack 5 -- Cross-Level Conflict (meta-consistency, signal only)
_state_surprise   = float(_IKG.transition_error) * float(_IKG.prediction_confidence)
_concept_surprise = float(_IKG.cluster_transition_error) * float(_IKG.cluster_prediction_conf)
_signed = _state_surprise - _concept_surprise
_IKG.cross_level_signed   = max(-1.0, min(1.0, _signed))
_IKG.cross_level_conflict = max(0.0, min(1.0, abs(_signed)))
_IKG._cc_sum   += _IKG.cross_level_conflict
_IKG._cc_sumsq += _IKG.cross_level_conflict * _IKG.cross_level_conflict
_IKG._cc_count += 1
```

Formula: conflict_unsigned = |TE×PC − TE_c×PC_c|; conflict_signed = (TE×PC) − (TE_c×PC_c). Range: unsigned ∈ [0,1], signed ∈ [−1,+1]. **Hard Constants**: No hardcoded thresholds; pure signal aggregation. **Law Invariants**: Both metrics clamped to ranges; updated every waking tick when both cluster and state ids known. **Waking Chain Placement**: Line 34643–34653 in ikigai.py; computed after both state and cluster confidence/error signals finalize. **Serialization Fields**: cross_level_conflict, cross_level_signed, _cc_sum, _cc_sumsq, _cc_count. **EEIL Interpretation**: Metacognitive consistency check; high conflict ⟹ model abstraction failing at this moment; signal for targeted investigation.

5.10 Day 39 Packs 6–11 — Meta-Control Stack (Conflict Attention, Safe Probe, Targeted Explore, Selective Override, Strategy Persistence)

Conflict-Gated Attention (via enable_conflict_attention flag)

Scientific Role / Description: Spike generator: when cross_level_signed < 0 AND conflict ≥ 0.6 AND cooldown elapsed (5-tick gap), trigger an attention window. K_CF = 0.002, strictly < all predecessors. Pure gating; no score modification.

```
# Day 39 Pack 6 -- Conflict-Gated Attention (default OFF -- zero drift)
if _IKG.enable_conflict_attention:
    _K_CF = 0.002
    if (float(getattr(_IKG, 'cross_level_signed', 0.0)) < 0 and
        float(getattr(_IKG, 'cross_level_conflict', 0.0)) >= 0.6 and
        tick - getattr(_IKG, '_conflict_last_tick', -999) >= _IKG.conflict_cooldown_ticks):
        _IKG.attention_timer = 5
        _IKG._conflict_last_tick = tick
```

Attention Persistence (via enable_attention_persistence flag)

Scientific Role / Description: Maintains temporal focus window (5 ticks, exponential decay) triggered by conflict. K_AP = 0.001, strictly < K_CF = 0.002. Active only when attention_timer > 0.

```
# Day 39 Pack 7 -- Attention Persistence (default OFF -- zero drift)
if _IKG.enable_attention_persistence and _IKG.attention_timer > 0:
    _K_AP = 0.001
    _IKG.attention_strength = 1.0 - (_IKG.attention_timer / 5.0)   # decay over 5 ticks
    _IKG.attention_timer -= 1
```

Safe-Probing under Conflict (via enable_safe_probe flag)

Scientific Role / Description: Injects small exploratory pressure only when attention window open AND energy > safety_margin (0.4). K_SP = 0.0015, strictly < K_AP. Investigates anomalies without breaking energy stability.

```
# Day 39 Pack 8 -- Safe-Probing under Conflict (default OFF -- zero drift)
if _IKG.enable_safe_probe and _IKG.attention_timer > 0:
    if float(getattr(_IKG, 'bg_energy', 0.5)) > 0.4:
        _K_SP = 0.0015
        score_explore += _K_SP * _IKG.attention_strength
```

Targeted Exploration (via enable_targeted_explore flag)

Scientific Role / Description: Boosts exploration only where model is meaningfully wrong: max(TE × PC, TE_c × PC_c) > 0.3. K_TE = 0.0012, strictly < K_SP. Focuses on high-signal error zones.

```
# Day 39 Pack 9 -- Targeted Exploration (default OFF -- zero drift)
if _IKG.enable_targeted_explore and _IKG.attention_timer > 0:
    _K_TE = 0.0012
    _target_signal = max(
        float(getattr(_IKG, 'transition_error', 0.0)) * float(getattr(_IKG, 'prediction_confidence', 0.0)),
        float(getattr(_IKG, 'cluster_transition_error', 0.0)) * float(getattr(_IKG, 'cluster_prediction_conf', 0.0))
    )
    if _target_signal > 0.3:
        score_explore += _K_TE * _target_signal
```

Selective Override (via enable_selective_override flag)

Scientific Role / Description: Reduces approach/wait dominance only when target_signal > 0.7 AND attention open AND energy safe. K_OV = 0.0008, strictly < K_TE. Prevents rigid action locking during anomalies.

```
# Day 39 Pack 10 -- Selective Override (default OFF -- zero drift)
if _IKG.enable_selective_override and _IKG.attention_timer > 0:
    if float(getattr(_IKG, 'bg_energy', 0.5)) > 0.4:
        _K_OV = 0.0008
        _concept_sig_ov = float(getattr(_IKG, 'cluster_transition_error', 0.0)) * float(getattr(_IKG, 'cluster_prediction_conf', 0.0
        if _concept_sig_ov > 0.7:
            score_approach -= _K_OV * _concept_sig_ov
```

Micro-Strategy Persistence (via enable_strategy_persistence flag)

Scientific Role / Description: Brief commitment (≤3 ticks) to last action when attention high + target_signal > 0.6. K_MS = 0.0006, smallest in hierarchy. No long-term lock; maximum 3 ticks enforces fine-grained correction cycles.

```
# Day 39 Pack 11 -- Micro-Strategy Persistence (default OFF -- zero drift)
if _IKG.enable_strategy_persistence and _IKG.attention_timer > 0:
    _K_MS = 0.0006
    _concept_sig_ms = float(getattr(_IKG, 'cluster_transition_error', 0.0)) * float(getattr(_IKG, 'cluster_prediction_conf', 0.0))
    if _concept_sig_ms > 0.6 and _IKG.strategy_timer == 0:
        _IKG.strategy_action = getattr(_IKG, 'last_action_type', None)
        _IKG.strategy_timer = 3
    if _IKG.strategy_timer > 0 and _IKG.strategy_action is not None:
        # Bias toward strategy_action in BG routing (implementation context-dependent)
        _IKG.strategy_timer -= 1
```

Formula:

- Pack 6 (Conflict): gate = (cross_signed < 0) ∧ (conflict ≥ 0.6) ∧ (cooldown elapsed)
- Pack 7 (Persist): strength = 1 − (timer / 5), timer decays [5 → 0]
- Pack 8 (SafeProbe): score_explore += K_SP × strength if energy > 0.4
- Pack 9 (Targeted): boost if max(TE×PC, TE_c×PC_c) > 0.3
- Pack 10 (Override): suppress approach if signal > 0.7
- Pack 11 (Persist): lock action ≤ 3 ticks if signal > 0.6

Hard Constants: K_CF=0.002, K_AP=0.001, K_SP=0.0015, K_TE=0.0012, K_OV=0.0008, K_MS=0.0006; conflict_threshold=0.6; signal_threshold_targeted=0.3; signal_threshold_override=0.7; signal_threshold_persist=0.6; strategy_max_ticks=3; conflict_cooldown=5; attention_window=5. **Law Invariants**: All K values form strict hierarchy: K_N > K_PE > K_CS > K_CTX > K_CF > K_AP > K_SP > K_TE > K_OV > K_MS. Attention window decays exponentially; strategy lock ≤ 3 ticks. All gates respect enable flags (zero drift when OFF). **Waking Chain Placement**: Lines 23040–23143 in ikigai.py, inside BG scoring conditional block (all packs 6–11 gated together). Computed after cross_level_conflict finalization. **Serialization Fields**: enable_conflict_attention, enable_attention_persistence, enable_safe_probe, enable_targeted_explore, enable_selective_override, enable_strategy_persistence (flags); attention_timer, attention_strength, strategy_timer, strategy_action, _conflict_last_tick, conflict_cooldown_ticks (state). **EEIL Interpretation**: Hierarchical meta-control: conflict detection → attention spike → safe investigation → targeted correction → action locking (brief). Operates at fine timescales (5 ticks) to enable rapid response to model failures while preserving energy stability (energy gate on all probes).

5.11 Day 40 Pack 1 — VSA Foundation (Vector Symbolic Architecture)

Hypervector Item Library (via enable_vsa flag, storage-only)

Scientific Role / Description: Fixed near-orthogonal binary hypervectors (400 bits, random seeded) representing 27 atomic concepts: A, B, STATE, ACTION, CONTEXT (base items) + CID_0..CID_15 (cluster identifiers) + ACT_approach, ACT_explore, ACT_wait, ACT_idle_recover, ACT_edit_code, ACT_run_experiment (action types). Idempotent initialization; zero side effects when VSA disabled.

```
def vsa_init_items(self, names=None, seed=42):
    """Generate fixed near-orthogonal binary hypervectors. Idempotent."""
    if self.vsa_items:
        return
    names = ["A", "B", "STATE", "ACTION", "CONTEXT"]
    for i in range(self.max_clusters if hasattr(self, "max_clusters") else 16):
        names.append(f"CID_{i}")
    for a in ("approach", "explore", "wait", "idle_recover", "edit_code", "run_experiment"):
        names.append(f"ACT_{a}")
    import random as _r
    rng = _r.Random(seed)
    n = self.vsa_dim
    for nm in names:
        self.vsa_items[nm] = bytearray(rng.getrandbits(1) for _ in range(n))
```

VSA Operators: Binding, Bundling, Similarity

Scientific Role / Description: XOR-based binding (self-inverse); majority-vote bundling; hamming-ratio similarity and cosine similarity (−1/+1 mapped).

```
def vsa_bind(self, a, b):
    """XOR binding. Self-inverse: bind(bind(A,B), A) == B."""
    return bytearray(x ^ y for x, y in zip(a, b))

def vsa_bundle(self, vectors):
    """Majority-rule bundling. Threshold > half of input count."""
    if not vectors:
        return bytearray(self.vsa_dim)
    n = len(vectors[0])
    half = len(vectors) / 2.0
    out = bytearray(n)
    for i in range(n):
        s = sum(v[i] for v in vectors)
        out[i] = 1 if s > half else 0
    return out

def vsa_similarity(self, a, b):
    """Hamming match ratio. Identical=1.0, random~0.5, orthogonal~0.5."""
    if a is None or b is None:
        return 0.0
    n = len(a)
    if n == 0:
        return 0.0
    matches = sum(1 for x, y in zip(a, b) if x == y)
    return matches / n

def vsa_cosine(self, a, b):
    """Cosine similarity over -1/+1 mapped binary vectors.
    Equivalent to 2 * hamming_match_ratio - 1."""
    return 2.0 * vsa_similarity(a, b) - 1.0 if a and b else 0.0
```

Formula: Binding: E = CID ⊕ ACT (XOR). Bundling: C = MAJ(E_1, E_2, ..., E_k) (bitwise majority). Similarity: hamming = matches / dim; cosine = 2×hamming − 1. **Hard Constants**: vsa_dim = 400; seed = 42 (deterministic); item count = 27; XOR self-inverse by definition (x⊕x=0, x⊕0=x). **Law Invariants**: vsa_items never modified after init; all operations bitwise-deterministic (no randomness); enable_vsa must be explicitly True (zero behavior coupling when OFF). **Waking Chain Placement**: Methods never called when enable_vsa=False; idempotent init on first vsa_encode_event call (line 34659–34661). **Serialization Fields**: enable_vsa, vsa_dim, vsa_items (dict: name → bytearray), vsa_current. **EEIL Interpretation**: Sparse distributed representation (400 bits = ~100 bytes per vector); storage-bounded (27 items × 400 bits = 1.35 KB); enables symbolic binding without dense multiplication.

5.12 Day 40 Pack 2 — Observational VSA Encoding (Core Integration)

Event Encoding & Circular Buffer (via enable_vsa flag, fixed 128 slots)

Scientific Role / Description: Per-waking-tick, (cluster_id, action_type) pairs are bound into event vectors E_t = bind(CID, ACT), appended to a circular buffer (128 slots, no growth). Multi-scale similarity aggregates: sim_1 (immediate vs prev), sim_5 (5-tick window mean), sim_20 (20-tick window mean). Pure observation; zero routing/TC/confidence coupling.

```
def vsa_encode_event(self, cid, action_type):
    """Build event vector E = bind(CID_<cid>, ACT_<action>)."""
    if cid is None or action_type is None:
        return None
    cid_key = f"CID_{cid}"
    act_key = f"ACT_{action_type}"
    cid_v = self.vsa_items.get(cid_key)
    act_v = self.vsa_items.get(act_key)
    if cid_v is None or act_v is None:
        return None
    return self.vsa_bind(cid_v, act_v)

# Waking chain integration (lines 34654-34676):
# ...
if _IKG.enable_vsa:
    if not _IKG.vsa_items:
        _IKG.vsa_init_items()
    _vsa_act = getattr(_IKG, 'last_action_type', None)
    _vsa_ev  = _IKG.vsa_encode_event(_ct_curr_cid, _vsa_act)
    if _vsa_ev is not None:
        _vsa_prev_for_chain = _IKG.vsa_event
        _IKG.vsa_record_event(_vsa_ev, curr_cid=_ct_curr_cid, prev_cid=_vsa_prev_cid,
                              curr_action=_vsa_act, prev_action=getattr(_IKG, 'vsa_prev_action', None))
        # ...
```

Multi-Scale Similarity Aggregation

Scientific Role / Description: sim_1 = cosine(E_t, E_); sim_5 / sim_20 = mean cosine over 5/20-tick windows. Variance ordering: var(sim_1) > var(sim_5) > var(sim_20) (e.g., 0.112 → 0.060 → 0.039). No smoothing; raw aggregation.

```
def vsa_record_event(self, event, curr_cid=None, prev_cid=None, curr_action=None, prev_action=None):
    """Append event to circular buffer; update sim_1 / sim_5 / sim_20 aggregates."""
    if event is None:
        return
    # sim_1 against most recent prior event
    s1 = None
    if self.vsa_event is not None:
        s1 = self.vsa_cosine(event, self.vsa_event)
        self.vsa_sim1_sum   += s1
        self.vsa_sim1_count += 1
    # sim_5 / sim_20 windowed means + pattern reuse
    n_buf = self.vsa_buffer_size
    cnt   = self.vsa_count
    if cnt > 0:
        for window, sum_attr, cnt_attr in ((5, 'vsa_sim5_sum', 'vsa_sim5_count'),
                                           (20, 'vsa_sim20_sum', 'vsa_sim20_count')):
            k = window if cnt >= window else cnt
            if k <= 0:
                continue
            acc = 0.0
            for j in range(1, k + 1):
                idx = (self.vsa_index - j) % n_buf
                prev = self.vsa_buffer[idx]
                if prev is None:
                    continue
                acc += self.vsa_cosine(event, prev)
            setattr(self, sum_attr, getattr(self, sum_attr) + acc / k)
            setattr(self, cnt_attr, getattr(self, cnt_attr) + 1)
    # Append to buffer + advance index
    self.vsa_buffer[self.vsa_index] = event
    self.vsa_index  = (self.vsa_index + 1) % n_buf
    self.vsa_count += 1
    self.vsa_prev_event  = self.vsa_event
    self.vsa_event       = event
```

Formula: sim_1(t) = cosine(E_t, E_) ∈ [−1,+1]. sim_k(t) = mean{ cosine(E_t, E_{t−j}) : j ∈ [1,k] }. Empirical: mean(sim_1)≈0.86, var(sim_1)≈0.112 for 86% stable regime + 14% transitions. **Hard Constants**: vsa_buffer_size = 128 (fixed); window sizes = 1, 5, 20 ticks; no smoothing; cosine range [−1,+1]. **Law Invariants**: Buffer never grows beyond 128; events cyclic (old events overwritten); sim aggregates unbounded (infinite accumulation over simulation lifetime). **Waking Chain Placement**: Line 34659–34676 in ikigai.py; called every waking tick when enable_vsa=True and cluster_id known. **Serialization Fields**: vsa_event, vsa_prev_event, vsa_buffer, vsa_index, vsa_count, vsa_sim1_sum, vsa_sim1_count, vsa_sim5_sum, vsa_sim5_count, vsa_sim20_sum, vsa_sim20_count. **EEIL Interpretation**: Temporal variance smoothing via windowing; multi-scale statistics reveal behavioral regime stability (high variance ⟹ transitions; low variance ⟹ stable clusters).

5.13 Day 40 Pack 3 — Semantic Validation (VSA Meaning Layer)

Spike Detection & Classification

Scientific Role / Description: Spikes mark regime transitions (sim_1 < 0.5, threshold raised from Pack 2). Classified by what changed: cluster_changed (prev_cid ≠ curr_cid), action_changed (prev_action ≠ curr_action), both_changed. 100% of spikes are real action changes (XOR construction guarantees zero spurious spikes); 30.3% also involve cluster changes.

```
# Pack 3: spike classification (lines 34345-34358 in state.py)
if s1 < self.vsa_spike_threshold:  # threshold = 0.5
    self.vsa_spike_count += 1
    # Classification — what changed at this transition?
    cluster_changed = (prev_cid is not None and curr_cid is not None
                       and prev_cid != curr_cid)
    action_changed  = (prev_action is not None and curr_action is not None
                       and prev_action != curr_action)
    if cluster_changed:
        self.vsa_spike_cluster_change += 1
    if action_changed:
        self.vsa_spike_action_change += 1
    if cluster_changed and action_changed:
        self.vsa_spike_both_change += 1
```

Cluster Separation Metrics

Scientific Role / Description: Same-cluster transitions (prev_cid = curr_cid) have mean_sim ≈ +0.90 (high coherence); different-cluster transitions have mean_sim ≈ −0.05 (near-orthogonal). Separation gap = +0.948 (near-perfect distinguishability in vector space).

```
# Pack 3: same/diff cluster separation (lines 34359-34366 in state.py)
if prev_cid is not None and curr_cid is not None:
    if prev_cid == curr_cid:
        self.vsa_same_cluster_sim_sum += s1
        self.vsa_same_cluster_count   += 1
    else:
        self.vsa_diff_cluster_sim_sum += s1
        self.vsa_diff_cluster_count   += 1
```

Pattern Reuse Counters

Scientific Role / Description: Per event, count how many prior buffer entries have cosine(event, prior) > threshold (0.8). Average 77 matches per 128-slot buffer → tight repetitive regime; indicates reusable behavioral patterns.

```
# Pack 3: pattern reuse (lines 34387-34398 in state.py)
matches = 0
scan_k = min(cnt, n_buf)
for j in range(1, scan_k + 1):
    idx = (self.vsa_index - j) % n_buf
    prev = self.vsa_buffer[idx]
    if prev is None:
        continue
    if self.vsa_cosine(event, prev) > self.vsa_pattern_threshold:  # threshold = 0.8
        matches += 1
self.vsa_pattern_match_count += matches
self.vsa_pattern_total       += 1
```

Formula: Spike iff sim_1 < 0.5. Classification bitmask via (cluster_changed, action_changed) pair. Cluster separation gap = mean_same − mean_diff. Pattern reuse = count(buffer entries with cosine > 0.8) per event. **Hard Constants**: vsa_spike_threshold = 0.5 (raised from Pack 2: 0.3); vsa_pattern_threshold = 0.8; empirical: same-cluster mean=+0.90, diff-cluster mean=−0.05, gap=+0.948; pattern reuse ≈77 per event. **Law Invariants**: Spike classification inclusive (both_change is subset of union); separation gap clamped [−2,+2]; pattern reuse ≤ min(count, buffer_size). **Waking Chain Placement**: Lines 34345–34398 in state.py, called within vsa_record_event when optional kwargs provided (Pack 3 harness passes curr_cid, prev_cid, curr_action, prev_action). **Serialization Fields**: vsa_spike_count, vsa_sim_spikes (alias), vsa_spike_cluster_change, vsa_spike_action_change, vsa_spike_both_change, vsa_same_cluster_sim_sum, vsa_same_cluster_count, vsa_diff_cluster_sim_sum, vsa_diff_cluster_count, vsa_pattern_match_count, vsa_pattern_total, vsa_pattern_threshold. **EEIL Interpretation**: XOR binding ensures cluster/action separation encodes as orthogonality in vector space; pattern reuse reveals behavioral stereotypy without learning (discovery via statistics alone).

5.14 Day 40 Pack 4 — Representation Manipulation (VSA Computation Layer)

Recovery via Self-Inverse XOR

Scientific Role / Description: Validates that bind(E, CID) recovers ACT, and bind(E, ACT) recovers CID, where E = bind(CID, ACT). Similarity of recovered vectors to originals measured via cosine. Empirical: sim=1.0000 exact (XOR is bit-exact when components clean).

```
def vsa_test_recovery(self, event, cid_vec, act_vec):
    """Day 40 Pack 4: bind(E, CID) -> ACT'; bind(E, ACT) -> CID'. Pure measurement."""
    if event is None or cid_vec is None or act_vec is None:
        return
    rec_act = self.vsa_bind(event, cid_vec)
    rec_cid = self.vsa_bind(event, act_vec)
    self.vsa_recover_act_sim += self.vsa_cosine(rec_act, act_vec)
    self.vsa_recover_cid_sim += self.vsa_cosine(rec_cid, cid_vec)
    self.vsa_recover_count   += 1
```

Chain Composition & Coherence

Scientific Role / Description: Chains bundle consecutive events: C_t = MAJ(E_t, E_). Measures how well chain preserves components: coherence = mean(cosine(C, E_t), cosine(C, E_)). Empirical: 86% stable (coherence ≈ 0.95) + 14% transitions (coherence ≈ 0.5) → expected mean 0.93, empirically observed.

```
def vsa_compose_chain(self, e_curr, e_prev):
    """Day 40 Pack 4: chain = bundle(E_t, E_{t-1}). Stores result; tracks coherence."""
    if e_curr is None or e_prev is None:
        return None
    chain = self.vsa_bundle([e_curr, e_prev])
    self.vsa_chain_event  = chain
    self.vsa_chain_sim   += (self.vsa_cosine(chain, e_curr) +
                             self.vsa_cosine(chain, e_prev)) / 2.0
    self.vsa_chain_count += 1
    return chain
```

Formula: Recovery: rec_A = bind(E, B) where E = bind(A, B). Similarity = cosine(rec_A, A). Perfect recovery ⟹ sim=1.0. Chain: C = MAJ(E_t, E_). Coherence = (cosine(C, E_t) + cosine(C, E_)) / 2. Prediction: coherence_expected = P(stable) × 1.0 + P(transition) × 0.5 = 0.86 + 0.07 = 0.93. **Hard Constants**: Majority bundling rule (threshold > half). Recovery test only called when both cid_vec and act_vec not None. Chain composition called every waking tick (Pack 4 harness). **Law Invariants**: Recovery similarity clamped [−1,+1]; chain coherence clamped [−1,+1]; all aggregates unbounded (infinite accumulation). **Waking Chain Placement**: Lines 34672–34676 in ikigai.py; vsa_test_recovery and vsa_compose_chain called sequentially after vsa_record_event, when enable_vsa=True. **Serialization Fields**: vsa_recover_act_sim, vsa_recover_cid_sim, vsa_recover_count, vsa_chain_event, vsa_chain_sim, vsa_chain_count. **EEIL Interpretation**: Validates symbolic computation (XOR self-inverse); chain coherence proves bundle preserves information (2-vector AND-bundle ≈ bitwise AND, preserving agreement bits); enables error-correction-free reasoning via hypervectors.

5.15 VSA Zero-Drift Invariant (Day 40 Core Requirement)

Behavioral Invariance: enable_vsa=True vs enable_vsa=False produces bit-identical action sequences, routing paths, and top_confidence values over 1500+ waking ticks (seeded, v1 validation). VSA system:

- Never writes to BG scoring variables (score_explore, score_approach, score_wait)
- Never modifies action_binding_gate.bound_action or routing decisions
- Never influences action_confidence_evaluator.top_confidence
- Operates only on vsa_* fields (observation-only, signal-only layers)

Empirical Signature (Day 40 Pack 1–4, 71/71 PASS):

- V1: Bit-identical (1447 waking ticks, seed=42, every tick atype|routing|TC match to 1e-9)
- V2: Recovery sim=1.0000 (1260 samples, zero variance)
- V3: Chain coherence 0.9293 (matches prediction 0.93 from regime stability distribution)
- V4: Spike rate 14.69% (target [5%, 30%], well-calibrated)
- V5: Cluster separation +0.948 (same-cluster sim +0.90, diff-cluster sim −0.05, near-perfect orthogonality)

Implementation Guarantee: All VSA code gated by `if _IKG.enable_vsa:` checks (lines 34659 onwards); hooks inject AFTER decision finalization (post-routing, post-action-commit); vsa_* aggregates never read by any behavioral path.

---

6. Mathematical Derivations

This mathematical appendix records the governing equations.

`AdrenalSystem` Equations:

- (blood half-life ~60–90 min → modelled as decay=0.998 per tick).
- Fix 3 (user review): lower_bound=0.02 — let CortisolSystem.apply_homeostasis()

AllostasisSystem Equations:

- - Cortisol in safe range + oxytocin = resilience recovery

VagalInteroceptionSystem Equations:

- - body_stress : integrated somatic burden = 0.6*cortisol + 0.4*heart_rate

HomeostasisSystem Equations:

- Deviation: D_i = x_i - x*_i
- Drive strength: d_i = |D_i| (clipped to [0,1])
- Global imbalance: E = Σ w_i * |x_i - x*_i|

ReplayBuffer Equations:

- sleep consolidation -- sample_for_replay(k=16) during SWS

EventCompressor Equations:

- 1. ticks are contiguous (gap <= 1)

SentenceGenerator Equations:

- High (>= 0.65) : "{A}, {B} {C}."
- Mid (>= 0.35) : "{A}. {B} {C}."

SleepConsolidator Equations:

- Sample replay buffer (salience-biased, k=16).
- Apply soft EMA tightening to centroid (alpha=0.98, very gentle).

NarrativeMemory Equations:

- 3. top ConceptGraph node unchanged (query_similar with k=1)
- flush_arc(reason="sleep_boundary") commits the arc at every sleep onset.
- ingest_tick() must NOT be called during sleeping==True ticks;

CognitivePlanner Equations:

- base = loop_risk * 0.50
- +0.25 if confidence_trend == 'destabilizing'
- -0.10 if confidence_trend == 'improving'

GoalExecutionBridge Equations:

- bias = plan.urgency × 0.70 + concept_support_norm × 0.30
- where concept_support_norm = support(target_focus) / max_support

TaskFramework Equations:

- 'high_priority_stabilization' (bias >= 0.80)
- 'focused_continuation' (bias >= 0.60)
- 'exploratory_probe' (bias >= 0.35)
- Override: 'research_expansion' (recent dominant_theme == 'discovery' AND bias >= 0.50)
- priority = (semantic_bias * 0.75) + (recent narrative confidence * 0.25)

ToolRouter Equations:

- route_confidence = (task_priority * 0.80) + (recent narrative continuity * 0.20)

ExecutionSandbox Equations:

- success_score = (route_confidence * 0.70) + (narrative continuity * 0.30)

ErrorReflector Equations:

- pressure = ((1 - success_score) * 0.70) + (task_priority * 0.30)
- If mismatch == 'aligned': pressure *= 0.25

RetryPlanner Equations:

- Retry planning layer: consumes mismatch reflection to formulate safe intent reiterations.
- confidence = (repair_pressure * 0.70) + ((1.0 - route_confidence) * 0.30)
- If mismatch == 'aligned': confidence *= 0.20

ProjectWorkspace Equations:

- Confidence = plan_confidence * 0.65 + progress_score * 0.35, clamped [0, 1].

ArtifactStateMemory Equations:

- Stability = workspace_confidence * 0.70 + improvement_score * 0.30, clamped [0, 1].

ConsistencyVerifier Equations:

- consistent : matching active_section + subgoal_state == 'progressing'
- stale : same artifact section + subgoal_state == 'stalled'
- drifted : subgoal_state == 'destabilized' + section mismatch (signature jump)
- Score = workspace_confidence * 0.60 + state_stability * 0.40, clamped [0, 1].

EditIntentGenerator Equations:

- Confidence = workspace_confidence * 0.70 + consistency_score * 0.30, clamped [0, 1].

PatchPreviewMemory Equations:

- preview_strength = old * 0.92 + new * 0.08 + 0.01
- repeat_count += 1
- Preview strength = state_stability * 0.60 + edit_confidence * 0.40, clamped [0, 1].

IntegrityScorer Equations:

- safe_patch : consistency_state == 'consistent' AND preview_strength > 0.70
- redundant_patch: consistency_state == 'stale' AND repeat_count > 1
- unsafe_patch : consistency_state == 'drifted'
- Score = preview_strength * 0.65 + consistency_score * 0.35, clamped [0, 1].

PatchExecutor Equations:

- Score = preview_strength * 0.70 + integrity_score * 0.30, clamped [0, 1].

VersionDiffTracker Equations:

- Score = execution_score * 0.65 + state_stability * 0.35, clamped [0, 1].
- diff_score = old * 0.90 + new * 0.10 + 0.01
- stagnation_count += 1

RollbackController Equations:

- rollback_to_last_stable: diff_state == 'protected' AND integrity_state == 'unsafe_patch'
- hold_previous : diff_state == 'protected' AND integrity_state != 'unsafe_patch'
- commit_forward : diff_state == 'advanced' AND integrity_state == 'safe_patch'
- Score = diff_score * 0.60 + integrity_score * 0.40, clamped [0, 1].

StructureMapBuilder Equations:

- map_confidence = min(1.0, region_count / 10.0) -- confident once 10+ regions found.

`DependencyLinkMemory` Equations:

- blast_radius = _BLAST_WEIGHTS.get(class_name, 0.30) * confidence, clamped [0, 1].
- confidence = mean edge weight across downstream_regions.

`RegionRewritePlanner` Equations:

- Risk score formula:
- risk = blast_radius * 0.60 + integrity_risk * 0.40 (clamped [0, 1])
- where integrity_risk = 1.0 - integrity_score.
- plan_confidence = guard_score * 0.60 + (1 - max_risk) * 0.40.

`InterfaceStabilityChecker` Equations:

- Score formula:

`CrossRegionCommitGraph` Equations:

- where source_region_slug = source_region[:20].replace(':', '_').replace(' ', '_')
- The most recent previous commit_id where success_score >= 0.60.
- If stability_state == 'interface_stable':
- new_score = old_score * 0.92 + raw_score * 0.08 + 0.02 (success)
- new_score = old_score * 0.95 - 0.03 (failure)
- raw_score = stability_score * 0.60 + (1 - max_risk) * 0.40.

`RewriteTrajectorySimulator` Equations:

- Prediction formulas:
- predicted_integrity = current_integrity * (1.0 - risk_score * 0.35)
- rollback_survivability = 1.0 - max(0.0, risk_score - 0.4)
- invariant_risk = risk_score * blast_radius * 0.5
- trajectory_confidence = (1 - max_invariant_risk) * 0.70 + current_integrity * 0.30

`InvariantDriftEstimator` Equations:

- Drift formula:
- drift = mean(1.0 - predicted_integrity across all trajectory nodes)
- localized_drift : 0.20 <= drift < 0.45 and no biology escalation
- rollback_protected : bool -- rollback survivability >= 0.60

`PreCommitOutcomeMemory` Equations:

- prediction_error = abs(actual_score - predicted_score)
- if prediction_error <= 0.10: (accurate prediction)
- future_bias = old_bias * 0.90 + prediction_error * 0.10 + 0.02
- future_bias = old_bias * 0.90 + prediction_error * 0.10 - 0.03

`FutureValueSelector` Equations:

- future_value = (

`BranchDeliberationConfidence` Equations:

- confidence = selected_branch_score * lineage_stability

`SelectedBranchIntentBridge` Equations:

- bridge_confidence = selected_branch_score * branch_deliberation_confidence * intent_schema_match

`SelectedBranchExecutionBridge` Equations:

- bridge_confidence >= 0.70
- audit_state == 'aligned'
- bridge_error <= 0.30
- rollback_survivability >= 0.65

- If selected branch carries biology escalation -> steering_bias = 0.0
- Steering bias formula (when gating passes and no biology):
- steering_bias = clamp(bridge_confidence * 0.35, 0.0, 0.35)

StrategicPolicyShaper Equations:

- failure_suppression -- EMA of steering_error (high = suppress future bias)
- Policy update law (EMA_ALPHA = 0.10):
- ema = old * 0.90 + new * 0.10
- policy_mature -- confidence >= 0.70
- policy_forming -- ema_success_rate >= 0.60

PolicyConditionedSubgoalRouter Equations:

- biology families receive zero routing (active_horizon=0, empty chain).
- if rollback_active: active_horizon -= 2, clamped to [2, 6]

ExecutionHorizonPlanner Equations:

- Max horizon = 6 enforced unconditionally (EEIL sparse compute law).

OutcomeCreditAssigner Equations:

- Family-level EMA statistics (alpha=0.10):
- compress : rollback_penalty > 0 or drift_penalty >= 0.20
- expand : family_reward >= 0.65 and net_credit >= 0.50

TemporalSequenceStabilityMonitor Equations:

- confidence into a smoothed stability_score via EMA (alpha=0.10).
- 'stable' : stability_score >= 0.70
- 'uncertain' : 0.50 <= stability_score < 0.70
- 'unstable' : 0.30 <= stability_score < 0.50

HorizonCompressionAdvisor Equations:

- safe_relaunch_tick = current_tick + 8 * max(1, |delta|)

FailurePatternShield Equations:

- A family becomes shielded when the same failure motif occurs >= 3 times
- 'rollback_heavy' : rollback_active == True
- 'divergence_heavy' : comparator_state == 'diverged'

CrossSessionPlanMemory Equations:

- Continuity score formula (clamped [0, 1]):
- goal_completion_prior = net_credit (OutcomeCreditAssigner)
- unfinished_priority_score = 1.0 if high, 0.50 if normal
- stability_score = TemporalSequenceStabilityMonitor EMA score
- failure_shield_safety = 1.0 - suppression_confidence (inverse shield)
- Updated every tick when continuity_score >= 0.60.

GoalThreadPersistence Equations:

- resume_confidence : EMA float [0, 1] (alpha=0.10)
- 'unfinished' : unfinished_priority == 'high' (from CrossSessionPlanMemory)
- 'complete' : net_credit >= 0.70 and continuity_score >= 0.60
- thread_priority formula (clamped [0, 1]):
- resume_confidence: EMA(old, thread_priority, alpha=0.10)

ContinuityDriftMonitor Equations:

- continuity score drops >= 0.20 within 8 ticks

ResumeConfidenceAdvisor Equations:

- Resume confidence formula (frozen):
- - resume_rank : int — rank among all threads (1 = best)

NarrativeCoherenceProbe Equations:

- - High if narrative_state == 'advancing' and few state transitions

GoalArbitrationEngine Equations:

- coherence, and dormant resurfacing bonus (+0.10 after >= 3 suppression

NarrativeConflictResolver Equations:

- ownership loops (>= 4 consecutive wins), and narrative provenance

PersistentIntentScheduler Equations:

- Schedules <= 4 intents with relaunch ticks and safe anchors.

SuppressedThreadArchive Equations:

- resurfacing eligibility (>= 3 suppression cycles).

DeferredResumptionGovernor Equations:

- from SuppressedThreadArchive, purges stale entries (>= 16 ticks), and

MilestoneMemorySynthesizer Equations:

- Salience formula per thread:

NarrativeArcMerger Equations:

- Merge convergent dormant arcs when identity overlap >= MERGE_THRESHOLD.

ProjectIdentityContinuityIndex Equations:

- Formula (frozen):
- identity_index =
- 'strong' : identity_index >= 0.70
- 'moderate' : identity_index >= 0.40

DormantMilestoneReviver Equations:

- suppression_count >= SUPPRESSION_THRESHOLD (3)
- initial_priority >= PRIORITY_FLOOR (0.40)
- revival_priority formula:

MilestoneDependencyGraph Equations:

- 'clear' : dependency_pressure == 0.0
- 'pressured' : dependency_pressure <= 0.50

BreakResumptionAnchor Equations:

- restart_priority formula:

ResumptionPriorityEngine Equations:

- If dependency_pressure >= 0.70, blocked milestones are promoted
- resume_confidence formula:

MilestoneBlockerResolver Equations:

- 'critical' : dependency_pressure >= 0.70
- 'unblocked' : dependency_pressure == 0.0

MomentumRecoveryPlanner Equations:

- recovery_confidence formula:

DormantFinalRevivalGate Equations:

- revival_viability formula:
- 'viable' : revival_viability >= 0.50
- 'low_viability' : revival_viability >= 0.20 and < 0.50

StaleNarrativePruner Equations:

- 3. suppression age >= 16 : starvation_duration >= STALE_THRESHOLD

WeekGapRestartSequence Equations:

- week_gap_confidence formula:

HypothesisPersistenceMemory Equations:

- salience formula per hypothesis:
- 'eeil_critical' : eeil_tagged and salience >= 0.60

OpenQuestionGraph Equations:

- A question with >= 2 milestone links is classified as a
- graph_density = link_count / max(1, node_count)
- critical_unknowns : list[str] -- question_ids with >= 2 links

EEILTensionTracker Equations:

- Tension score formula (frozen):
- tension_score =
- energy_tradeoff = 1.0 - resume_confidence
- continuity_risk = 1.0 - identity_index
- publication_value = milestone_salience
- Only hypotheses flagged eeil_tagged=True are evaluated.

ExperimentGapAnalyzer Equations:

- state. Gaps with gap_confidence >= 0.70 (Rule 3) surface into
- gap_confidence formula varies by gap_type:

PaperClaimContinuityBridge Equations:

- claim_confidence formula:
- publication_ready: confidence >= 0.60

PostBreakResearchCatalyst Equations:

- 1. Experiment gap with gap_confidence >= 0.70 -> rank_1 (forced)
- catalyst_confidence formula:

ResearchAgendaComposer Equations:

- (frozen law); PUBLICATION_BONUS=+0.10; at least one EEIL-tradeoff item.
- Scoring formula:
- item_score = base_score

ExperimentPriorityScheduler Equations:

- Priority formula:

- priority_score = agenda_rank_score * 0.50
- scheduler_confidence = agenda_confidence * 0.70 + gap_confidence * 0.30

ValidationSweepForecaster Equations:

- SWEEP_DEPTH_CAP = 6 — hard cap, never exceeded (Rule 3).
- forecast_base = agenda_confidence * 0.50
- step_conf[i] = max(0, forecast_base * (1 - 0.08 * i))

PaperRoadmapSynthesizer Equations:

- section_score = claim_confidence * 0.40
- next_claim_section: highest-scoring section with claim_confidence >= 0.30.

EEILTradeoffWorkbench Equations:

- Executes explicit EEIL tradeoff studies. Mandatory — always >= 1 active
- study_score = tension_score * 0.40
- eeil_gain_projection = mean(study_score) across all studies.

PostBreakExperimentLaunchpad Equations:

- launch_confidence = agenda_confidence * 0.35

WeekBreakFutureSimulator Equations:

- SIMULATION_HORIZON = 7 (frozen, Rule 1)
- Milestone strengthening law (Rule 2): salience >= 0.70 → +0.05
- simulation_confidence = clamp(

HypothesisDriftForecaster Equations:

- dormant hypotheses (0.30 <= salience < 0.50) may resurface.
- drift_pressure = fraction of hypotheses that decay

MilestoneSalienceProjector Equations:

- - Strengthening law (Rule 2): salience >= 0.70 → +0.05
- salience_delta = projected_mean - original_mean

AgendaReorderPredictor Equations:

- launch value delta >= 0.15 triggers a reorder (Rule 4).
- reorder_delta = max absolute delta seen across all items.

PaperClaimMaturationEstimator Equations:

- maturation_projection = claim_confidence * 0.40
- If maturation_projection >= 0.60: flagged 'likely_publication_ready' (Rule 5).
- maturation_projection (class-level) = mean across all claims.

PostHiatusValueConvergence Equations:

- convergence_confidence = maturation_projection * 0.30
- salience_delta_norm = clamp(salience_delta / 0.10) (normalised to [0,1])

ForecastReliabilityAssessor Equations:

- FRAGILE_CAP = 5 (Rule 1 -- no meta clutter)
- False confidence penalty (Rule 2): if simulation_confidence >= 0.80 AND
- fragility_score >= 0.60 → reliability_score -= 0.15 (clamped).
- reliability formula:
- raw_reliability = simulation_confidence * 0.35
- if sim_conf >= 0.80 and fragility_score >= 0.60:
- raw_reliability -= FALSE_CONFIDENCE_PENALTY (0.15)

- reliability = clamp(raw_reliability)

BreakStrategyMetaLearner Equations:

- law_score = forecast_reliability * 0.50 + convergence_confidence * 0.30
- strategy_delta = law_score - previous_law_score (0 on first tick).
- Learning: top law = max(law_score).

ContinuityFragilityMapper Equations:

- FRAGILE_CAP = 5 (Rule 1 joint cap).
- fragility_score = clamp(
- risk_reason: 'score=X fragile_count=Y'

MilestoneStrengthCauseAnalyzer Equations:

- cause_confidence = clamp(

AgendaAssumptionAuditor Equations:

- DEPENDENCY_SENSITIVITY_THRESHOLD = 0.70 → 'fragile_assumption' (Rule 4)
- dependency_sensitivity per item = proj_score * dep_press + (1 - agenda_conf)
- audit_confidence = forecast_reliability * 0.40

FutureHiatusPolicyRefiner Equations:

- policy_confidence = forecast_reliability * 0.30
- ideal_milestone_cap : int (6 if delta >= 0 else 5)

HiatusIdentityProjector Equations:

- identity_projection_conf = clamp(

NarrativeSelfEvolutionModel Equations:

- narrative_shift = projected_identity_conf - prev_identity_conf
- narrative_score = identity_projection_conf * 0.50

ScientificPersonaStabilityIndex Equations:

- Frozen formula:
- persona_stability = clamp(
- Rule 4: persona_stability >= 0.60 must be preserved.

MilestoneMeaningReweighter Equations:

- Rule 2: any milestone with projected_salience_delta >= 0.10
- receives meaning_weight_bonus = +0.05.
- base_weight = projected_salience (clamped)
- if delta >= 0.10: base_weight += MEANING_WEIGHT_BONUS (0.05)
- meaning_shift = projected_mean_weight - original_mean_weight
- 'selfhood_defining' : meaning_weight >= 0.75

CuriosityVectorRealigner Equations:

- top matured claim receive identity_priority_bonus = +0.10 (frozen).
- base_priority = hypothesis_salience (from forecast)
- if tied to rank_1 / launch / matured_claim: += PRIORITY_BONUS (0.10)
- realignment_delta = projected_mean_priority - base_mean_priority

PostBreakSelfContinuityAnchor Equations:

- anchor_confidence = persona_stability * 0.35

LongGapDoctrineSynthesizer Equations:

- REUSABLE_LAW_BONUS = +0.10 (Rule 2): law receives bonus if evidenced
- doctrine_law_score = base_score
- synthesis_confidence = clamp(

HiatusLawArchive Equations:

- Rule 2: reusable_law_bonus = +0.10 for laws evidenced in 2+ phases.
- law_reuse_score = clamp(

BreakOutcomePrincipleExtractor Equations:

- principle_confidence = clamp(

DoctrineConflictResolver Equations:

- Rule 3: any doctrine rule with conflict_score >= 0.50 → 'conditional_only'.
- conflict_score = clamp(
- conflict_class: 'conditional_only' if score >= 0.50, else 'global_law'.

FutureSilenceProtocol Equations:

- Rule 4: protocol_confidence >= 0.65 required before promotion.
- protocol_confidence = clamp(
- milestone_retention : 'strict' if synthesis_conf >= 0.70 else 'relaxed'
- promoted : bool (protocol_confidence >= 0.65)

InstitutionalContinuityKernel Equations:

- kernel_confidence = clamp(

SilenceDurationClassifier Equations:

- duration_confidence = clamp(kernel_confidence * 0.60 + reliability * 0.40)

TemporalContinuityScaler Equations:

- Scaling table (all ms_cap = 6, milestone identity preserved):
- micro_break : restart_depth=1, hyp_pres=0.95, breadth=5, strict=0.90
- short_gap : restart_depth=2, hyp_pres=0.90, breadth=5, strict=0.85
- week_gap : restart_depth=3, hyp_pres=0.80, breadth=4, strict=0.75
- long_gap : restart_depth=4, hyp_pres=0.70, breadth=4, strict=0.65
- semester_gap : restart_depth=5, hyp_pres=0.55, breadth=3, strict=0.55
- seasonal_hiatus: restart_depth=6, hyp_pres=0.40, breadth=3, strict=0.45
- scaling_delta = strictness - BASE_STRICTNESS (week_gap baseline = 0.75)

AdaptiveRestartAggression Equations:

- 1-3 (gentle) -> gentle_restart, score=0.15/0.25
- 7-14 (standard) -> normal_restart, score=0.50/0.60
- 30-90 (aggressive)-> blocker_first_aggressive, score=0.80/0.90

MilestoneRetentionCurve Equations:

- Rule 3: retention_floor >= 0.40 at all durations (identity collapse
- All values >= RETENTION_FLOOR (0.40).

HypothesisHalfLifeModel Equations:

- or critical dependency receives HALF_LIFE_BONUS = +0.15 added to its
- survival_pressure = clamp(surviving/total + half_life_bonus_if_tagged)

AgendaBreadthRegulator Equations:

- breadth_confidence = clamp(dur_conf * 0.60 + kernel_conf * 0.40)

`LongitudinalIdentityStabilizer` Equations:

- identity_stability = clamp(
- >= 0.70 -> 'low'
- >= 0.50 -> 'moderate'

`UniversalHiatusPolicyKernel` Equations:

- kernel_confidence = clamp(

`ProjectPivotClassifier` Equations:

- pivot_confidence = clamp(
- if conflict_class == 'conditional_only' -> domain_shift; default -> rename.

`DoctrineTransferMapper` Equations:

- Rule 2 (frozen floor): doctrine_transfer >= 0.60 always.
- transfer_confidence = clamp(
- conflict_penalty = 0.20 if pivot_type in {'domain_shift','research_fork'} else 0.05

`CrossDomainMilestoneTranslator` Equations:

- translation_integrity = clamp(

`HypothesisMigrationBridge` Equations:

- or critical experiment lineage receives migration_bonus = +0.15.
- migration_pressure = clamp(

`AgendaForkResolver` Equations:

- fork_conflict_score = clamp(
- If fork_conflict_score >= 0.50: dominant branch = 'blocker_first'
- else: dominant branch = 'catalyst_first'

`IdentityThreadPorter` Equations:

- port_confidence = clamp(

`ResearchLineagePreserver` Equations:

- lineage_strength = clamp(

`MetaProjectContinuityKernel` Equations:

- kernel_confidence = clamp(

`ThoughtChainConstructor` Equations:

- Constructs bounded reasoning chains. CHAIN_DEPTH_CAP = 6 (Rule 1 frozen).

`ConstraintReconciliationSolver` Equations:

- CONFLICT_THRESHOLD = 0.50 (Rule 2 frozen): branch conflict >= 0.50 must route

`HypothesisBranchPruner` Equations:

- BRANCH_FLOOR = 2 (Rule 3 frozen): no greedy collapse below 2 surviving branches.

`LongHorizonReasoningBridge` Equations:

- HORIZON_BONUS = +0.15 (Rule 4 frozen) for threads touching

`ExplanationSynthesisEngine` Equations:

- Rule 2 compliance: if conflict_score >= 0.50 routes through

ActiveCognitionKernel Equations:

- kernel_confidence = clamp(

DialogueTurnContinuity Equations:

- blocker carry-forward. TURN_FLOOR = 2 (Rule 3 frozen): at least 2 prior turns

CodeRationaleNarrator Equations:

- CLARITY_BONUS = +0.15 (Rule 4 frozen) for code_rationale packets.

ScientificExplanationFormatter Equations:

- and paper-grade summaries. CLARITY_BONUS = +0.15 (Rule 4 frozen).

LongFormResponsePlanner Equations:

- explanation. RESPONSE_PARA_CAP = 6 (Rule 1 frozen).

LanguageCognitionKernel Equations:

- kernel_confidence = clamp(

ClarificationQuestionPlanner Equations:

- Generates bounded clarification questions. CLARIFICATION_CAP = 3 (Rule 1 frozen).
- Rule 2: ambiguity_score >= 0.50 must route through this planner before

CollaborativeRevisionLoop Equations:

- constraints, and latest working plan. REVISION_FLOOR = 2 (Rule 3 frozen):

GoalNegotiationEngine Equations:

- NEGOTIATION_BONUS = +0.15 (Rule 4 frozen) for milestone_blocker, code_patch,

AmbiguityResolutionBridge Equations:

- AMBIGUITY_THRESHOLD = 0.50 (Rule 2 mirror): routes through
- ClarificationQuestionPlanner output when ambiguity_score >= 0.50.

PartneredPlanningSynthesizer Equations:

- active language kernel. PLAN_STEP_CAP = 6. NEGOTIATION_BONUS = +0.15

ConversationalAgencyKernel Equations:

- kernel_confidence = clamp(

CircadianSystem Equations:

- Sleep phase: phase > 0.65 (35% of cycle = wake-permitted window ≈ 8.4 h equivalent).
- Wake phase: phase ≤ 0.65 (65% of cycle = maximum sleep suppression window).

PredictiveSleepSystem Equations:

- adenosine_trend = EMA(adenosine) α=0.02 (slow)
- energy_trend = EMA(energy) α=0.02
- PSP = adenosine_trend + β × (1 - energy_trend)
- where β = 0.60 weights energy depletion as an amplifier of sleep pressure.

SpatialNavigationSystem Equations:

- `G_k(x,y) = cos(f·(x·cos(θ_k) + y·sin(θ_k) + φ_k))`
- `where f = spatial frequency, θ_k = orientation, φ_k = phase offset.`
- `PC_i(x,y) = exp(-((x-x_i)² + (y-y_i)²) / (2σ²))`
- `where σ = place field radius (default 2.5 units).`
- `Familiarity at current position = mean place cell activation.`
- `Novelty = 1 - familiarity.`
- `Step size = 0.35 units/tick, bounded to ±100 in each axis.`

`EpisodicReplaySystem` Equations:

- `2. During SWS: samples 5-15 state segments at p=0.05/tick.`
- `3. Applies emotional attenuation: replayed cortisol > 0.6 → fear_trace *= 0.95`
- `P_replay = α × sleep_depth (α = 0.05)`

`PlanningSystem` Equations:

- `V = w1 × C + w2 × R - w3 × S`
- `C = curiosity reward (spatial novelty of path)`
- `R = expected reward (from CognitiveMapSystem)`
- `S = predicted stress (cortisol at current state)`
- `w1 = 0.4, w2 = 0.4, w3 = 0.2`
- `explore_drive += best_path_value × 0.2 (bounded to +0.30 max)`

`CognitiveMapSystem` Equations:

- `Explored ratio = (visited nodes) / (total nodes within 10×10 grid)`
- `reward_estimate = EMA(valence, α=0.15)`

`CuriositySystem` Equations:

- `Information gain as intrinsic reward. Curiosity = novelty x safety.` Note: all variables adhere strictly to EMA clamping domains (typically 0.0 to 1.0) and conservation rules capping divergence.

---

7. Waking Chain Atlas

Map of the entire waking chain from tick start → `edit_intent_generator.generate()`. Every system shown executes once per tick inside `if not sleeping:` branch unless marked (S) = sleep branch only, or (B) = both branches.

Layer 1 — Neural Substrate (executed first)

1. **`Neuron.tick(inp, tick, ht_level, ne_level, sleeping)`**
   - Inputs: synaptic input sum, HT level (refractory scaling), NE level (threshold suppression)
   - Outputs: `fired: bool`, `voltage` (reset to 0 on spike), `spike_count` incremented
   - Signal flow: `fired` feeds Synapse.transmit() → used for all downstream neuromodulators
   - Laws: voltage reset on spike; threshold clamped [0.3, 1.5]; calcium decay ×0.95^3 during sleep
2. **`Synapse.transmit()` / `compute_eligibility()` / `apply_three_factor(da, ht, boost)`**
   - Inputs: `Neuron.fired`, `Synapse.ado_level` (class variable from AdenosineSystem)
   - Outputs: transmitted signal float; eligibility trace updated; `pending_weight_change` accumulated
   - Signal flow: transmitted signals feed next-layer neuron input; eligibility feeds 3-factor plasticity
   - Laws: transmit() waking + sleeping; apply_three_factor() waking; consolidate() (S) only

Layer 2 — Neuromodulators (called after all neuron ticks)

3. **`DopamineSystem.update(output_fired, tick, actual_reward)`**
   - Inputs: `output_fired` (BG output neuron), `actual_reward` (optional explicit signal)
   - Outputs: `da.level` ∈ [-0.25, 1.25]; `da.rpe` signed; `da.phasic`; `da.tonic`
   - Signal flow: `da.level` → Synapse.apply_three_factor(); `da.level` → CortisolSystem cortisol suppressor; `da.tonic` → OXT recovery
4. **`SerotoninSystem.update(total_spikes, max_possible)`**

- Inputs: total fired neuron count this tick, max possible
- Outputs: `serotonin.level` ∈ [0, 1]
- Signal flow: `serotonin.level` as `ht_level` → Synapse.apply_three_factor()

5. **`NorepinephrineSystem.update(PE_signal, cortisol, pain, amygdala_threat)`**

- Inputs: prediction error, cortisol.level, pain_sudden, amygdala BLA valence
- Outputs: `ne.level` ∈ [0, 1]; `ne.surprise: bool`
- Signal flow: `ne.level` → Neuron.tick() effective threshold (−0.25 × NE per neuron)

6. **`AcetylcholineSystem.update(novelty)`** / **`get_gain()`**

- Inputs: spatial or PE-based novelty signal [0, 1]
- Outputs: `ach.level`; `ach.get_gain()` = level × 0.3
- Signal flow: `ach.get_gain()` as boost → Synapse.apply_three_factor()

7. **`CortisolSystem.update(out_fired, ne_elev, tick, energy)` (B)**

- Inputs: output neuron fired, NE.elevated_ticks, tick, avg_energy
- Outputs: `cort.level` ∈ [0, 1]; `cort.chronic` counter; `cort.dmg`
- Signal flow: `cort.level` → HPAAxisSystem; → HomeostasisSystem safety drive; → DA.tonic suppression; → OXT inhibition
- Sleep branch: `cort.apply_homeostasis(tick)` circadian recovery; `cort.apply_atrophy(targets)` (S)

8. **`AdenosineSystem.update(cortical_spikes, energy, sleeping)` (B)**

- Inputs: total spike count, avg cortical energy, sleeping flag
- Outputs: `ado.level` ∈ [0, 1] written to `Synapse.ado_level` (global)
- Signal flow: global `Synapse.ado_level` → all Synapse transmit() fail_prob; → Neuron leak; → ACh suppression; → HomeostasisSystem sleep drive

9. **`OxytocinSystem.update(out_fired, da_level)`**

- Inputs: output fired, `da.level`
- Outputs: `oxt.level` ∈ [0, 1]; `oxt.trust: bool`
- Signal flow: `oxt.level` → CortisolSystem (`cort.level -= 0.02×oxt`); → DA tonic recovery; sleep (S): `apply_pruning()` near-zero weight removal

Layer 3 — HPA Axis Cascade (Day 14)

10. **`HypothalamusSystem.update(...)`**

- Inputs: amygdala_threat, PE, metabolic_stress, pain, hippo_inhibition, pfc_regulation, adenosine, current_cortisol
- Outputs: `crh` ∈ [0, 1]

11. **`PituitarySystem.update(crh)`**

- Inputs: `crh` from HypothalamusSystem
- Outputs: `acth` ∈ [0, 1] (5–10 tick integration lag)

12. **`AdrenalSystem.update(acth, sleeping, oxytocin)`**

- Inputs: `acth`, sleeping flag, oxytocin
- Outputs: cortisol contribution blended 8% per tick into `cort.level`

13. **`HPAAxisSystem.step(...)`** — orchestrator

- Calls all three HPA components in sequence; blends output into `CortisolSystem.level`
- Outputs: logged `crh`, `acth`, `cortisol`, `hippo_inhibition`, `pfc_regulation`

Layer 4 — Interoception & Self-Model

14. **`SelfModelSystem`**
- Inputs: cortisol trajectory, energy trajectory, valence transitions
- Outputs: predicted future cortisol; `prediction_error_self` ∈ [0, 1]; `regulation_confidence`
- Signal flow: self-model PE contributes to arousal system PE blend (Day 18)

Layer 5 — Allostasis & Interoception

15. **`AllostasisSystem`**

- Inputs: `cort.level`, `oxt.level`, `avg_energy`, sleeping flag

- Outputs: updated setpoints for HomeostasisSystem; `allostatic_load`; `resilience`
- Laws: setpoint changes propagate to HomeostasisSystem only; zero survival contamination
- Signal flow: `load` → CortisolSystem setpoint drift; sleep recovery: `load *= 0.97`

16. **`VagalInteroceptionSystem`**

- Inputs: cortisol, heart_rate (derived), sleeping
- Outputs: `vagal_tone` ∈ [0,1]; `body_stress = 0.6×cort + 0.4×heart_rate`
- Signal flow: `vagal_tone` suppresses `cort.level` (proportional); `body_stress` amplifies BLA threat

17. **`HomeostasisSystem`**

- Inputs: avg_energy, cortisol, oxytocin, adenosine, prediction_error; circadian from CircadianSystem
- Outputs: drives dict {hunger, safety, social, sleep, curiosity}; `global_imbalance`; sleep onset/end signals
- Signal flow: drives → BasalGangliaSystem action bias; sleep signals → sleeping flag

Layer 6 — Cognition Substrate (waking only)

18. **`LatentStateVector`** (64-dim)

- Inputs: cortisol, NE, DA, avg_energy, prediction_error, oxytocin, adenosine, spike activity
- Outputs: 64-element float vector snapshot; `snapshot_vector()`; `decay(0.001)` during sleep
- Signal flow: snapshot → ReportBus, ReplayBuffer, ActionReasoningLog

19. **`ActionReasoningLog`**

- Inputs: latent_pre snapshot, selected_action, reason_stage (wm/explore/floor), energy_delta, cortisol_delta
- Outputs: deque(maxlen=1000) of tick records; `latest()` → NarrativeMemory, PlanningSystem

20. **`ReplayBuffer`**

- Inputs: high-salience transitions from ActionReasoningLog (|energy_delta|+|cortisol_delta| biased)
- Outputs: deque(maxlen=2000); `sample_for_replay(k=16)` during SWS → SleepConsolidator

21. **`EventCompressor`**

- Inputs: ReplayBuffer transitions; cosine similarity of consecutive latent vectors
- Outputs: compressed event episodes (start_tick, end_tick, dominant_action, peak_salience)
- Laws: contiguous ticks AND cosine_sim>0.90 AND same action family to merge; flush on sleep onset

22. **`ConceptGraph`**

- Inputs: EventCompressor compressed events; cosine similarity to existing node centroids
- Outputs: directed concept graph; `query_similar(latent, k)` → NarrativeMemory focus
- Laws: merge if cosine>similarity_threshold + matching action family; evict lowest support×recency on overflow

23. **`ReportBus`**

- Inputs: LatentStateVector, ConceptGraph, ActionReasoningLog each tick
- Outputs: scene snapshot dict (latent_focus top-8 dims, top_concepts, last_action, workspace_confidence)
- Signal flow: scene → LanguageReadout → SentenceGenerator → NarrativeMemory chain

24. **`LanguageReadout`**

- Inputs: ReportBus scene snapshot (latent_focus dims, state polarity inference)
- Outputs: proposition dict {state_polarity, action_intent, causal_basis, confidence}
- Polarity rules: mean(latent_focus)>0.60→stress; <0.40→recovery; spread>0.25→explore; else→neutral

25. **`SentenceGenerator`**

- Inputs: LanguageReadout proposition; confidence tier (low<0.35 / mid<0.65 / high)
- Outputs: grounded English sentence; tick-rotated variant selection prevents repetition

26. **`SleepConsolidator`** (S — sleep branch only)

- Inputs: ReplayBuffer (k=16 samples), EventCompressor events, ConceptGraph, SentenceGenerator utterances
- Outputs: reinforced concept nodes (+support); reinforced edges; confidence calibration record
- Laws: Bounded deques, zero-drift isolated observer updates.

27. `NarrativeMemory`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions

- Laws: Bounded deques, zero-drift isolated observer updates.

28. `ReflectiveReasoner`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

29. `CognitivePlanner`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

30. `GoalExecutionBridge`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

31. `TaskFramework`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

32. `ToolRouter`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

33. `ExecutionSandbox`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

34. `ErrorReflector`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

35. `RetryPlanner`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

36. `RouteMutator`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

37. `MutationGuard`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

38. `RetryOutcomeTracker`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

39. `StrategyLearner`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

40. `PolicyShaper`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

41. `FailureAtlas`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

42. `AbstractTaskEngine`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

43. `PlanGraphMemory`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

44. `SubgoalEvaluator`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

45. `ProjectWorkspace`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

46. `ArtifactStateMemory`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

47. `ConsistencyVerifier`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

48. `EditIntentGenerator`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

49. `PatchPreviewMemory`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

50. `IntegrityScorer`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

51. `PatchExecutor`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

52. `VersionDiffTracker`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions

- Laws: Bounded deques, zero-drift isolated observer updates.

53. `RollbackController`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

54. `StructureMapBuilder`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

55. `DependencyLinkMemory`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

56. `SemanticRegionGuard`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

57. `RegionRewritePlanner`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

58. `InterfaceStabilityChecker`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

59. `CrossRegionCommitGraph`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

60. `RewriteTrajectorySimulator`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

61. `InvariantDriftEstimator`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

62. `PreCommitOutcomeMemory`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

63. `TrajectoryBranchGenerator`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

64. `FutureValueSelector`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

65. `BranchDeliberationConfidence`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

66. `BranchReasoningTrace`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

67. `BranchOutcomeArchive`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

68. `SelectedBranchIntentBridge`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

69. `IntentConsistencyAuditor`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

70. `BridgeOutcomeMemory`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

71. `SelectedBranchExecutionBridge`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

72. `ExecutionOutcomeComparator`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

73. `StrategicPolicyShaper`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

74. `PolicyConditionedSubgoalRouter`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

75. `ExecutionHorizonPlanner`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

76. `OutcomeCreditAssigner`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions
- Laws: Bounded deques, zero-drift isolated observer updates.

77. `TemporalSequenceStabilityMonitor`

- Inputs: Neural & Drive Tensors
- Outputs: Serialized constraints & actions

- Laws: Bounded deques, zero-drift isolated observer updates.

78. `HorizonCompressionAdvisor`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

79. `FailurePatternShield`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

80. `CrossSessionPlanMemory`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

81. `GoalThreadPersistence`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

82. `NarrativeProjectState`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

83. `ContinuityDriftMonitor`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

84. `ResumeConfidenceAdvisor`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

85. `NarrativeCoherenceProbe`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

86. `GoalArbitrationEngine`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

87. `NarrativeConflictResolver`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

88. `PersistentIntentScheduler`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

89. `SuppressedThreadArchive`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

90. `DeferredResumptionGovernor`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

91. `MilestoneMemorySynthesizer`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

92. `NarrativeArcMerger`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

93. `ProjectIdentityContinuityIndex`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

94. `DormantMilestoneReviver`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

95. `MilestoneDependencyGraph`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

96. `BreakResumptionAnchor`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

97. `ResumptionPriorityEngine`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

98. `MilestoneBlockerResolver`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

99. `MomentumRecoveryPlanner`

    - Inputs: Neural & Drive Tensors
    - Outputs: Serialized constraints & actions
    - Laws: Bounded deques, zero-drift isolated observer updates.

100. `DormantFinalRevivalGate`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

101. `StaleNarrativePruner`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

102. `WeekGapRestartSequence`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

103. `HypothesisPersistenceMemory`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

104. `OpenQuestionGraph`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

105. `EEILTensionTracker`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

106. `ExperimentGapAnalyzer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

107. `PaperClaimContinuityBridge`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

108. `PostBreakResearchCatalyst`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

109. `ResearchAgendaComposer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

110. `ExperimentPriorityScheduler`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

111. `ValidationSweepForecaster`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

112. `PaperRoadmapSynthesizer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

113. `EEILTradeoffWorkbench`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

114. `PostBreakExperimentLaunchpad`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

115. `WeekBreakFutureSimulator`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

116. `HypothesisDriftForecaster`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

117. `MilestoneSalienceProjector`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

118. `AgendaReorderPredictor`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

119. `PaperClaimMaturationEstimator`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

120. `PostHiatusValueConvergence`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

121. `ForecastReliabilityAssessor`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

122. `BreakStrategyMetaLearner`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

123. `ContinuityFragilityMapper`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

124. `MilestoneStrengthCauseAnalyzer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

125. `AgendaAssumptionAuditor`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

126. `FutureHiatusPolicyRefiner`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

127. `HiatusIdentityProjector`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

128. `NarrativeSelfEvolutionModel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

129. `ScientificPersonaStabilityIndex`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

130. `MilestoneMeaningReweighter`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

131. `CuriosityVectorRealigner`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

132. `PostBreakSelfContinuityAnchor`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

133. `LongGapDoctrineSynthesizer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

134. `HiatusLawArchive`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

135. `BreakOutcomePrincipleExtractor`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

136. `DoctrineConflictResolver`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

137. `FutureSilenceProtocol`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

138. `InstitutionalContinuityKernel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

139. `SilenceDurationClassifier`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

140. `TemporalContinuityScaler`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

141. `AdaptiveRestartAggression`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

142. `MilestoneRetentionCurve`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

143. `HypothesisHalfLifeModel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

144. `AgendaBreadthRegulator`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

145. `LongitudinalIdentityStabilizer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

146. `UniversalHiatusPolicyKernel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

147. `ProjectPivotClassifier`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

148. `DoctrineTransferMapper`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

149. `CrossDomainMilestoneTranslator`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

150. `HypothesisMigrationBridge`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

151. `AgendaForkResolver`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

152. `IdentityThreadPorter`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

153. `ResearchLineagePreserver`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

154. `MetaProjectContinuityKernel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

155. `DeliberationWorkspace`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

156. `ThoughtChainConstructor`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

157. `MultiHopInferenceEngine`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

158. `ConstraintReconciliationSolver`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

159. `HypothesisBranchPruner`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

160. `LongHorizonReasoningBridge`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

161. `ExplanationSynthesisEngine`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

162. `ActiveCognitionKernel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

163. `ReasoningParagraphComposer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

164. `InstructionGroundingBridge`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

165. `DialogueTurnContinuity`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

166. `CodeRationaleNarrator`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

167. `ScientificExplanationFormatter`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

168. `LongFormResponsePlanner`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

169. `NarrativeCoherenceRegulator`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

170. `LanguageCognitionKernel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

171. `IntentEvolutionTracker`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

172. `ClarificationQuestionPlanner`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

173. `CollaborativeRevisionLoop`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

174. `GoalNegotiationEngine`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

175. `AmbiguityResolutionBridge`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

176. `TurnLevelMemoryBinder`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

177. `PartneredPlanningSynthesizer`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

178. `ConversationalAgencyKernel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

179. `DevelopmentMetrics`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

180. `CircadianSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

181. `PredictiveSleepSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

182. `SpatialNavigationSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

183. `EpisodicReplaySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

184. `PlanningSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

185. `CognitiveMapSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

186. `AmygdalaSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

187. `SomaticMarkerSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

188. `HippocampusSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

189. `ThalamusSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

190. `SleepStateManager`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

191. `RegionalActivityTracker`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

192. `EIBalanceTracker`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

193. `CriticalPeriodSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

194. `CellAssemblySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

195. `MirrorNeuronSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

196. `BridgeNeuronSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

197. `SemanticEmergenceSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

198. `InternalSpeechSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

199. `NarrativeSelfSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

200. `WorkingMemorySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

201. `PredictiveProcessingSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

202. `SensoryEnvironment`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

203. `SensoryIntegrationSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

204. `ExecutiveFunctionSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

205. `ConflictResolutionSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

206. `GrammarEmergenceSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

207. `LanguageMemorySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

208. `OtherEntitySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

209. `SocialAwarenessSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

210. `AttachmentSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

211. `EmpathySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

212. `TheoryOfMindSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

213. `PersistenceSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

214. `Population`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

215. `PredictionMatrix`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

216. `CorticalColumn`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

217. `MultiColumnCortex`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

218. `ActionChannel`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

219. `BasalGangliaSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

220. `Microcircuit`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

221. `CompassionSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

222. `GratitudeSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

223. `SelfCompassionSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

224. `MissionSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

225. `CuriositySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

226. `EmotionalRegulationSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

227. `MetacognitionSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

228. `LearningAwarenessSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

229. `SelfImprovementAwareness`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

230. `DreamSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

231. `EpisodicMemorySystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

232. `AutobiographicalRetrievalSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

233. `TemporalSelfSystem`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

234. `SleepSystem_L18`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

235. `SystemRigimeTracker`

- Inputs: Neural & Drive Tensors - Outputs: Serialized constraints & actions - Laws: Bounded deques, zero-drift isolated observer updates.

---

8. EEIL Theoretical Interpretation

Energy Efficiency Intelligence Law (EEIL)

EEIL states that sustainable intelligence must minimize total free energy expenditure across all cognitive operations while maintaining EES (Energy-Ecological-Social) alignment. All architectural decisions in ikigai are constrained by this law.

Seven EEIL Mechanisms in ikigai

Mechanism	Implementation	Biological Parallel
Sparse Reasoning Caps	SWEEP_DEPTH_CAP=6, CHAIN_DEPTH_CAP=6, HORIZON_CAP=6	Prefrontal working memory limit (Miller 1956)
Sleep-Gated Plasticity	Synapse.consolidate() sleeping branch only	SHY hypothesis (Tononi 2006) — sleep renormalization
Adenosine Budget	ado.level accumulates; gates STDP, fail_prob, ACh	Process S (Borbely 1982) metabolic debt
Homeostatic Drive Minimization	global_imbalance = mean(drives); BG selection toward drive reduction	Allostasis (Sterling & Eyer 1988)
Biology Exclusion Law	Zero routing for {Neuron, Synapse, HomeostasisSystem, HPAAxisSystem, cortisol, adenosine}	Structural conservation: rewiring the substrate from within destabilizes the system
Bounded Deques	All cognitive observers: deque(maxlen=96-128); no unbounded memory	Memory capacity limits prevent catastrophic interference (McCloskey & Cohen 1989)
Energy-EES Coupling	`avg_energy < 0.5` accelerates adenosine; `energy < 0.4` suppresses cortisol multiplicatively	Metabolic state gates behavior — starvation forces rest

EEIL Experimental Evidence (from NeuroSeed experiments)

Experiment	EEIL Prediction	Result
Day 016C/D	Sleep reduces PE	No-sleep PE=0.1059 vs with-sleep PE=0.005 — **21× gap**
Day 017	r(energy, hunger) negative	r=-0.9483 PASS
Day 019	SHY: PE_post > PE_pre at wake onset	PE_post=0.040 > PE_pre=0.014 PASS
EEIL Phase A-D	Regulation+alignment > alignment alone	A>B>D>C ordering; A3 monoculture (approach 94%, entropy<0.32)
Phase E	Sleep rescues A3 energy	A3: S1 E=0.197 → S3 E=0.628 after sleep regimes
Exp X	Regulation entropy > alignment-only	R1 entropy=0.829 > R2 entropy=0.486; R2→86.6% approach degeneration

EEIL Formal Statement

```
EEIL = structure (alignment + regulation)
Learning = amplifier (magnifies both good and bad structure)

Degeneration requires ALL THREE:
  1. alignment (without regulation)
  2. absence of regulation (no entropy floor)
  3. learning dynamics (amplification)

Absence of any one factor prevents collapse.
```

---

9. Empirical Validation Index

Core Biological Invariants (Experiment-Validated)

Invariant	Test	Value	Status
r(energy, hunger)	Day 017	-0.9483	PASS
r(hunger, DA)	Day 017	+0.8095	PASS
r(cortisol, DA)	Day 017	-0.1032	PASS
Sleep rate SWS	Day 017	90-92%	PASS
OWC sleep fraction	Day 019.5	34.2%	PASS
SHY PE ratio	Day 019	post/pre = 2.86×	PASS
DA shock response	Day 019D	-9.4% arousal 7.4×	PASS
Phase E A-invariance	Phase E	entropy range=0.028 across S1/S2/S3	PASS
Exp X regulation entropy	Exp X	+0.343 bits over alignment-only	PASS
Exp Y structural effect	Exp Y	+0.002 EES, +0.027 bits (bounded)	PASS

Blast-Radius Registry (from `_BLAST_WEIGHTS`)

Protected classes with blast_radius ≥ 0.80 (SemanticRegionGuard blocks edits):

Class	Blast Radius	Protection Reason
`HomeostasisSystem`	0.95	All drives, sleep gate, BG input
`HPAAxisSystem`	0.90	Full cortisol cascade
`Neuron`	0.90	Every spike in every column
`Synapse`	0.90	All synaptic transmission + plasticity
`CortisolSystem`	0.85	Chronic stress, atrophy, circadian
`AdenosineSystem`	0.80	Global Synapse.ado_level gating

Serialization Completeness

All ~235 classes implement `to_dict()` / `from_dict()` with bounded deque reconstruction. State round-trips verified. Zero filesystem writes from cognitive observers (all internal artifact cognition is symbolic only).

---

10. Scientific References + Internal Citations

Primary Biological References

1. **Borbely (1982)** Two-Process Sleep Model (Process S + C). → AdenosineSystem, CircadianSystem, HomeostasisSystem sleep gate.
2. **Cannon (1932) / Sterling & Eyer (1988)** Homeostasis / Allostasis. → HomeostasisSystem drives, AllostasisSystem setpoint drift.
3. **Turrigiano (1998, 2008)** Homeostatic synaptic plasticity. → Neuron threshold adaptation (eta=0.001/N_scale), Synapse anti-saturation.
4. **Schultz (1997) / Tobler et al. (2005)** Reward prediction error dopamine. → DopamineSystem signed RPE, tonic/phasic separation.
5. **McEwen (1998)** Allostatic load, hippocampal GR feedback. → CortisolSystem GR feedback, AllostasisSystem.
6. **Herman et al. (2003)** HPA axis afferent pathways. → HypothalamusSystem drive weights (BLA=0.45).
7. **Aston-Jones & Cohen (2005)** LC-NE Adaptive Gain Theory. → NorepinephrineSystem multi-source weights.
8. **Neumann (2002)** Oxytocin HPA inhibition. → OxytocinSystem cort suppression.
9. **Tononi (2006)** Synaptic Homeostasis Hypothesis (SHY). → AdenosineSystem-gated STDP, sleep-only consolidation.
10. **Friston (2010)** Active Inference / Free Energy Principle. → PredictiveProcessingSystem, SelfModelSystem.
11. **Carandini & Heeger (2012)** Divisive normalization. → Synapse.transmit() pop_norm.
12. **Markram (2015)** Population-scaled inhibition. → Synapse inhibitory pop_scale.
13. **Fields (2008)** Myelination from activity. → Synapse myelination at 100/200 uses.
14. **Faisal et al. (2008)** Neural noise / stochasticity. → `random.seed()` unseeded Day 20; Neuron intrinsic noise σ=0.005.
15. **Gurney et al. (2001)** Basal ganglia direct/indirect pathway. → Neuron motor competition (-0.08 suppression).

Internal Architecture Citations

16. **EEIL Theorem 1** (Prince Siddhpara, NeuroSeed 2026). Energy Efficiency Intelligence Law — alignment + regulation + learning = full cognitive integrity.
17. **Day 19.5/19.6** Sleep-never-end bug fix (ado clearance + additive Borbely + foraging floor).
18. **Day 20** True Agency Test — 5/5 PASS; approach floor 2% random (Turrigiano variability + Friston EFE).
19. **Phase E** Sleep regime experiments — A-invariance across S1/S2/S3; A3 depletion reversal.
20. **Exp X** Q-learning EEIL validation — R1 (aligned+regulated) > R2 (aligned) degeneracy confirmed.
21. **Exp Y** Rule-based EEIL validation — structural effect bounded without learning amplifier; V4 FAIL expected.

---

11. Appendix: Key Code Snippets from ikigai.py

This appendix provides verbatim excerpts of the most architecturally significant sections of `ikigai.py` for direct reference.

A. Main Tick Loop — Sleep Transition Control (lines ~26792–26900)

```
for local_tick in range(TICKS):
    tick = total_ticks + local_tick
    circadian.update()
    _circadian_signal = math.sin((tick + _CIRC_OFFSET) * (2 * math.pi / CIRCADIAN_PERIOD))
    _arousal_signal = max(0.0, _arousal_signal * 0.95)  # noradrenergic clearance tau~14
    homeostasis._arousal_override = _arousal_signal > 0.30

    # -- Sleep transition (Borbely Process S) --
    if homeostasis.should_sleep_onset(circadian):
        sleeping = True
        homeostasis.mark_sleep_start(); predictive_sleep.mark_sleep_start()
        episodic_replay.reset_sleep_counter()
        event_compressor.flush_current_event(reason='sleep_onset')
        report_bus.publish(tick=tick, ..., sleep_onset=True)
        narrative_memory.flush_arc(reason='sleep_boundary', ...)

    elif homeostasis.should_sleep_end() and sleeping:
        sleeping = False
        homeostasis.mark_sleep_end(); predictive_sleep.mark_sleep_end()
        ne.level = 0.4; ne.surprise = True  # wake-up arousal reset
        dream_sys.apply_waking_effects(semantic, curiosity_sys, conflict)

    # -- Sleep branch --
    if sleeping:
        for k in l23.energy: l23.energy[k] = min(1.0, l23.energy[k] + 0.01)
        ado.level = max(0.0, ado.level * 0.98)   # adenosine clearance
        Synapse.ado_level = ado.level
        latent_state_vec.decay(0.001)             # NREM slow forgetting
        if ado.level >= 0.20:
            homeostasis.drives['sleep'] = 0.50    # sleep maintained — Process S not cleared
        else:
            homeostasis.drives['sleep'] = 0.0     # adenosine cleared → wake next tick
        sleep_consolidator.consolidate(tick=tick, replay_buffer=replay_buffer, ...)
        reflective_reasoner.reflect(tick=tick, ...)
        cognitive_planner.plan(tick=tick, ...)
        [for synapse in synapses: synapse.consolidate()]  # weight updates committed
```

B. DopamineSystem — Signed RPE Core (lines ~323–358)

```
def update(self, output_fired, tick=0, actual_reward=None):
    reward = self._clip(float(actual_reward if actual_reward is not None else
                              (1.0 if bool(output_fired) else 0.0)), -1.0, 1.0)
    self.rpe = reward - self.expected
    # Adaptive predictor — faster under uncertainty
    self.uncertainty += self.k_uncertainty * (abs(self.rpe) - self.uncertainty)
    pred_lr = self.pred_lr_min + (self.pred_lr_max - self.pred_lr_min) * min(1.0, self.uncertainty)
    self.expected += pred_lr * self.rpe
    # Phasic burst/dip
    signed_impulse = math.tanh(self.rpe / self.rpe_scale)
    headroom = 1.0 / (1.0 + abs(self.phasic))
    self.phasic = (self.phasic * self.k_phasic) + (self.rpe_gain * signed_impulse * headroom)
    # Tonic climate
    self.reward_ema += self.k_reward * (reward - self.reward_ema)
    self.tonic_target = self.setpoint + self.tonic_reward_gain * (self.reward_ema - 0.5)
    self.tonic += self.k_tonic * (self.tonic_target - self.tonic)
    if hasattr(self, 'cortisol_level'):
        self.tonic *= (1.0 - 0.15 * self.cortisol_level)   # glucocorticoid suppression
    if hasattr(self, 'oxytocin_level') and hasattr(self, 'cortisol_level'):
        self.tonic += 0.02 * self.oxytocin_level * (1.0 - self.cortisol_level)  # VTA disinhibition
    self._refresh_level()
```

C. HypothalamusSystem.update — CRH Drive Cascade (lines ~658–710)

```
def update(self, amygdala_threat, prediction_error, metabolic_stress, pain_aversive,
           hippocampal_inhibition=0.0, pfc_regulation=0.0, adenosine_level=0.0, current_cortisol=0.0):
    self.sensitivity = min(1.25, 1.0 + 0.4 * max(0.0, adenosine_level - 0.5))
    threat_signal = (
        0.45 * max(0.0, amygdala_threat) + 0.25 * prediction_error +
        0.20 * metabolic_stress          + 0.10 * pain_aversive
    ) * self.sensitivity
    threat_signal -= hippocampal_inhibition * 0.30   # GR-mediated negative feedback
    threat_signal -= pfc_regulation        * 0.20   # top-down reappraisal
    threat_signal -= current_cortisol      * 0.25   # ultra-short cortisol feedback (Dallman 1984)
    raw_crh = 1.0 / (1.0 + math.exp(-(threat_signal * 4.0 - 2.0)))
    raw_crh += random.gauss(0.0, 0.008)
    if raw_crh > self.crh: self.crh = 0.70 * self.crh + 0.30 * raw_crh   # fast rise
    else:                  self.crh = 0.97 * self.crh + 0.03 * raw_crh  # slow decay
```

D. CortisolSystem — Circadian Homeostasis (lines ~551–571)

```
def apply_homeostasis(self, tick=0):
    baseline = 0.15 + 0.08 * math.sin(tick * 0.002)   # circadian (Pruessner 1997)
    decay_rate = 0.08
    self.level += decay_rate * (baseline - self.level)
    # Hippocampal GR negative feedback (McEwen 1998) — sub-acute range only
    if self.setpoint < self.level < 0.6:
        self.level += -0.02 * (self.level - self.setpoint)
    # Recovery guarantee post-stress — continuous exponential relaxation
    elif self.level >= 0.6:
        self.level += -0.015 * (self.level - self.setpoint)
    # Ultra-slow allostatic setpoint correction toward evolutionary baseline
    self.setpoint += 1e-4 * (0.15 - self.setpoint)
    self.level = max(0.0, min(1.0, self.level))
```

E. Neuron — Homeostatic Threshold Plasticity (lines ~137–155)

```
# Fix 1: Intrinsic homeostatic plasticity (Turrigiano 1998)
# Neurons track their own firing rate and adapt threshold to maintain ~10% target
if '-Ih' not in self.name and not self.is_inhibitory:
    target_rate = 0.10
    _scale = getattr(Synapse, '_N_scale', 1.0)
    eta = 0.0010 / _scale    # scales with 1/sqrt(N) for network-size stability
    self.avg_rate = 0.95 * self.avg_rate + 0.05 * (1.0 if self.fired else 0.0)
    self.threshold += eta * (self.avg_rate - target_rate)
    # Relax toward baseline: alpha=0.0003 — critically damped oscillation suppression
    self.threshold += 0.0003 * (self.base_threshold - self.threshold)
    self.threshold = min(max(self.threshold, 0.3), 1.5)
```

F. AdenosineSystem — Borbely Process S (lines ~578–592)

```
class AdenosineSystem:
    def __init__(self):
        self.level = 0.0
    def update(self, cortical_spikes, cortical_energy, sleeping=False):
        if sleeping:
            self.level *= 0.95   # exponential SWS clearance
        else:
            inc = cortical_spikes * 0.0004
            if cortical_energy < 0.5:
                inc *= 1.3       # metabolic depletion accelerates sleep pressure
            self.level += inc
        self.level = max(0.0, min(1.0, self.level))
        Synapse.ado_level = self.level   # global synaptic gating signal
```

G. Blast-Radius Registry (_BLAST_WEIGHTS, lines ~6401–6429)

```
_BLAST_WEIGHTS = {
    'HomeostasisSystem':   0.95,  # all drives, sleep gate, BG input
    'HPAAxisSystem':       0.90,  # full cortisol cascade
    'Neuron':              0.90,  # every spike in every column
    'Synapse':             0.90,  # all transmission + plasticity
    'CortisolSystem':      0.85,  # chronic stress + atrophy
    'AdenosineSystem':     0.80,  # global Synapse.ado_level gating
    'ReportBus':           0.75,  # scene snapshot consumers
    'ConceptGraph':        0.75,  # learnable concept nodes
    'LatentStateVector':   0.70,  # 64-dim interoceptive context
    'ActionReasoningLog':  0.65,  # causal trace
    'ReplayBuffer':        0.65,  # salience-biased episodes
    'NarrativeMemory':     0.60,  # autobiographical arcs
    'SentenceGenerator':   0.60,  # language surface
    # ... (lower coupling classes truncated)
}
```

[Internal Class Index / Formula Index — Listed within Sections 4 and 5]